



大语言模型及应用

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr

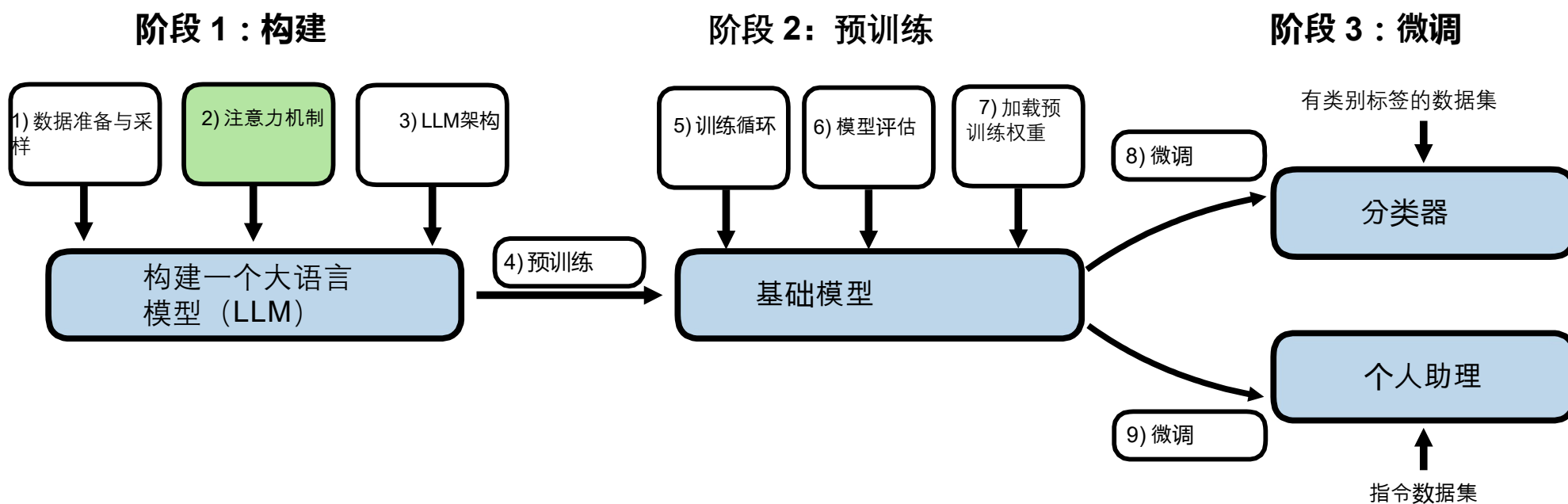


Table of contents

1. The Transformer Architecture
2. Pre-training and Fine-tuning
3. Transformer Applications
4. Case study - Large Language Models



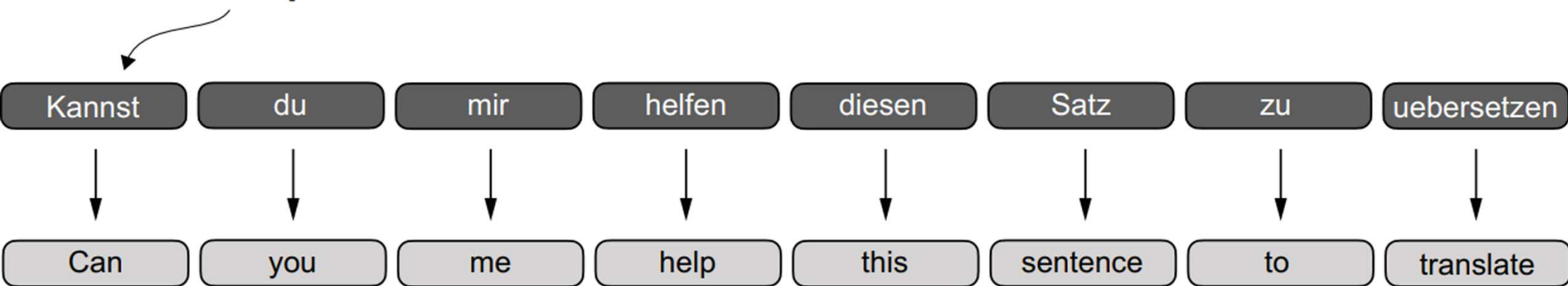
注意力机制 (1)



The problem with modeling long sequences

- 翻译模型遇到的问题
 - 逐字翻译导致语法错误

German input sentence to translate



The word-by-word translation results in a grammatically incorrect sentence

The problem with modeling long sequences

- 翻译模型遇到的问题
 - 正确的翻译



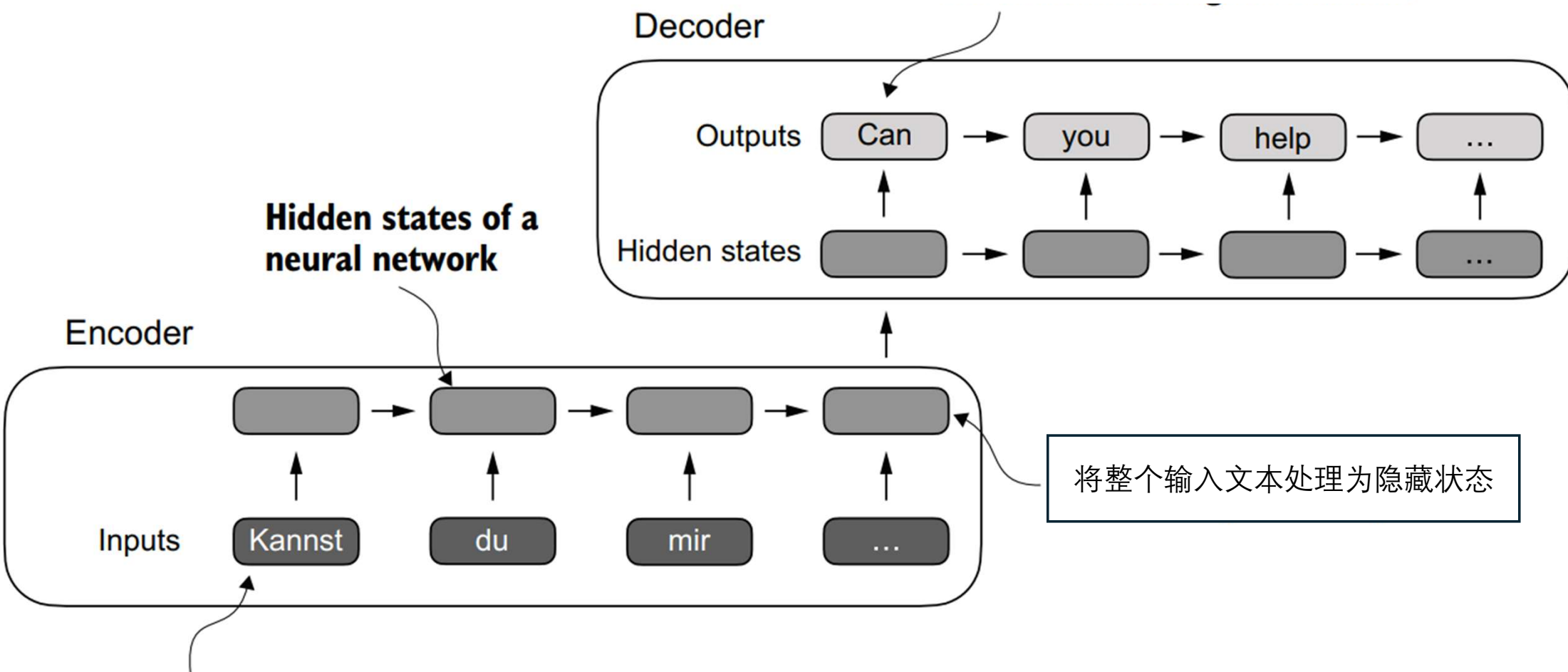
The correct translation

Certain words in the generated translation require access to words that appear earlier or later in the original sentence.

Encoder(编码器)-Decoder(解码器)结构

解码器采用隐藏状态来产生输出

e



Encoder(编码器)-Decoder(解码器)结构

解码器采用隐藏状态来产生输出

仅依赖于封装了所有相关信息的隐藏状态时，这可能会导致上下文丢失，尤其是在依赖关系可能跨越很长距离的复杂句子中。

Hidden states of a
neural network

Encoder

Inputs

Kannst

du

mir

...

Decoder

Outputs

Can

you

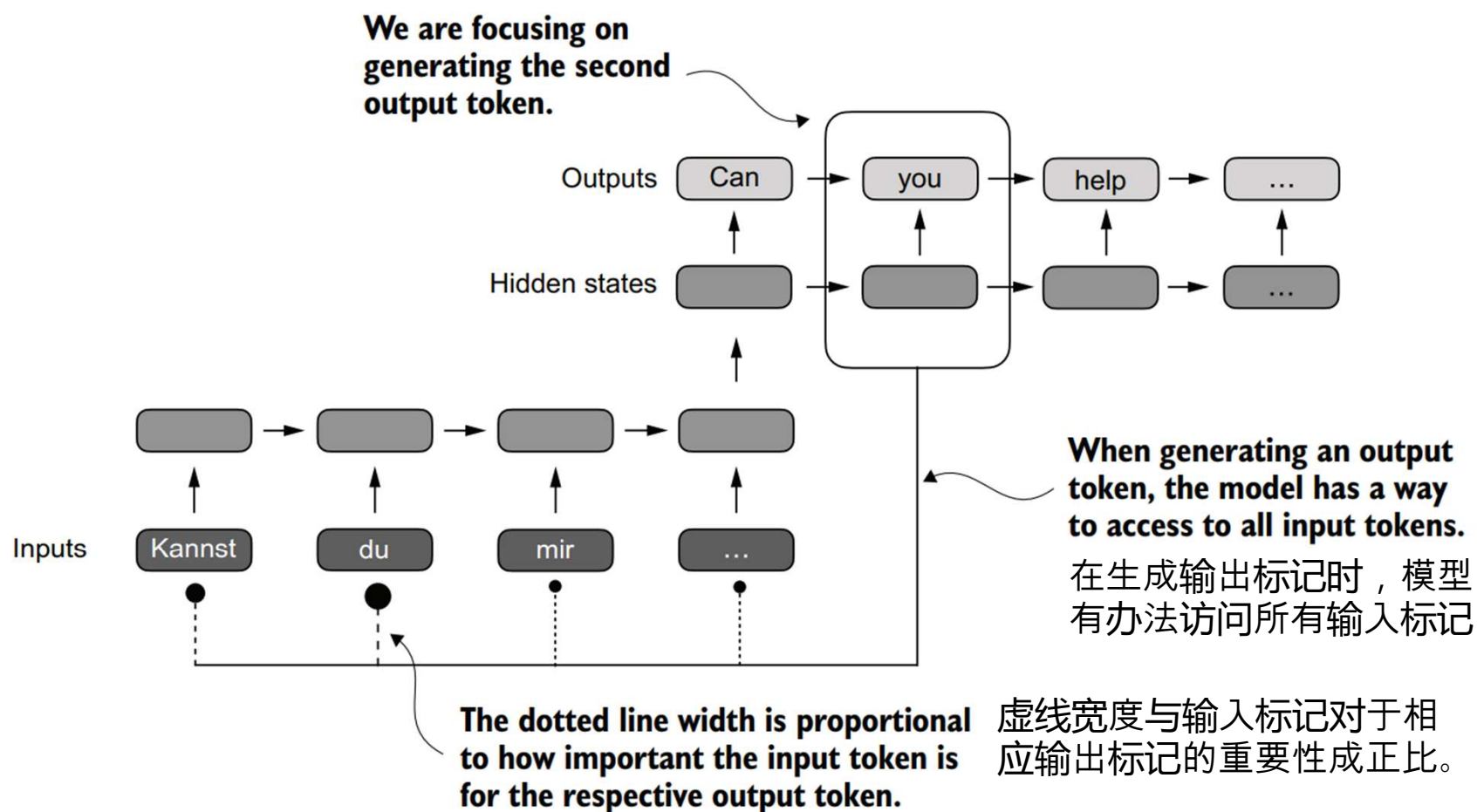
help

...

Hidden states

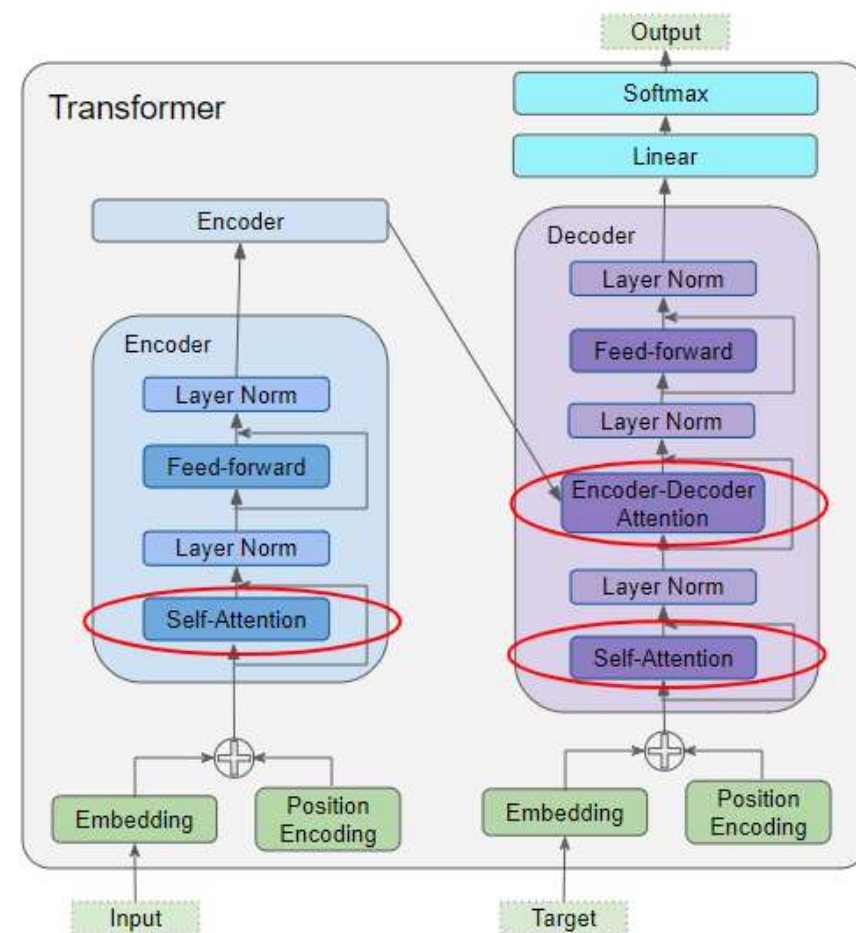
将整个输入文本处理为隐藏状态

在每个解码步骤中选择性地访问输入序列的不同部分



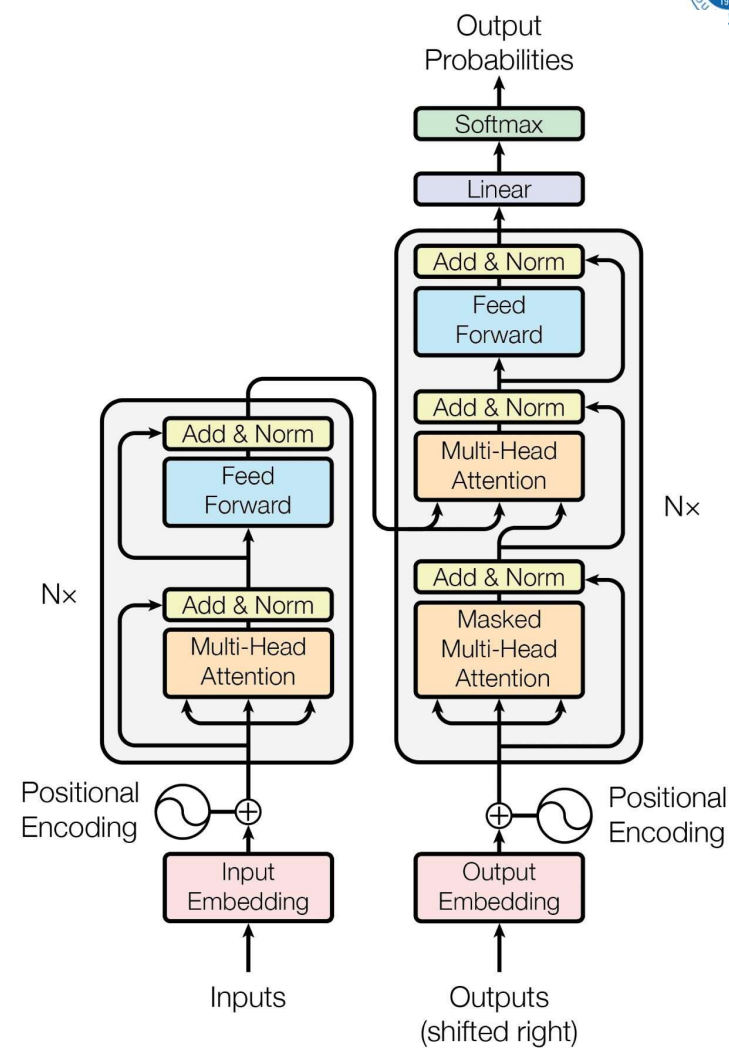
Transformer

- Encoder-Decoder结构
- 每个encoder和decoder block都包含自注意力(Self Attention)



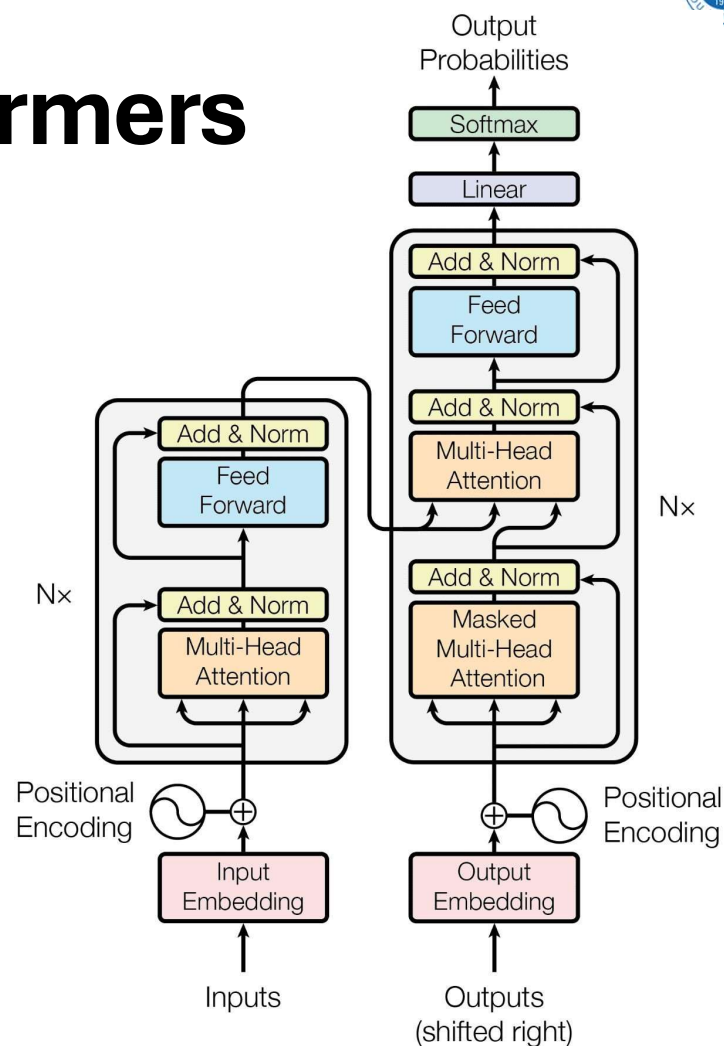
Part 1

Transformer Architecture



Transformers

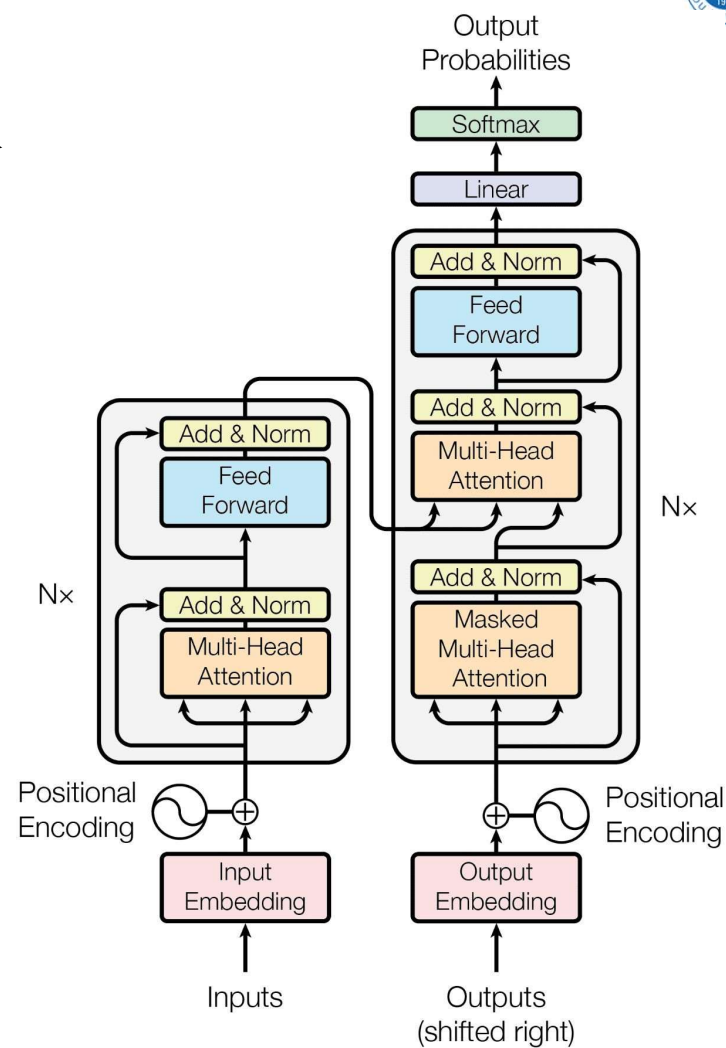
- 分词
- 输入嵌入
- 位置编码
- Query, Key, & Value
- 注意力机制
- 自注意力
- 多头注意力
- Feed Forward
- Add & Norm
- 编码器
- 因果注意力机制
- 编码器-解码器注意力机制
- 线性层
- Softmax
- 解码器
- 编码器-解码器



输入

处理输入

Inputs
I ate an apple

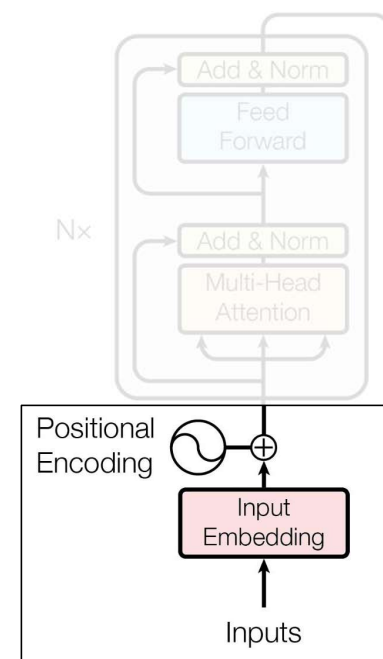


分词

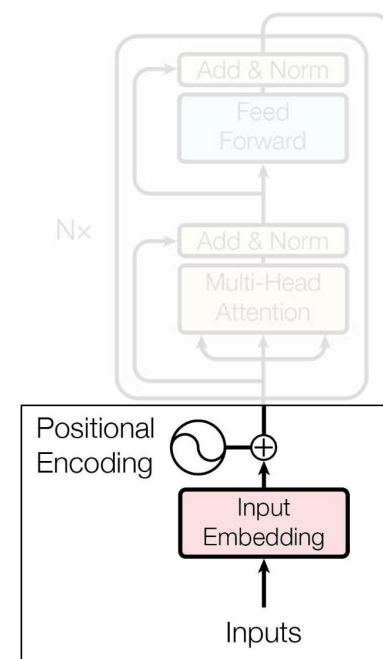
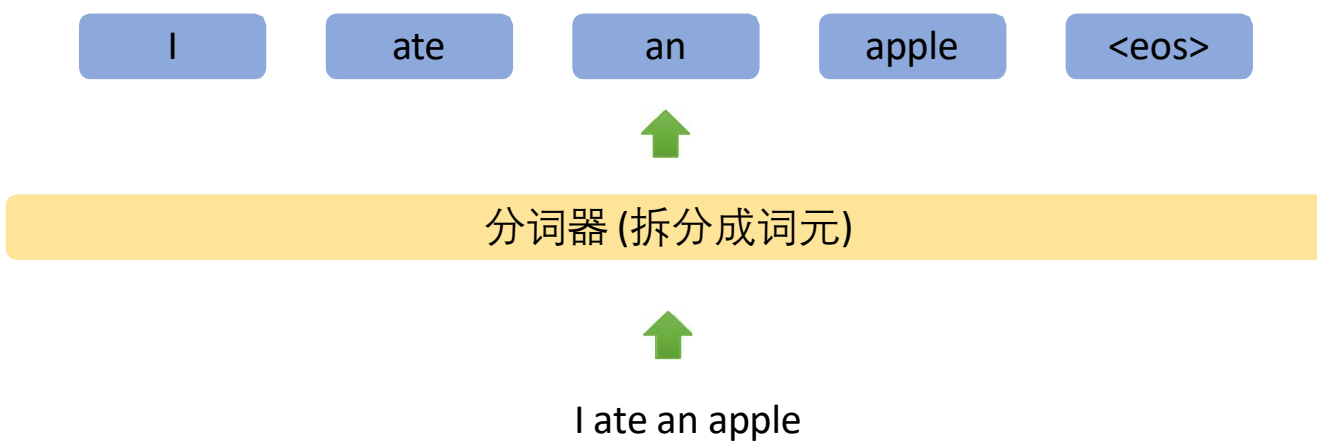
分词器 (拆分成词元)



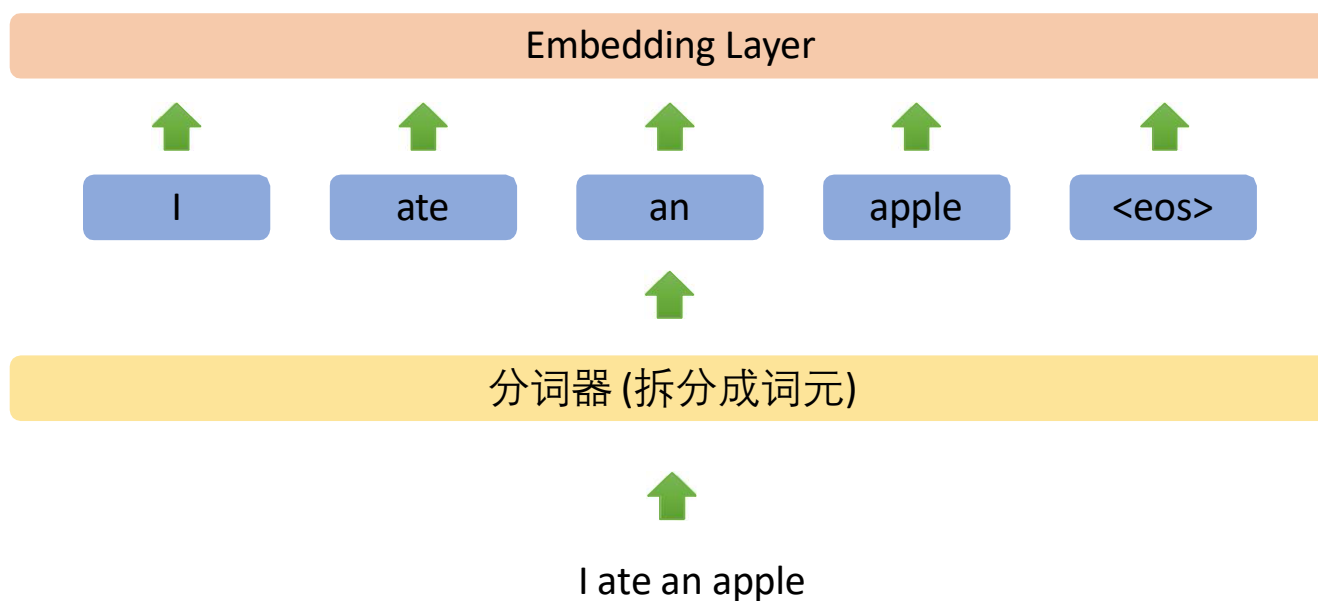
I ate an apple



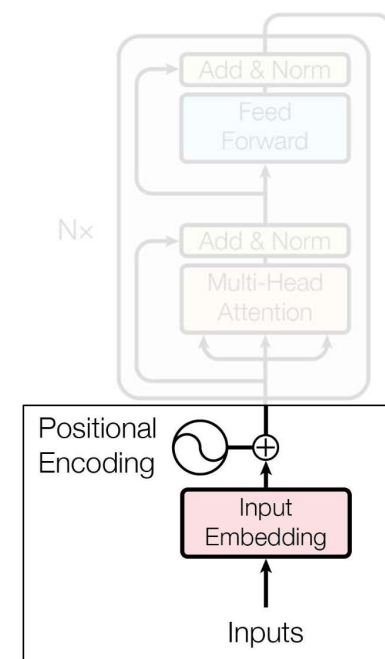
分词



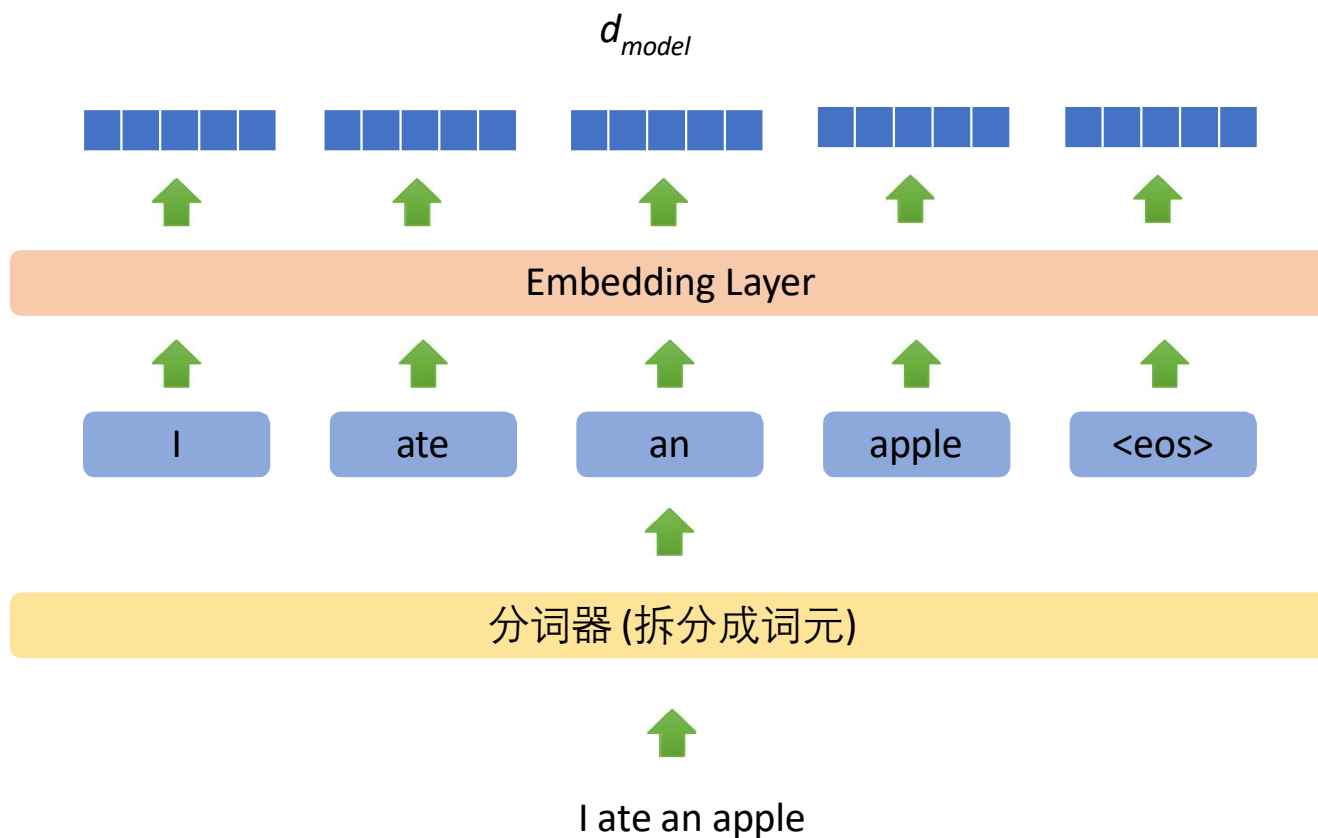
输入嵌入



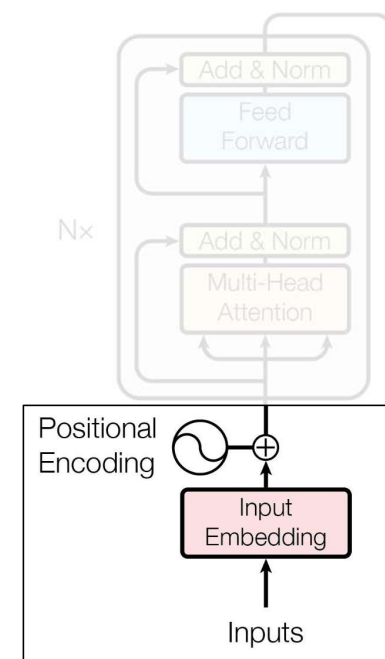
生成 输入嵌入



输入嵌入



生成 输入嵌入



位置编码

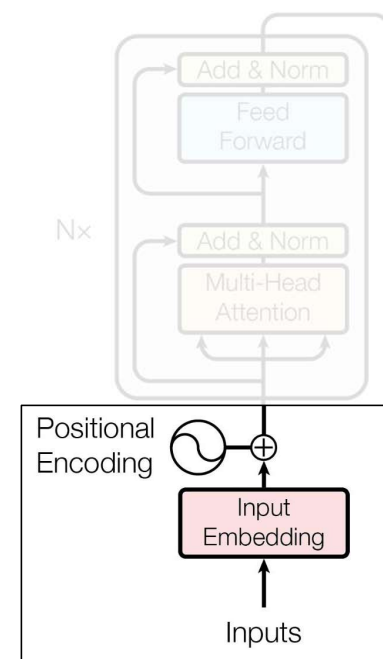
I

ate

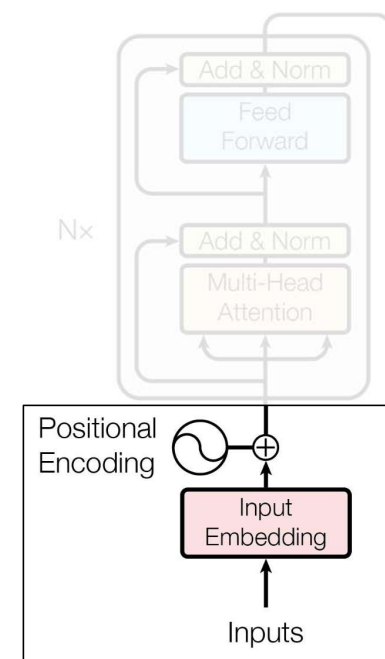
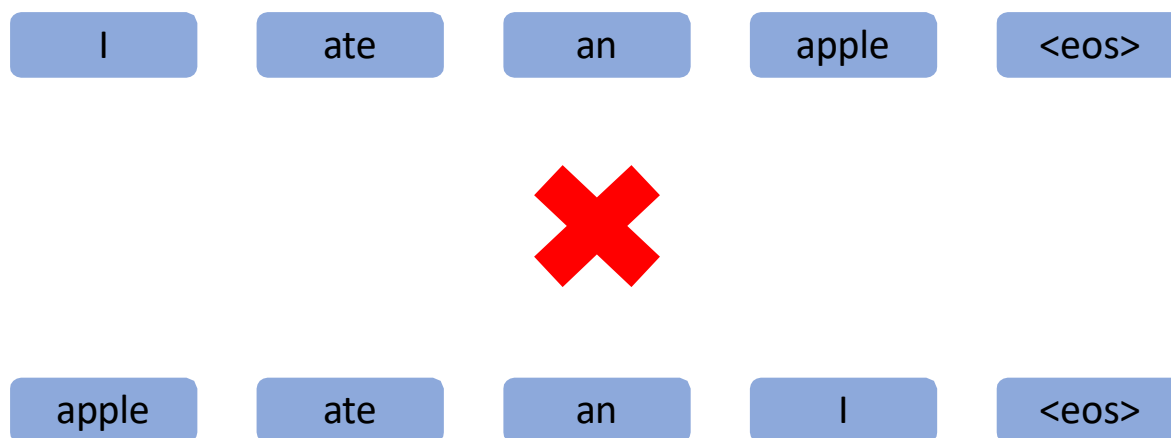
an

apple

<eos>

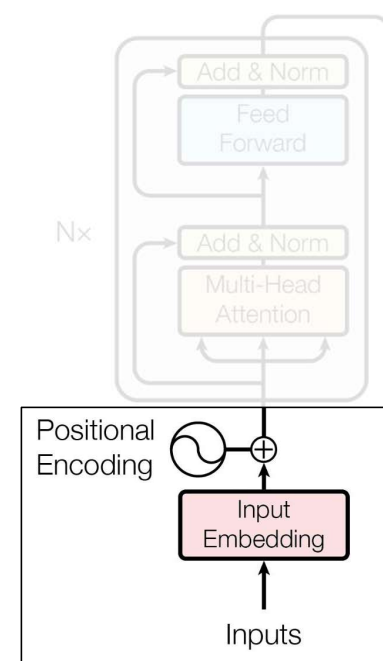


位置编码

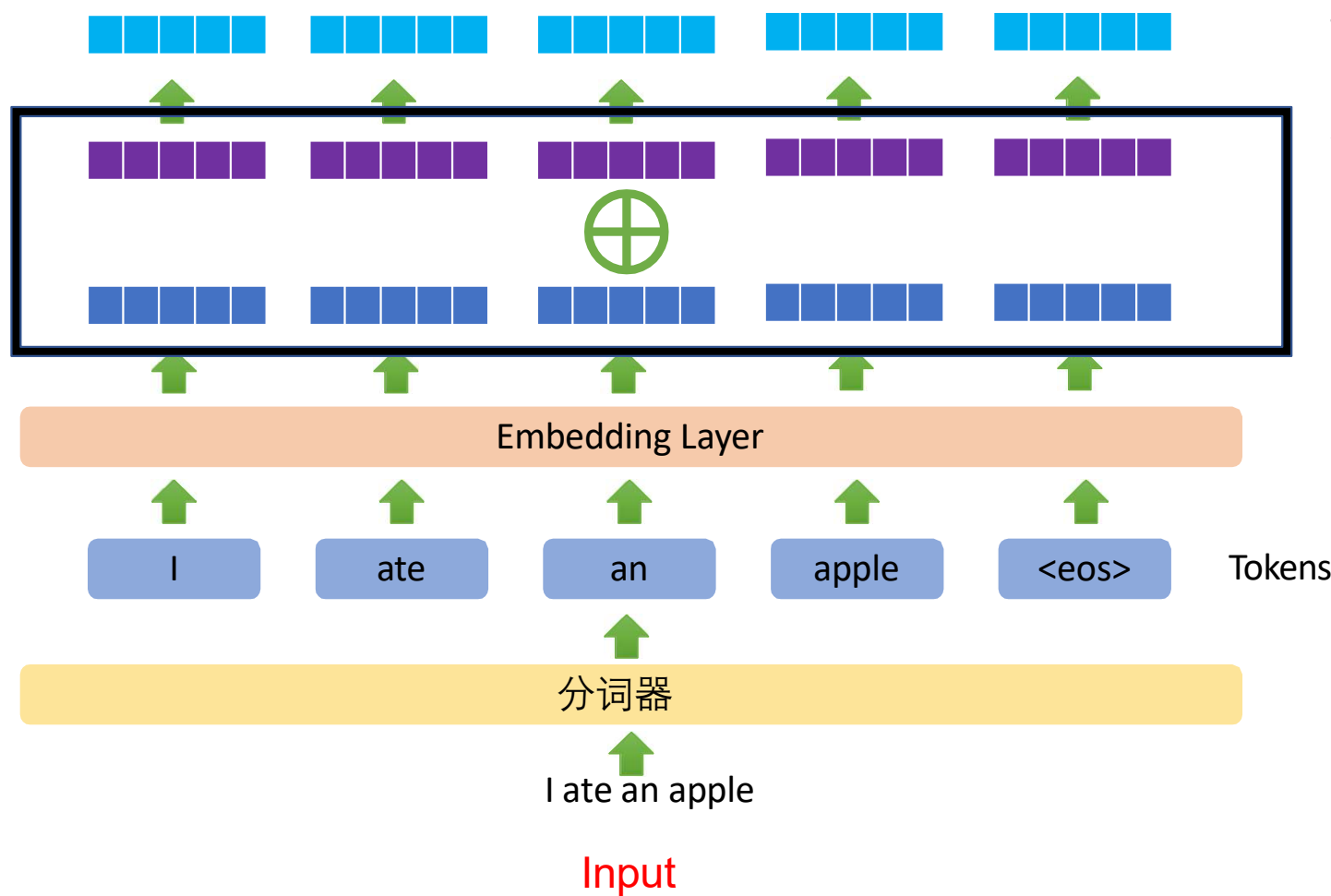


Requirements for Positional Encodings???

位置编码



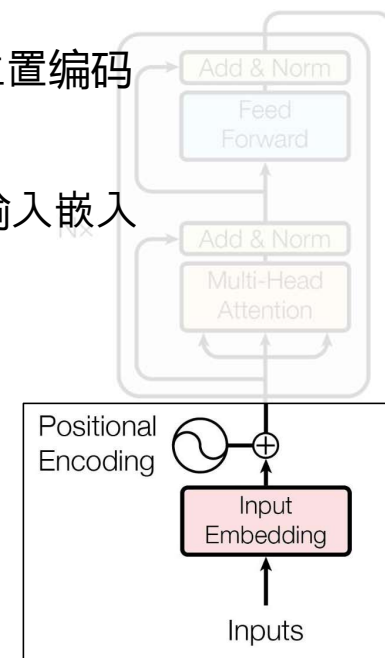
位置编码



最终输入嵌入

位置编码

输入嵌入



Transformers

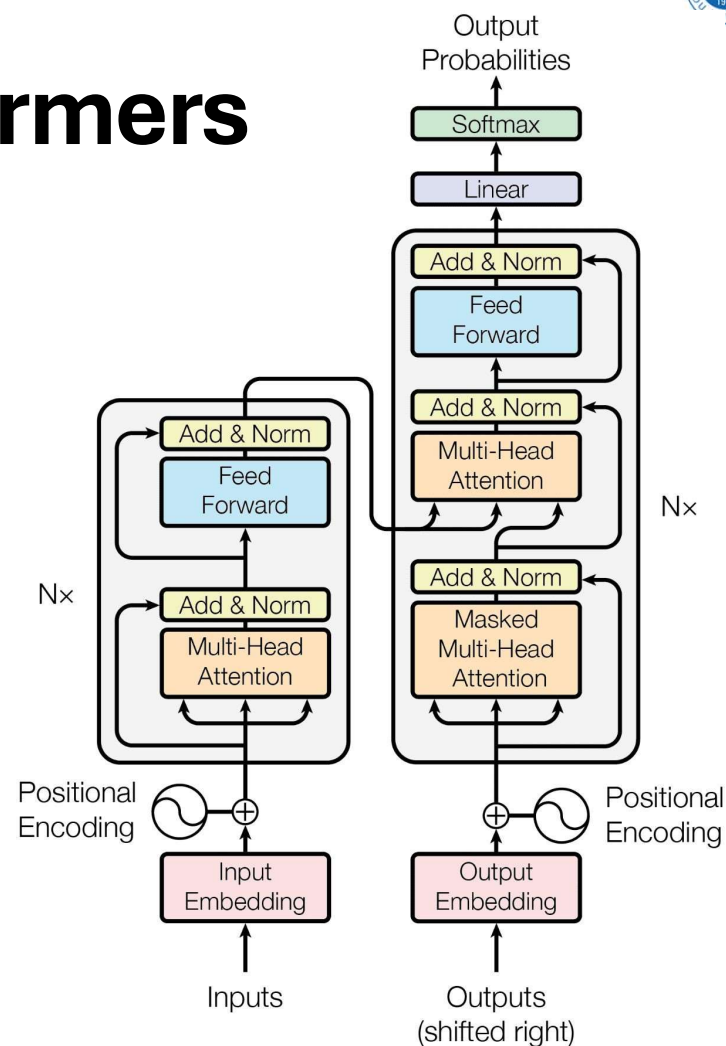
✓ Tokenization

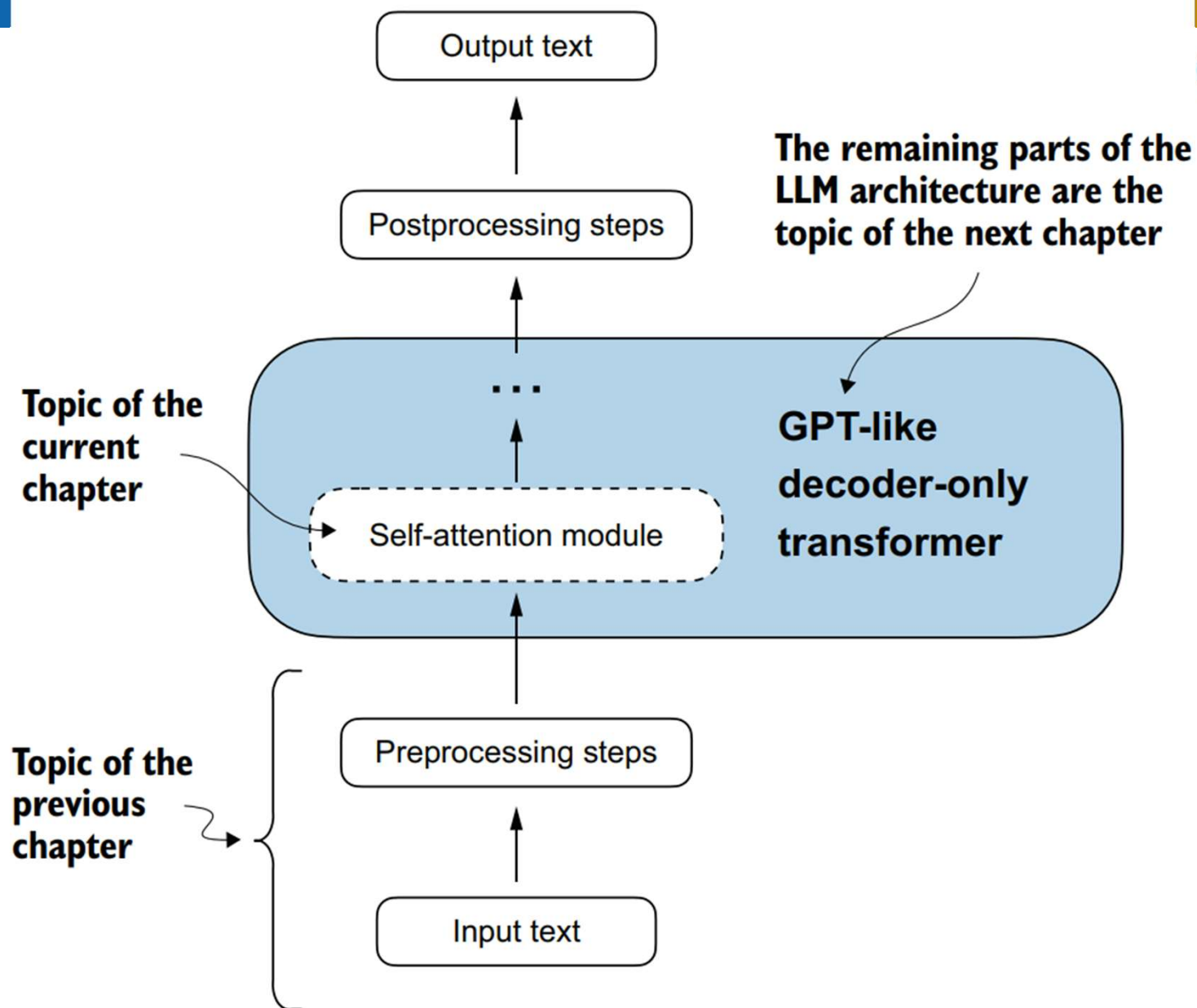
✓ 输入嵌入

✓ 位置编码

- Query, Key, & Value
- 注意力机制
- 自注意力
- 多头注意力
- Feed Forward
- Add & Norm
- 编码器

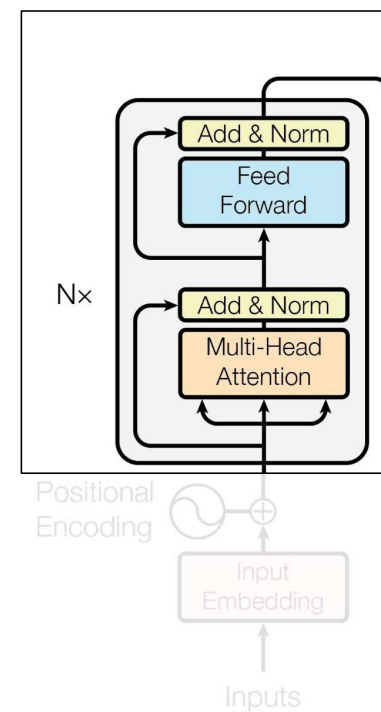
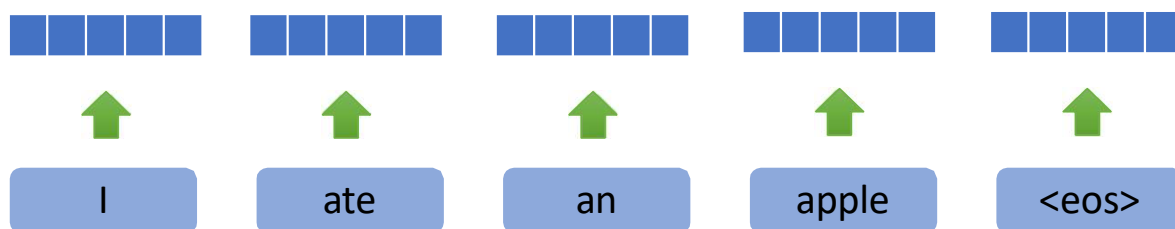
- 掩码注意力机制
- 编码器-解码器注意力机制
- Linear
- Softmax
- 解码器s
- 编码器-解码器 Models





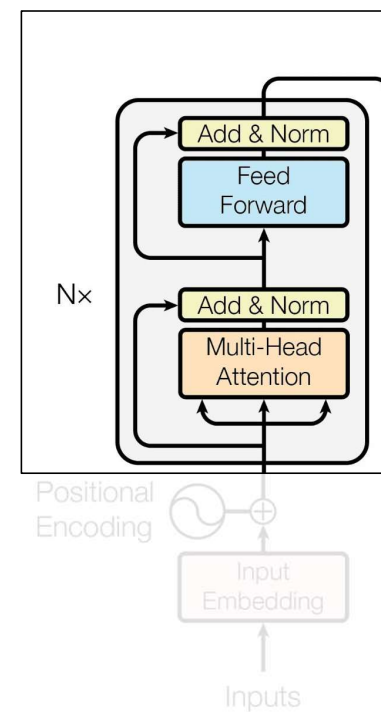
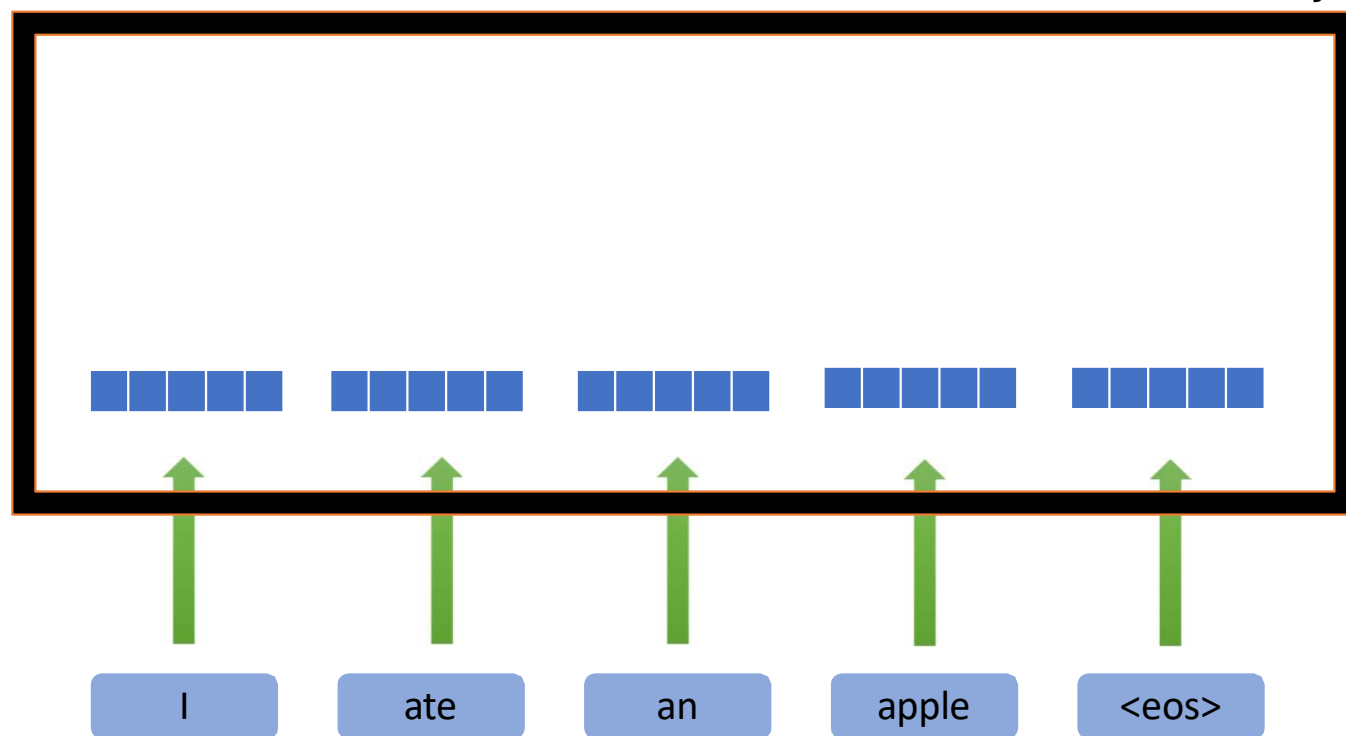
编码器

WHERE IS THE
CONTEXT?



编码器

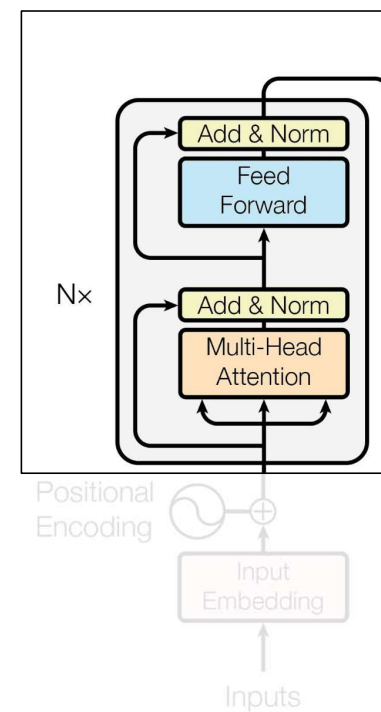
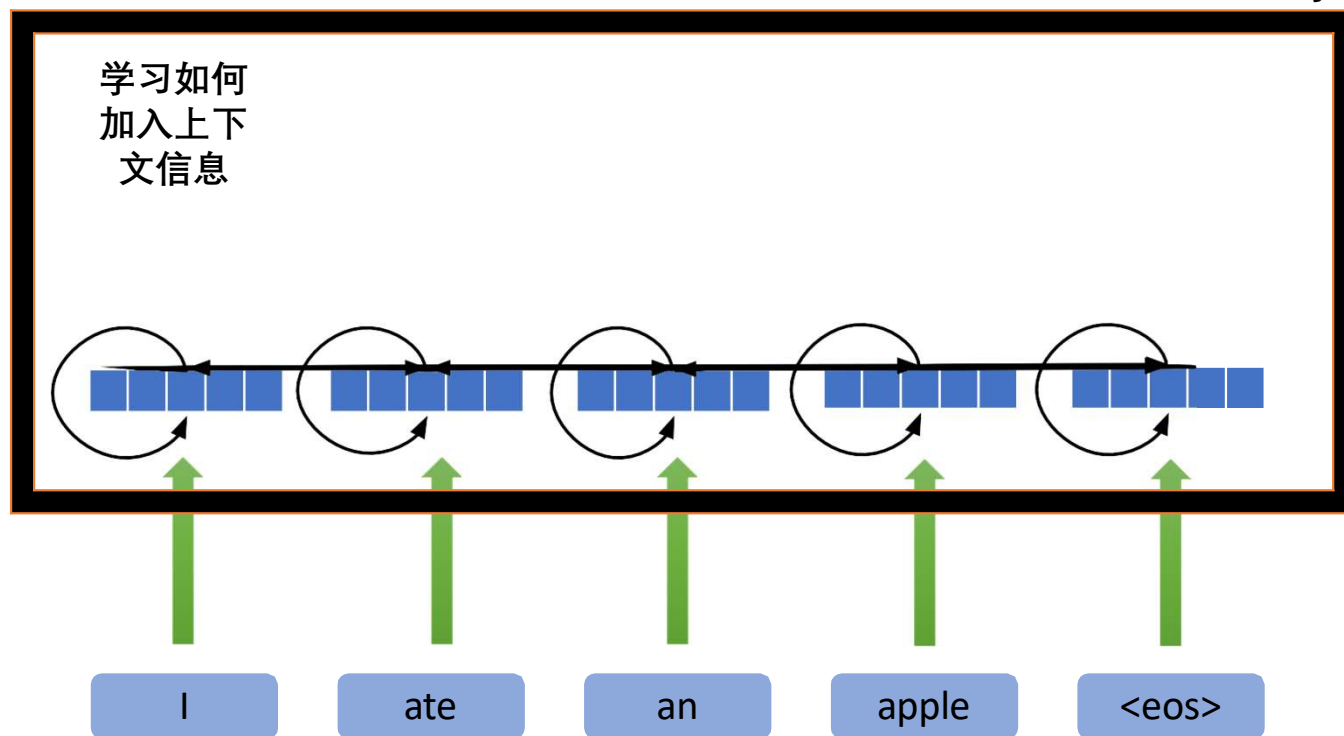
某种“黑盒子”



编码器

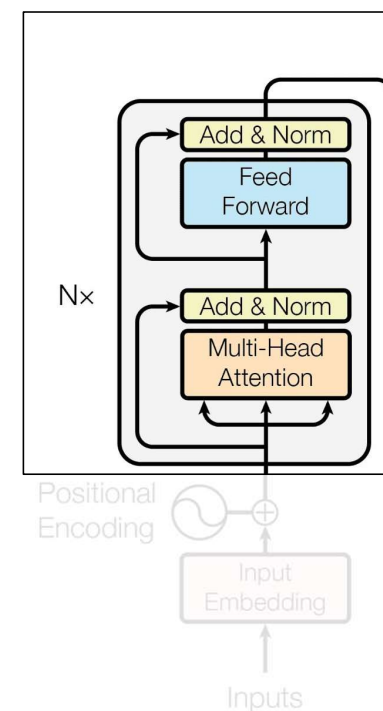
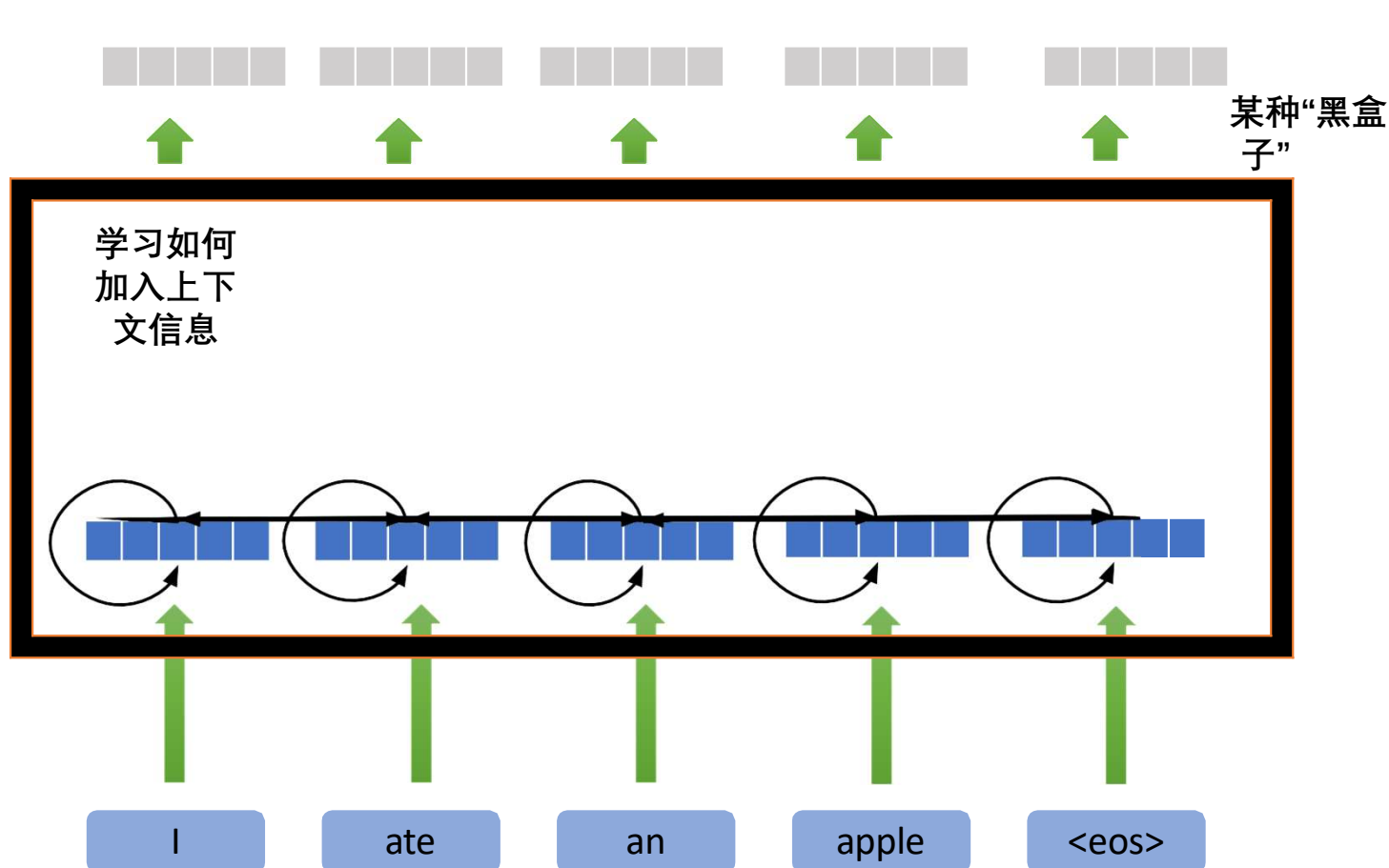
某种“黑盒子”

学习如何
加入上下
文信息



编码器

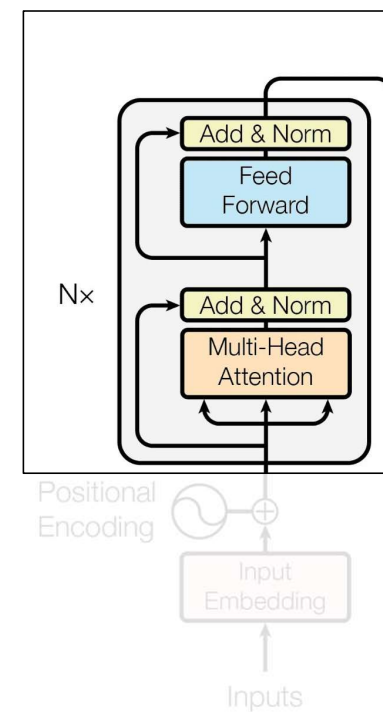
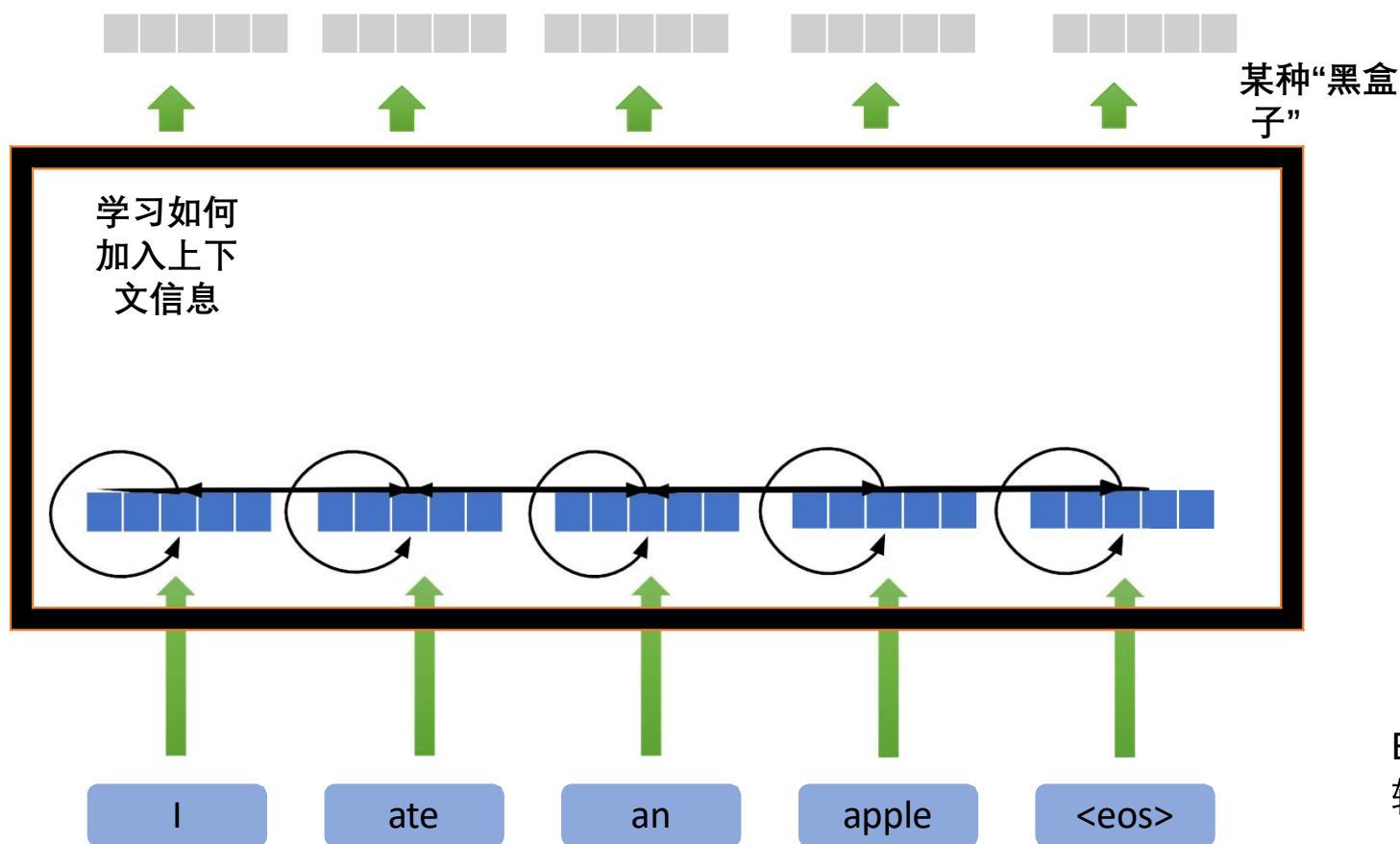
包含上下文信息的嵌入表示



编码器

α
[ij]

输出：包含上下文信息的嵌入表示



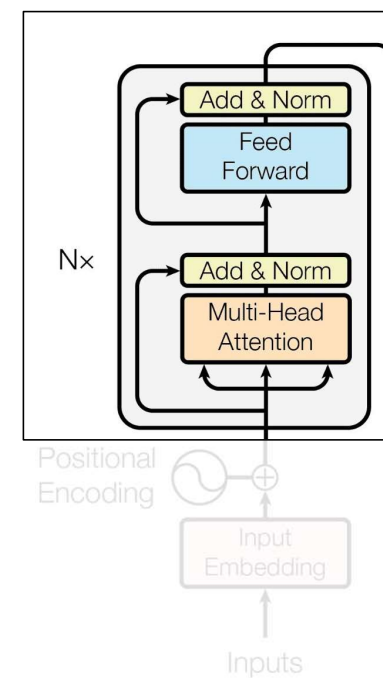
Encoder:

输入词向量 → 输出“带上下文”的词向量

注意力机制

$\alpha_{[ij]}$?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



大纲

A simplified self-attention technique to introduce the broader idea

1) Simplified self-attention

简化版自注意力

简化版自注意力

2) Self-attention

Self-attention with trainable weights that forms the basis of the mechanism used in LLMs

A type of self-attention used in LLMs that allows a model to consider only previous and current inputs in a sequence, ensuring temporal order during the text generation

因果注意力
3) Causal attention

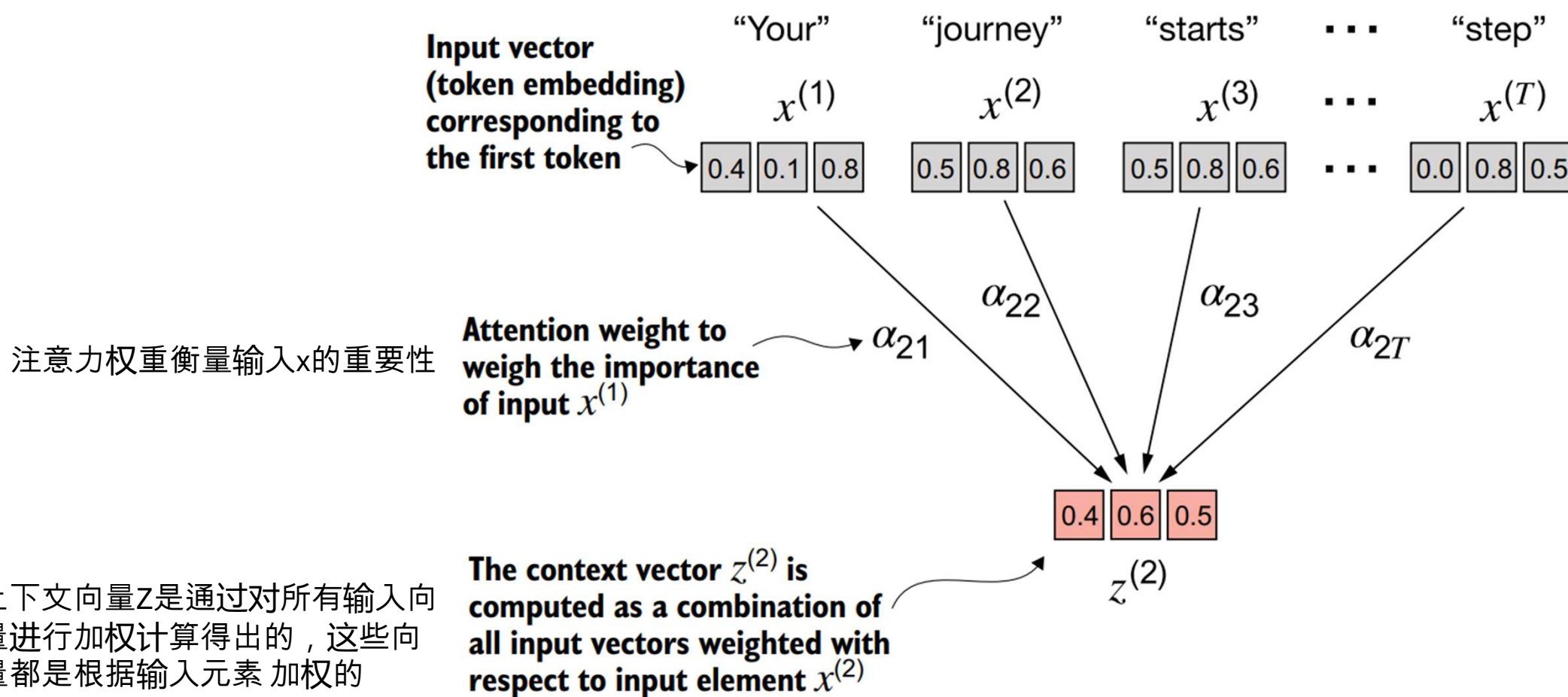
多头注意力

4) Multi-head attention

An extension of self-attention and causal attention that enables the model to simultaneously attend to information from different representation subspaces

不包含任何可训练的权重

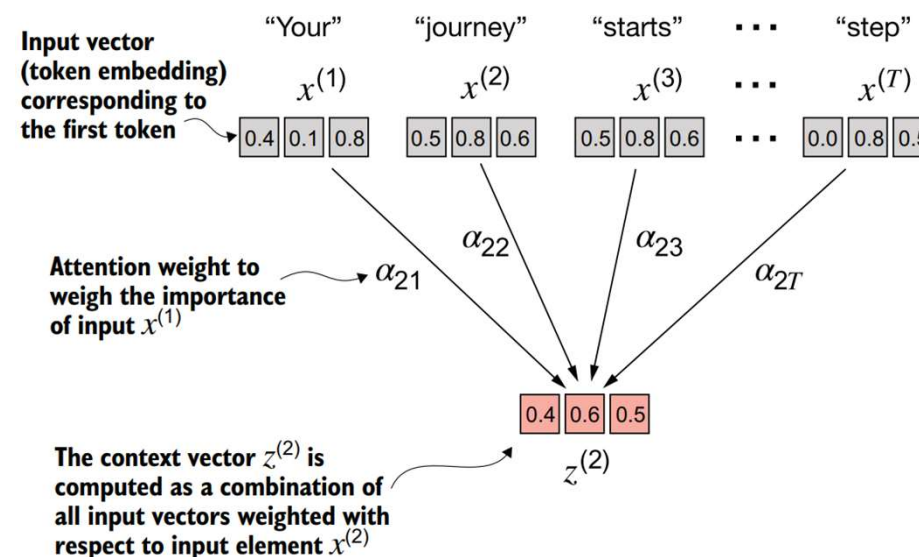
自注意力机制（简化版）



不包含任何可训练的权重

自注意力机制（简化版）

- 在自注意力机制中，我们的目标是计算输入序列中每个元素 x 的上下文向量 z 。
 - 通过整合序列中所有其他元素的信息来创建输入序列（如句子）中每个元素的丰富表示



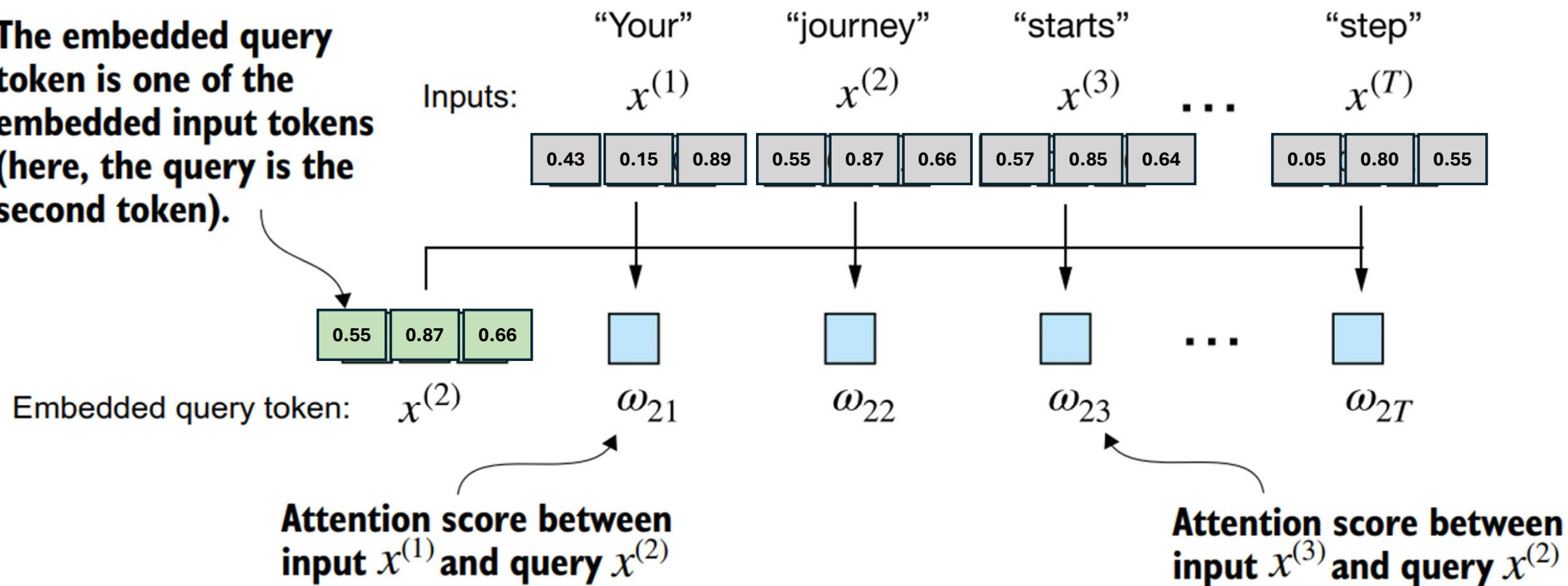
自注意力机制 (简化版)

点积本质上是两个向量逐个元素相乘
然后对乘积求和

使用第二个输入元素 x 作为查询来计算上下文向量 z

• 通过点积计算中间值 注意力分数 ω

The embedded query token is one of the embedded input tokens (here, the query is the second token).

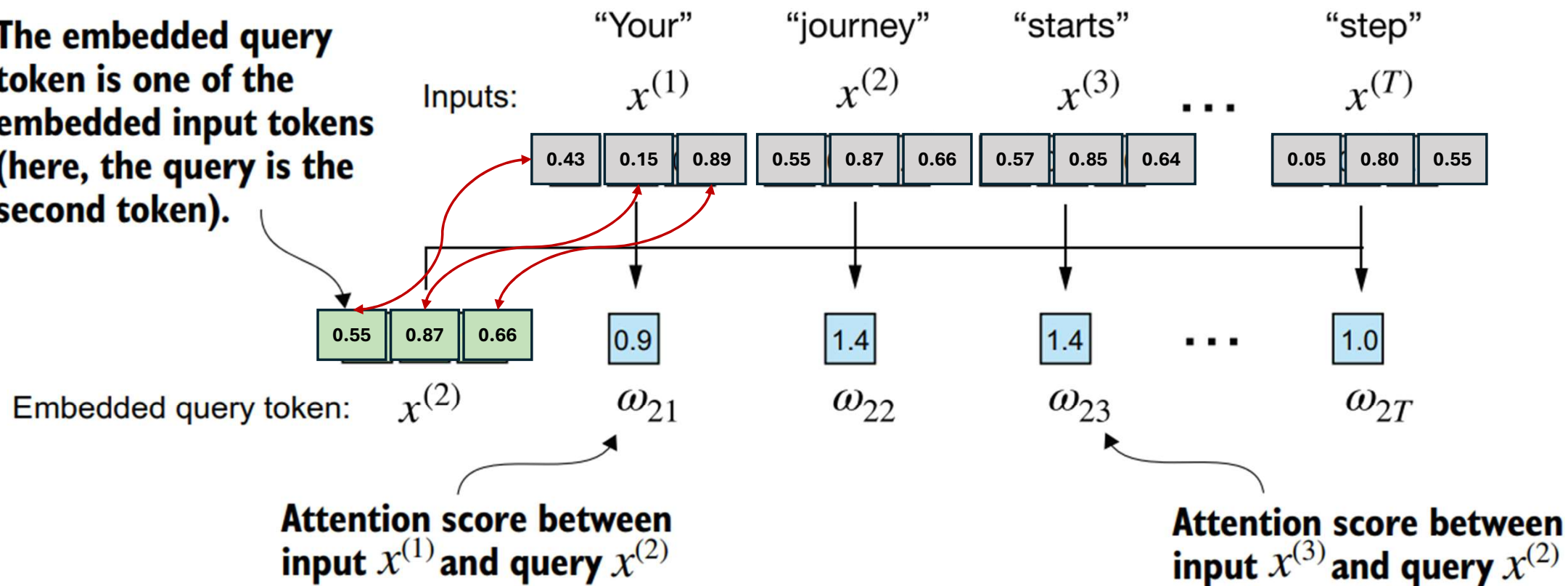


自注意力机制（简化版）

使用第二个输入元素 x 作为查询来计算上下文向量 z

- 通过点积计算中间值 注意力分数 ω

The embedded query token is one of the embedded input tokens (here, the query is the second token).

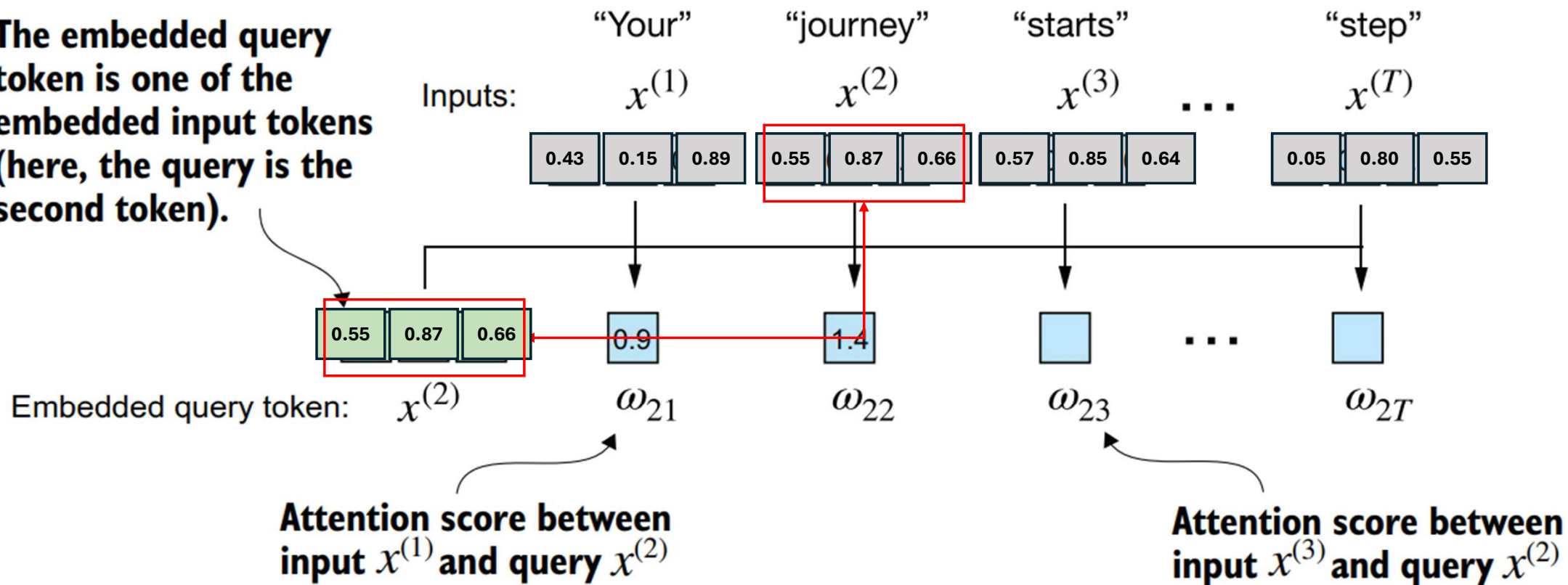


自注意力机制 (简化版)

使用第二个输入元素 x 作为查询来计算上下文向量 z

• 通过点积计算中间值 注意力分数 ω

The embedded query token is one of the embedded input tokens (here, the query is the second token).

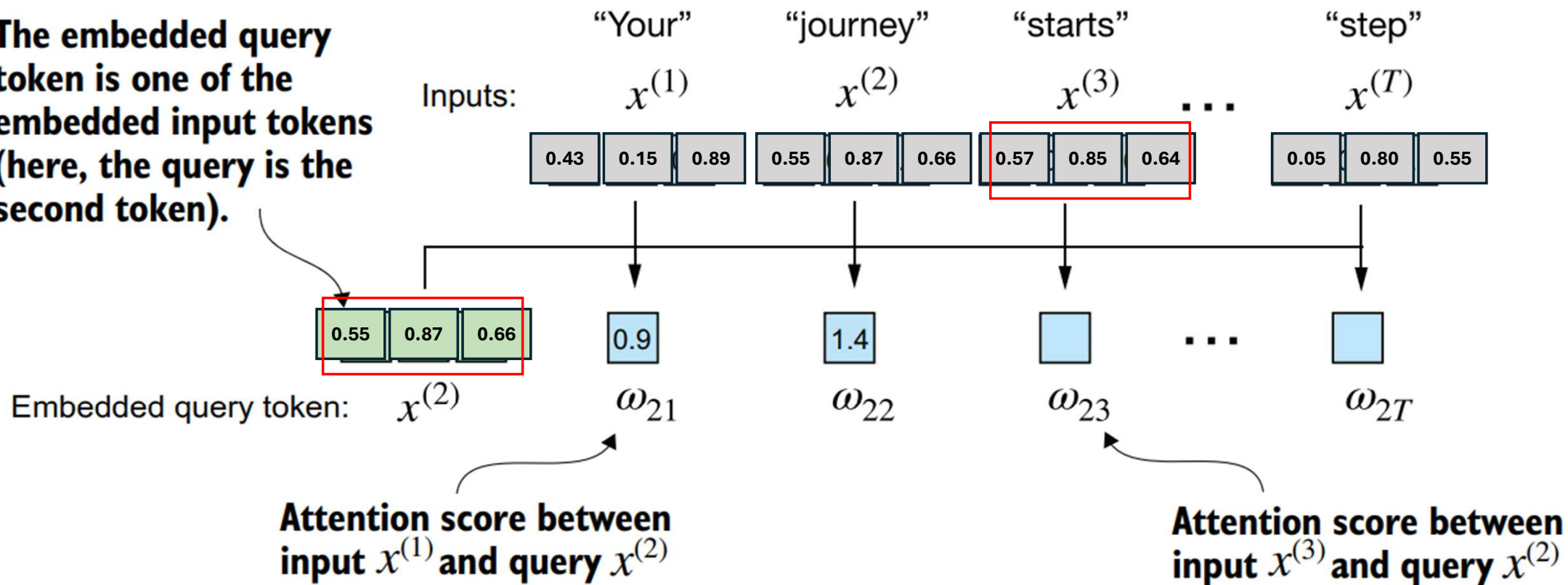


自注意力机制（简化版）

使用第二个输入元素 x 作为查询来计算上下文向量 z

- 通过点积计算中间值 注意力分数 ω

The embedded query token is one of the embedded input tokens (here, the query is the second token).

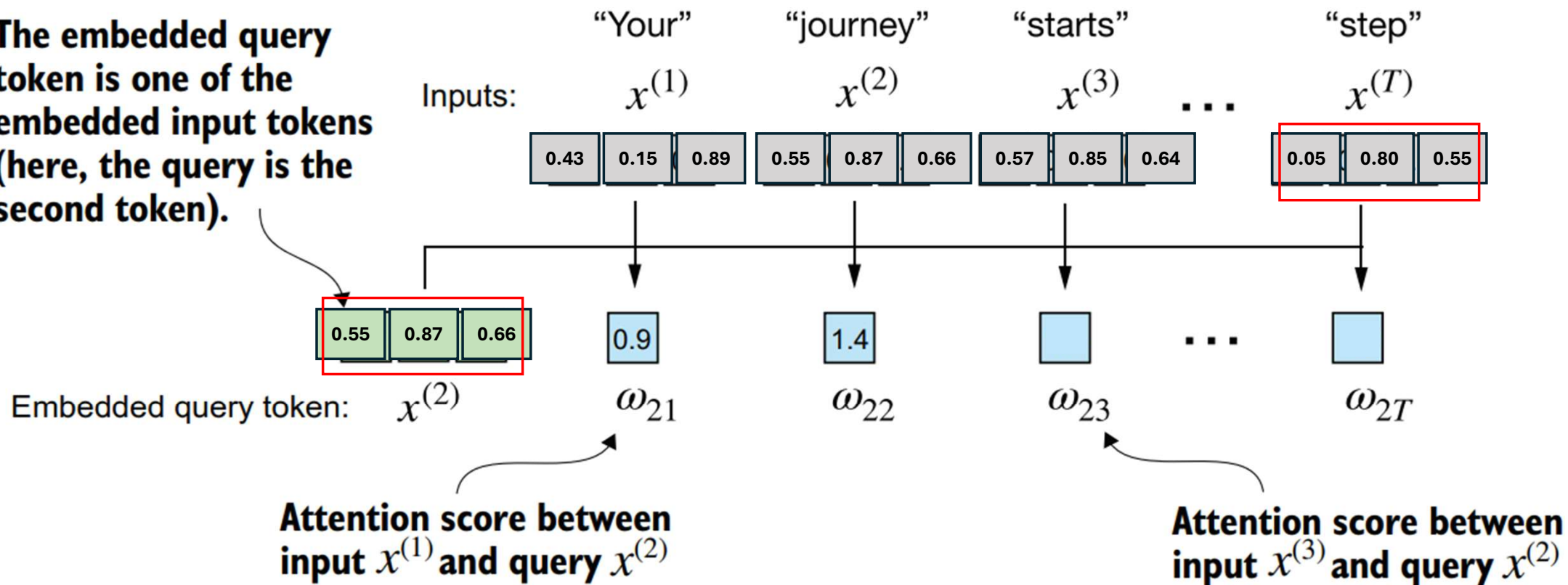


自注意力机制 (简化版)

使用第二个输入元素 x 作为查询来计算上下文向量 z

- 通过点积计算中间值 注意力分数 ω

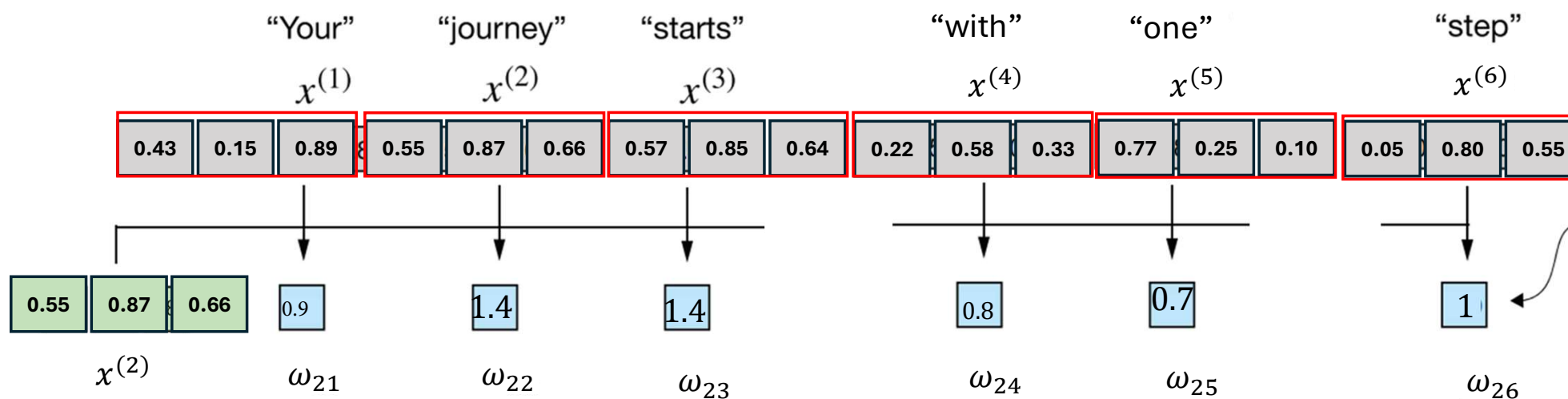
The embedded query token is one of the embedded input tokens (here, the query is the second token).



课堂练习-1

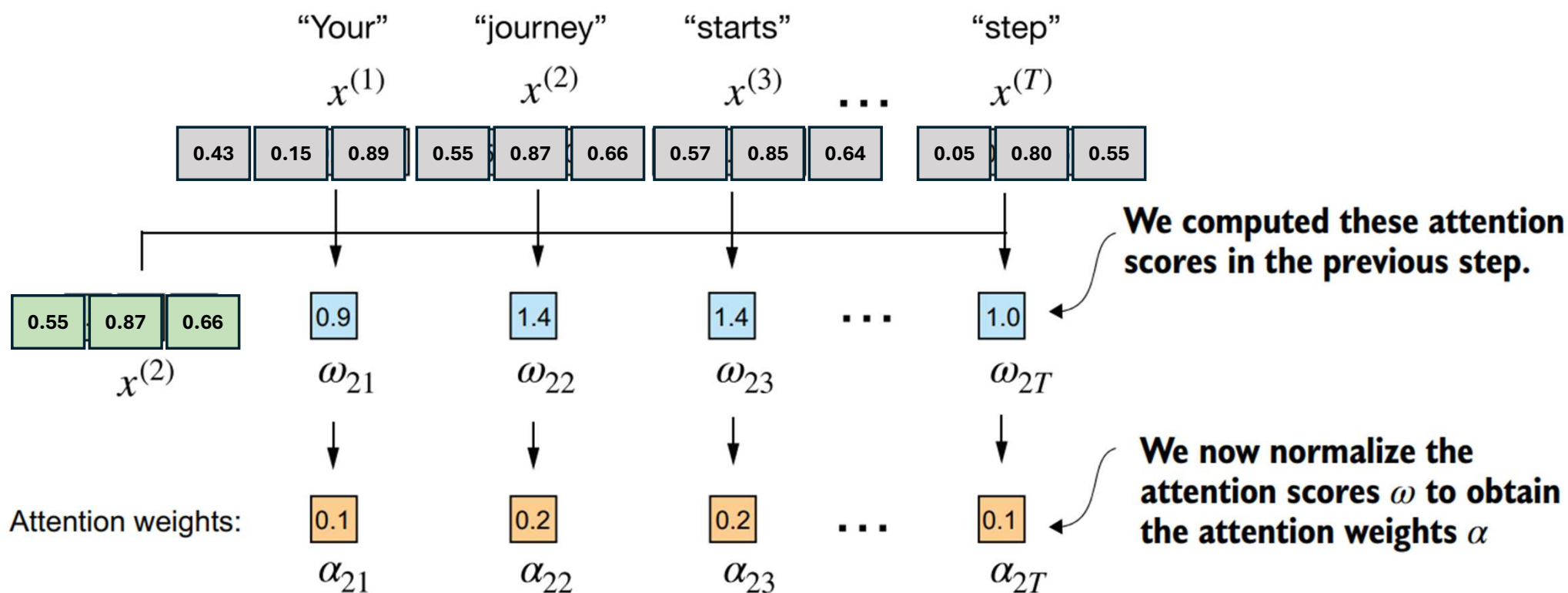
Softmax 计算器: [Softmax Function, Calculator and Formula](#)

- 计算出 $x^{(1)}$ 的注意力分数 ω , 保留小数点后1位



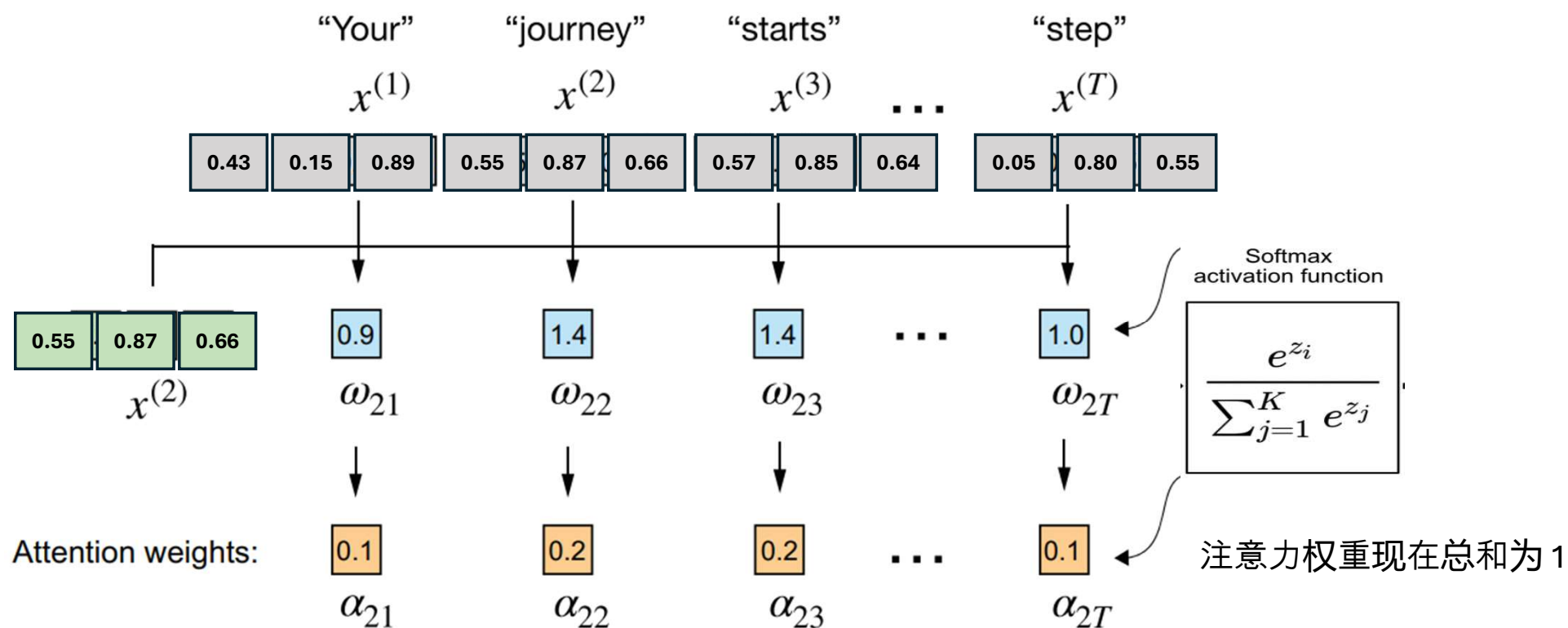
自注意力机制（简化版）

- 注意力权重 α ：对之前计算的每个注意力得分 ω 进行归一化（ $[0,1]$ ）



自注意力机制（简化版）

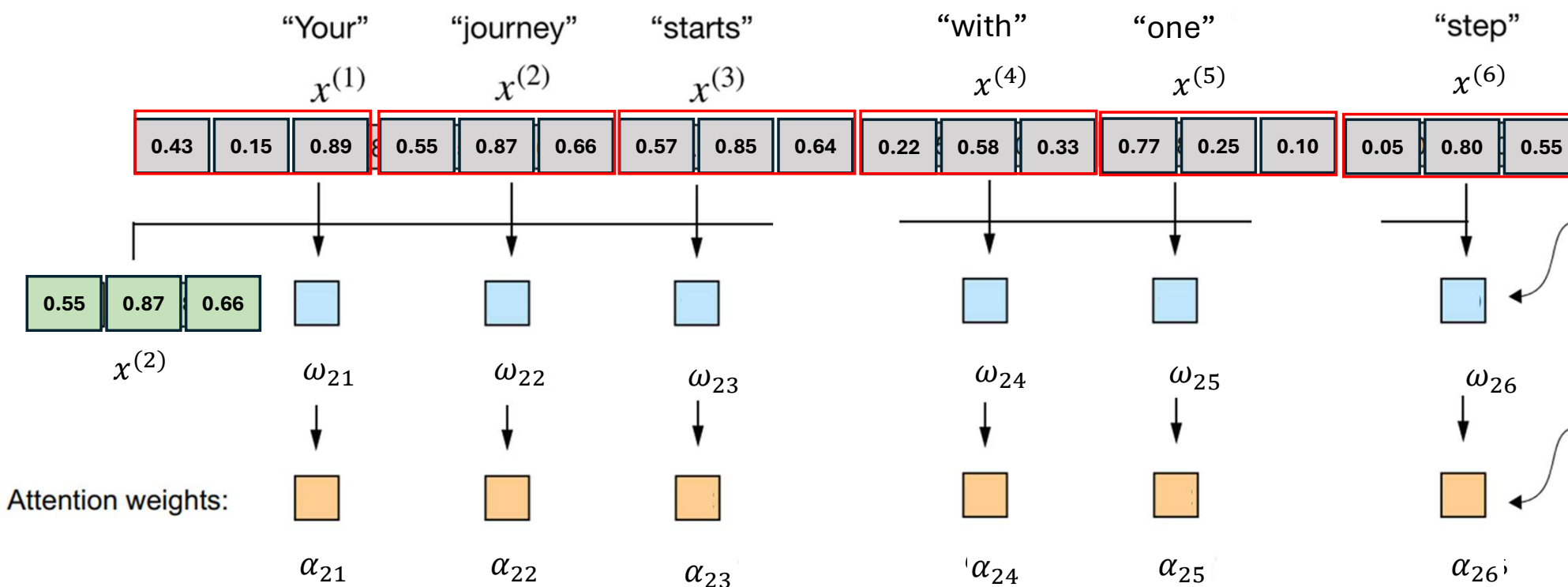
- 注意力权重 α ：对之前计算的每个注意力得分 ω 进行归一化（ $[0,1]$ ）



课堂练习-2

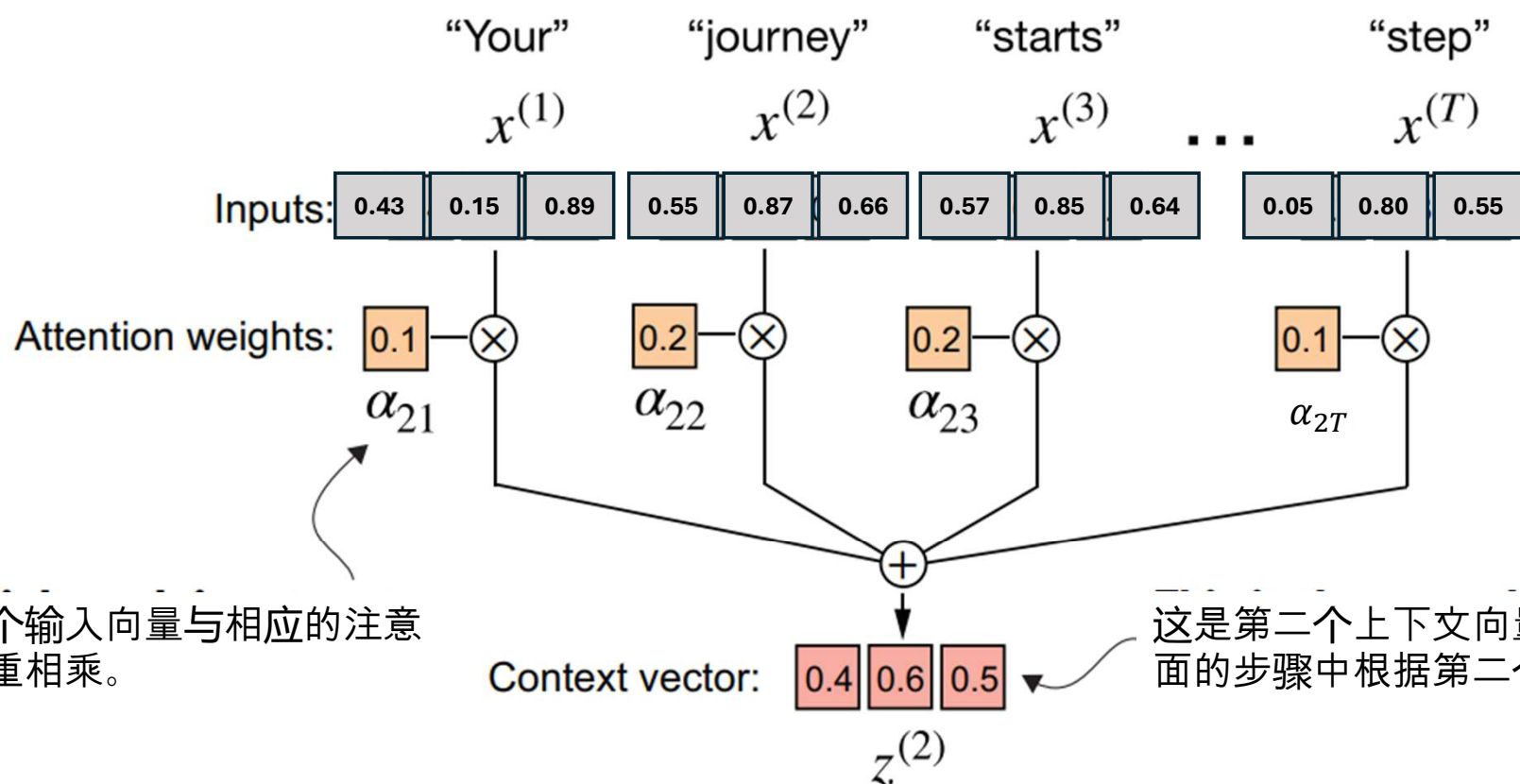
Softmax 计算器: [Softmax Function, Calculator and Formula](#)

- 计算出 $x^{(1)}$ 的注意力分数 ω ，保留小数点后1位



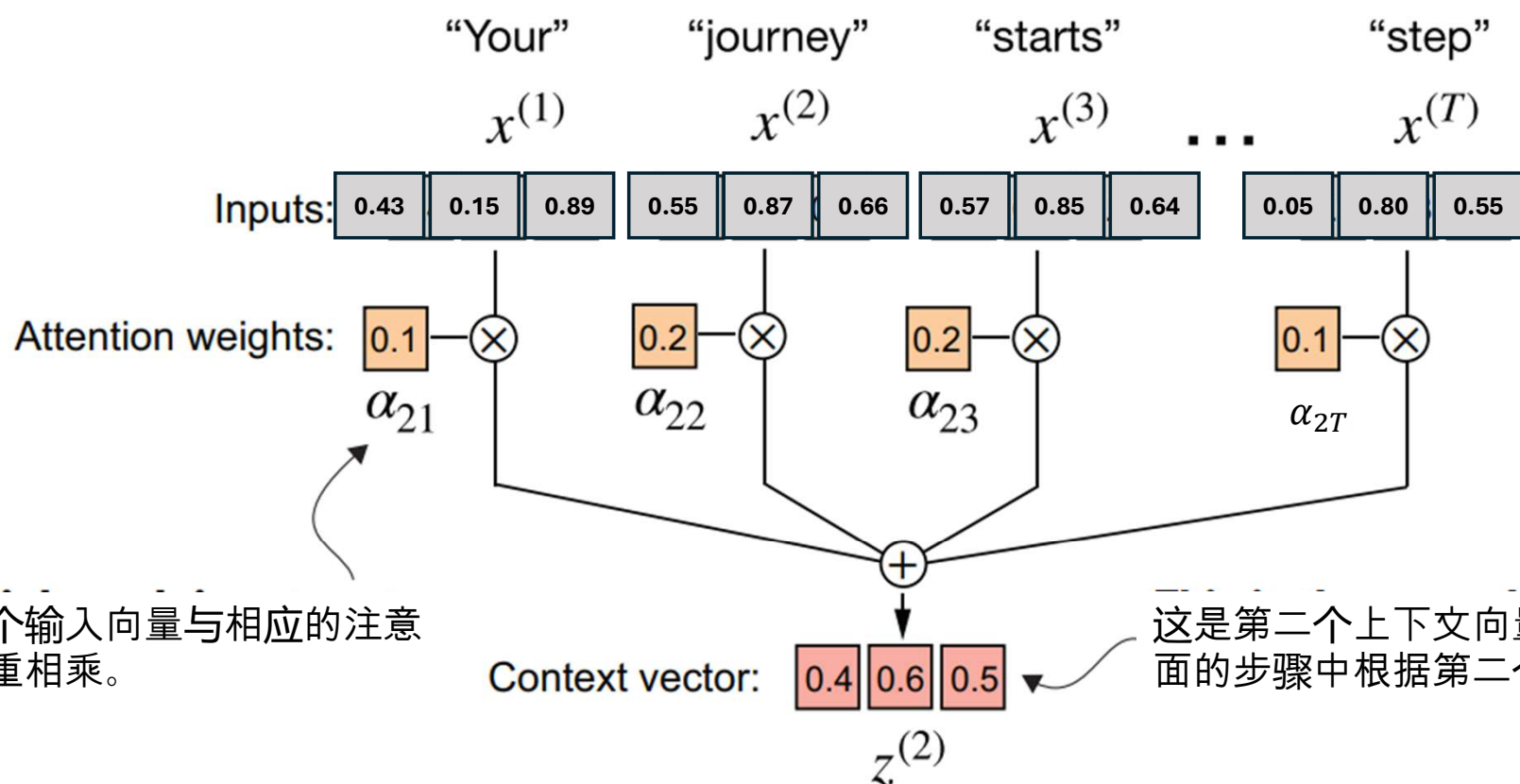
自注意力机制（简化版）

- 计算“journey”的上下文向量(Context vector)



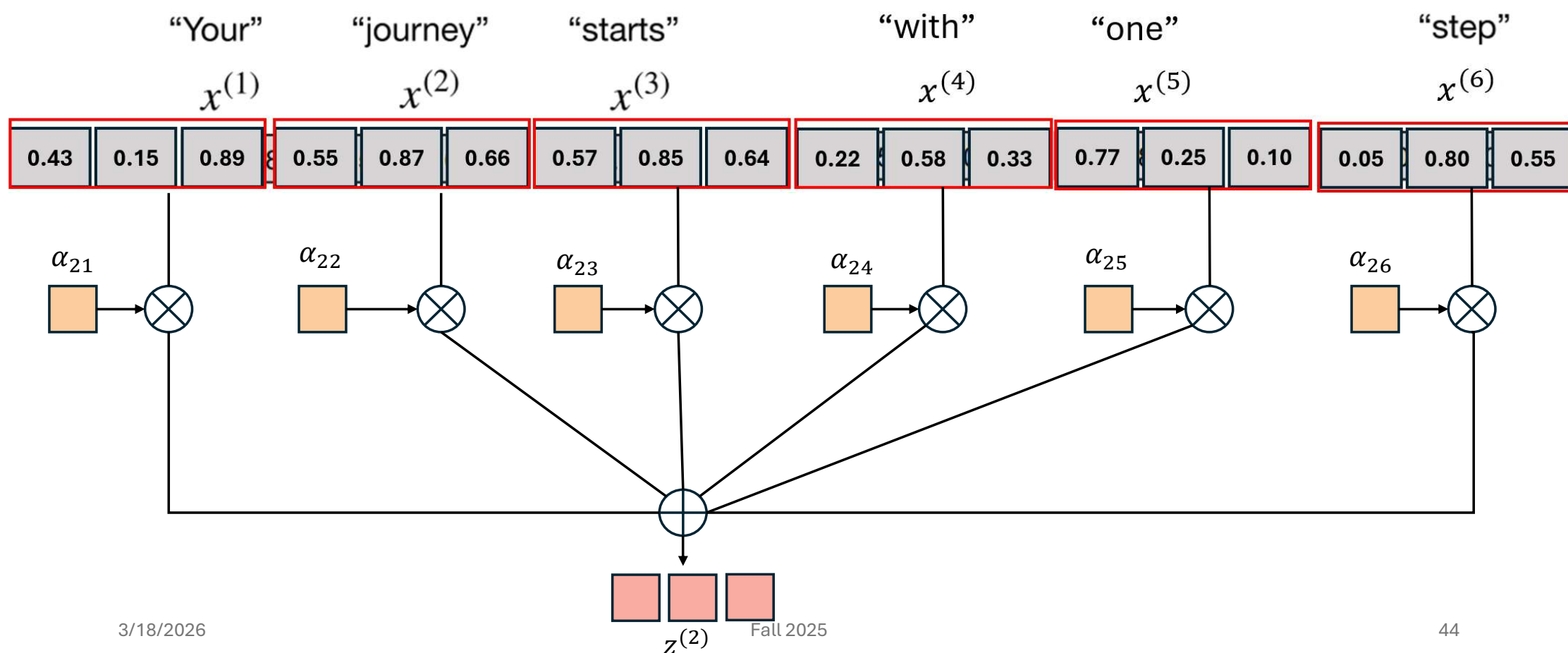
自注意力机制（简化版）

- 计算“journey”的上下文向量(Context vector)



课堂练习-3

计算“journey”的上下文向量(Context vector)



自注意力机制（简化版）

• 计算“journey”的上下文向量

注意力权重

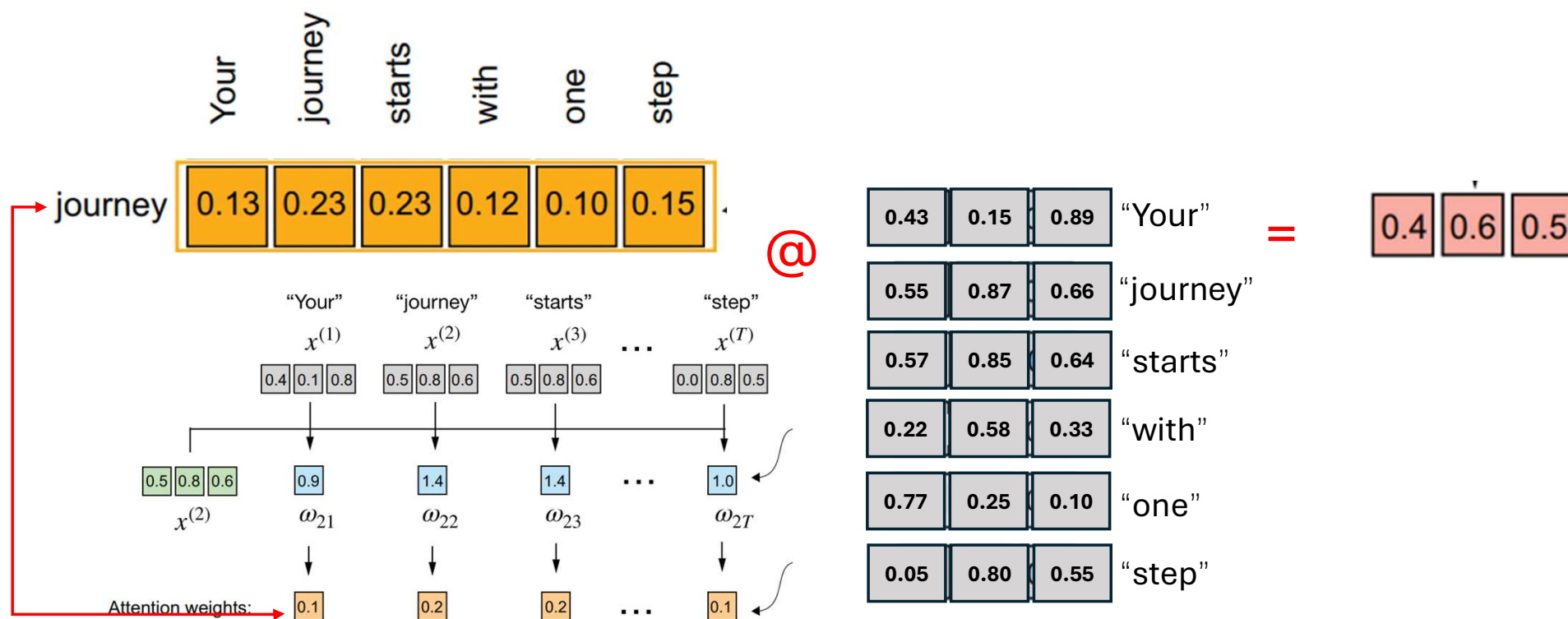
1x7

词向量矩阵

7x3

上下文向量

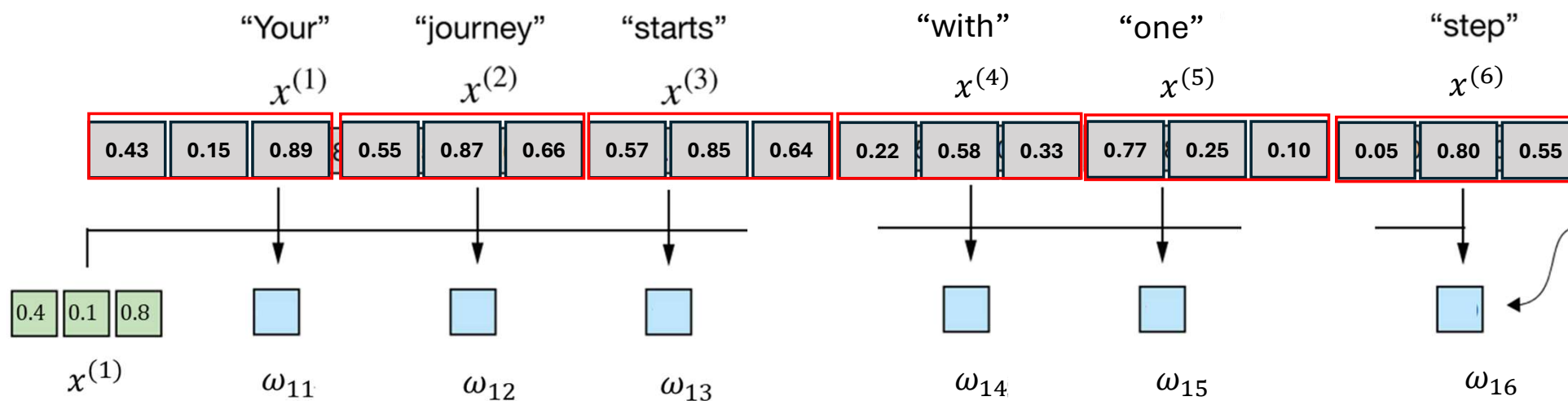
1x3



题目-1

Softmax 计算器: [Softmax Function, Calculator and Formula](#)

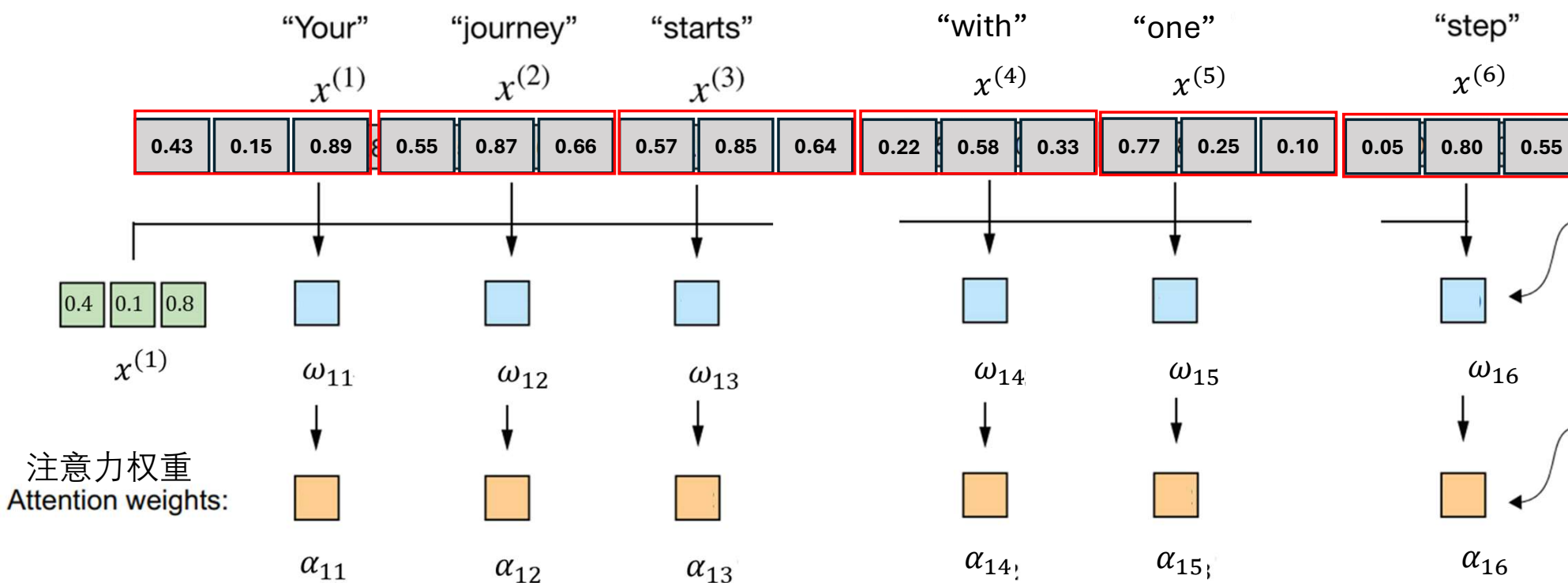
- 计算出 $x^{(1)}$ 的注意力分数 ω , 保留小数点后1位



题目-2

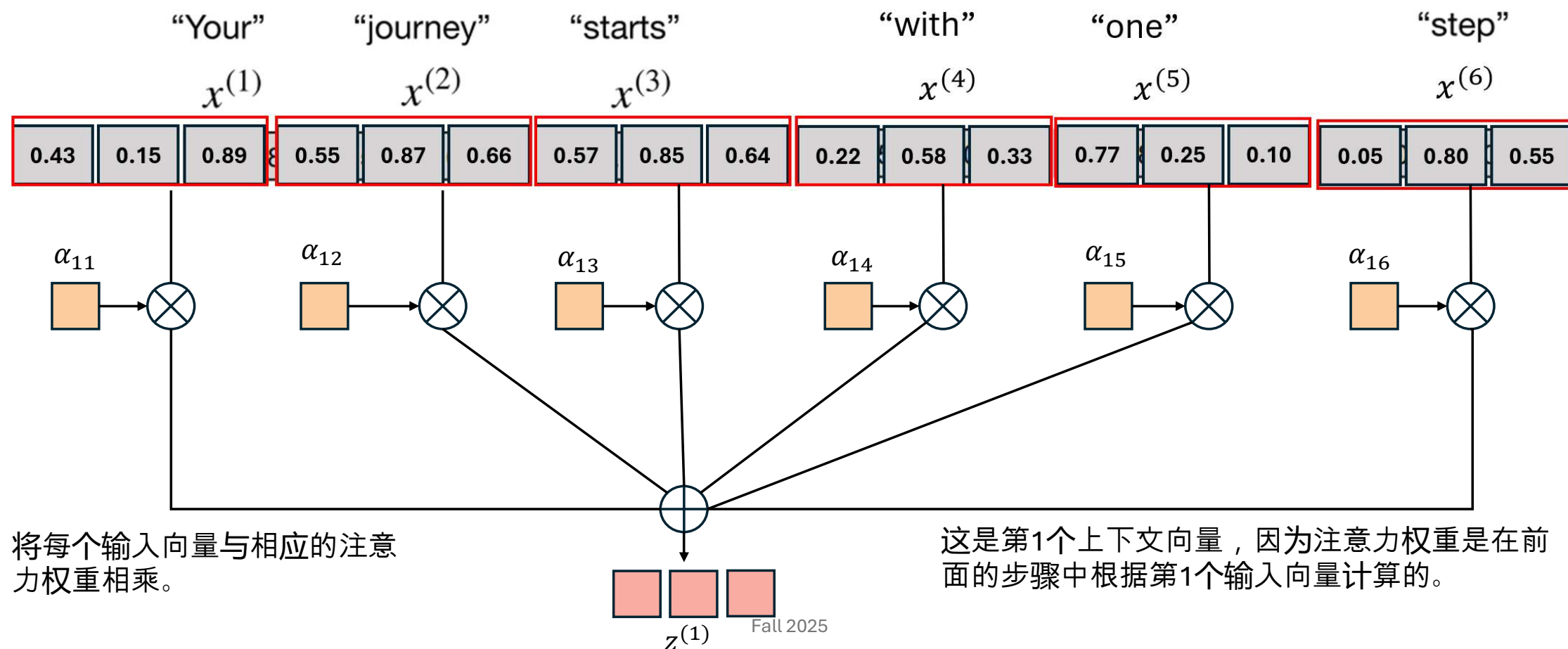
Softmax 计算器: [Softmax Function, Calculator and Formula](#)

- 计算出 $x^{(1)}$ 的注意力分数 ω 和注意力权重 α ，保留小数点后1位



题目3

计算”Your”的上下文向量(Context vector)



自注意力机制（简化版）

• 计算“You”的上下文向量

注意力权重 1×7

词向量矩阵 7×3

上下文向量 1×3

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14

@

0.43	0.15	0.89
0.55	0.87	0.66
0.57	0.85	0.64
0.22	0.58	0.33
0.77	0.25	0.10
0.05	0.80	0.55

“Your”

=



“journey”

“starts”

“with”

“one”

“step”

自注意力机制（简化版）

• 计算“You”的上下文向量

注意力权重

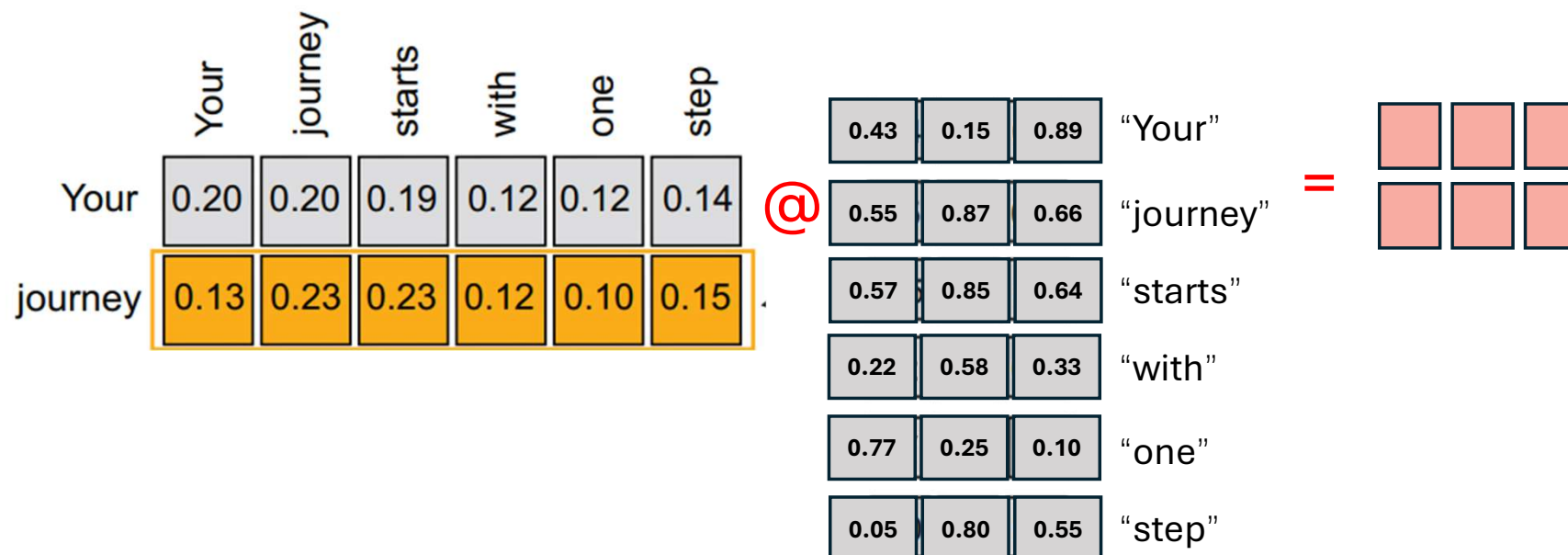
2x7

词向量矩阵

7x3

上下文向量

2x3



自注意力机制（简化版）

- 计算所有token的上下文向量

注意力权重

6x6

词向量矩阵

6x3

上下文向量

6x3

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

0.43	0.15	0.89	"Your"
0.55	0.87	0.66	"journey"
0.57	0.85	0.64	"starts"
0.22	0.58	0.33	"with"
0.77	0.25	0.10	"one"
0.05	0.80	0.55	"step"

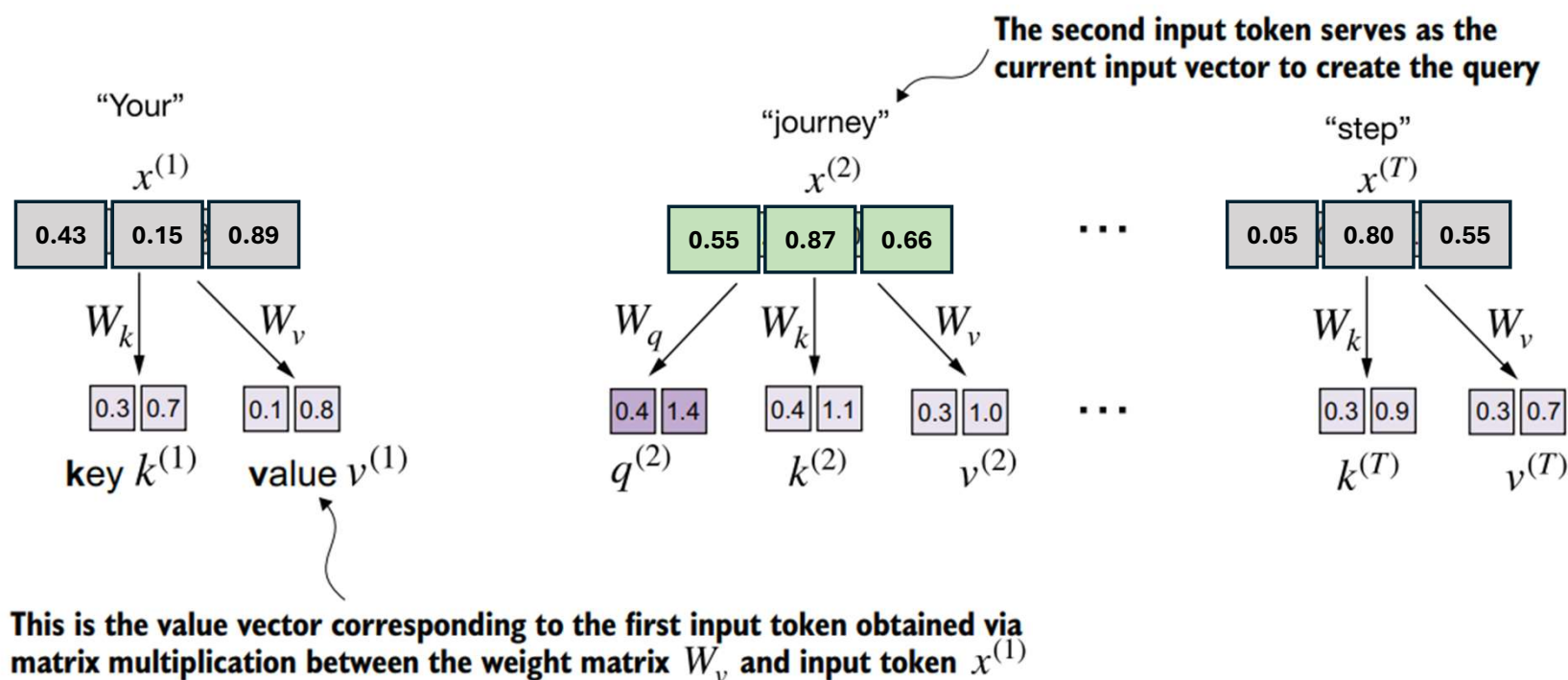
自注意力机制（简化版）

- 计算所有token的上下文向量

	Your	journey	starts	with	one	step	
Your	0.20	0.20	0.19	0.12	0.12	0.14	→ 和为1
journey	0.13	0.23	0.23	0.12	0.10	0.15	→ 和为1
starts	0.13	0.23	0.23	0.12	0.11	0.15	→ 和为1
with	0.14	0.20	0.20	0.14	0.12	0.17	→ 和为1
one	0.15	0.19	0.19	0.13	0.18	0.12	→ 和为1
step	0.13	0.21	0.21	0.14	0.09	0.18	→ 和为1

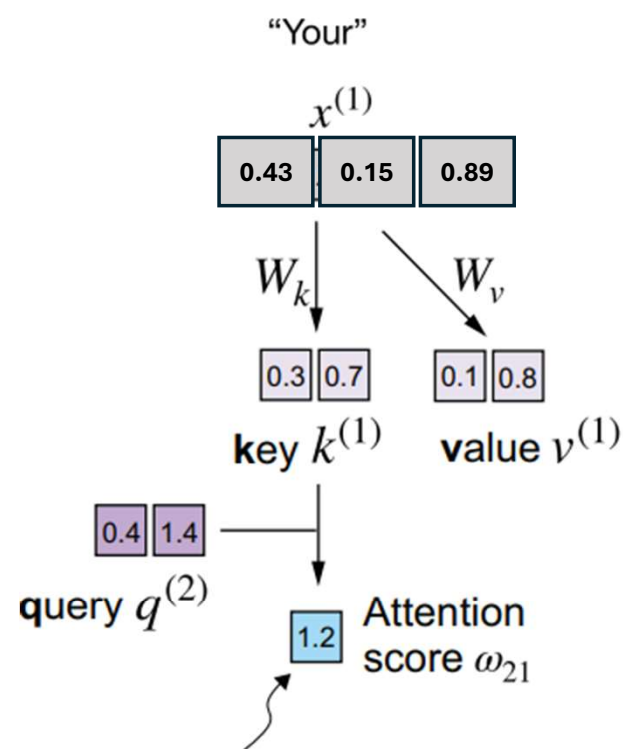
自注意力机制

- 通过引入三个可训练的权重矩阵 W_q 、 W_k 和 W_v 来逐步实现自注意力机制。这三个矩阵分别用于将token的词向量 x 投影到Query(查询)、Key(键)和Value(值)向量中

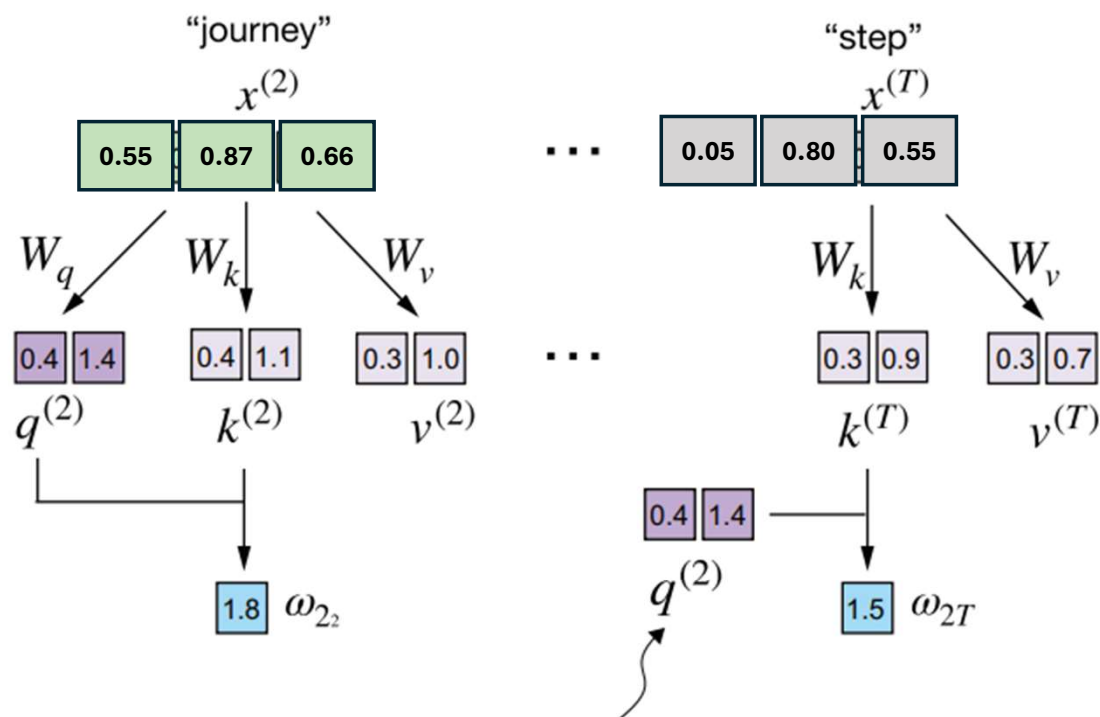


自注意力机制

- 计算“journey”的注意力分数



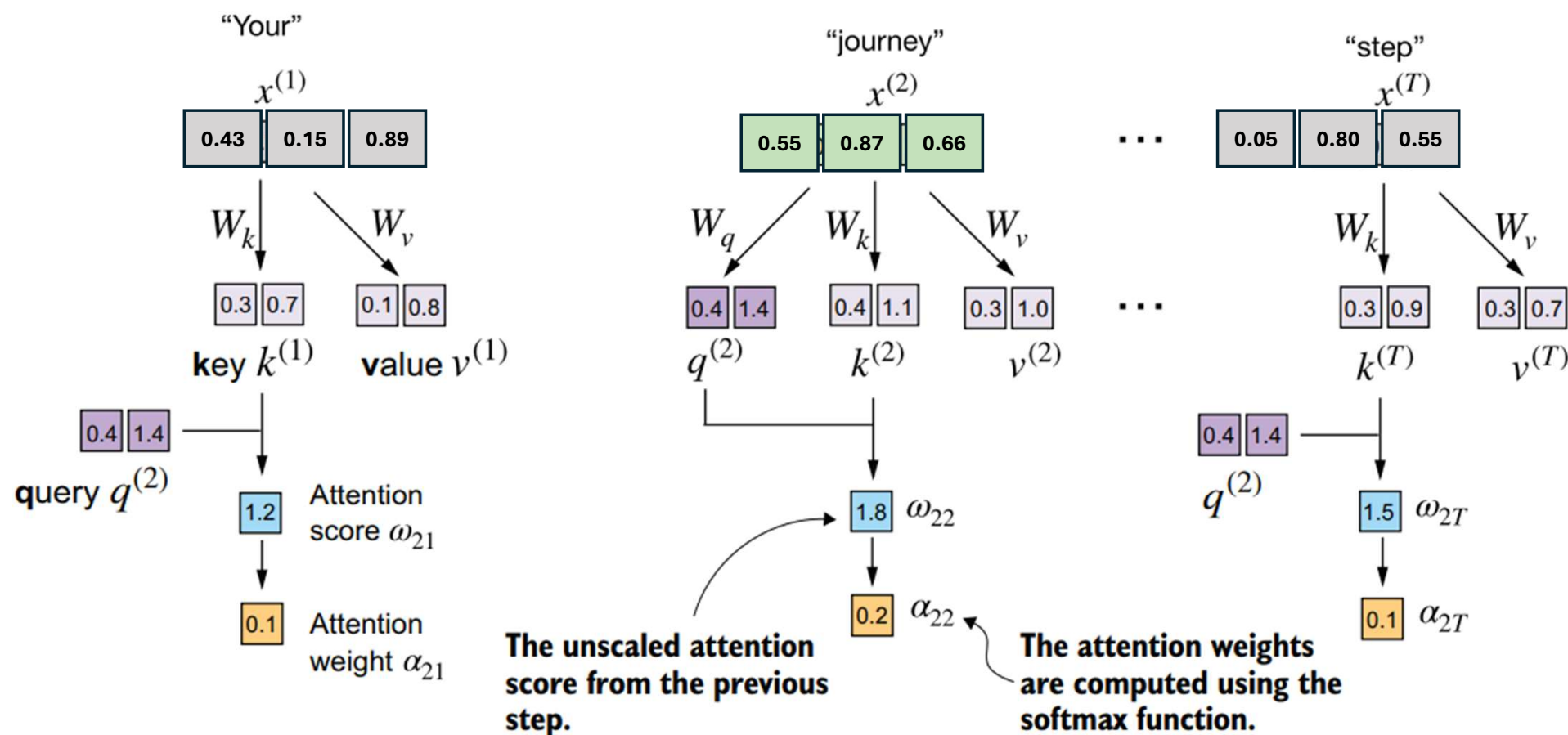
The unscaled attention score is computed as a dot product between the query and the key vectors.



Since we want to compute the context vector for the second input token, the query is derived from that second input token.

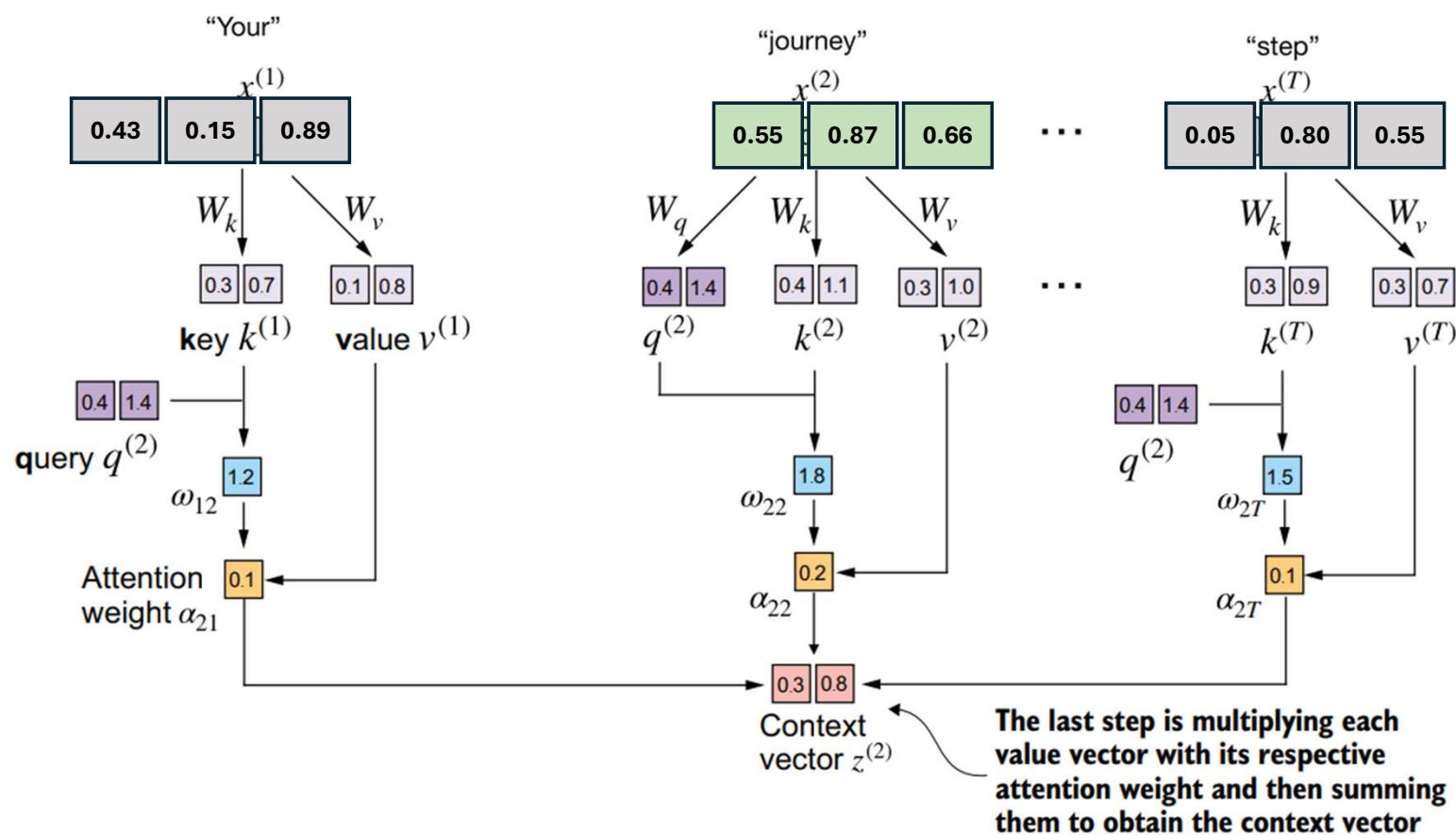
自注意力机制

- 计算“journey”的注意力权重



自注意力机制

- 计算“journey”的上下文向量

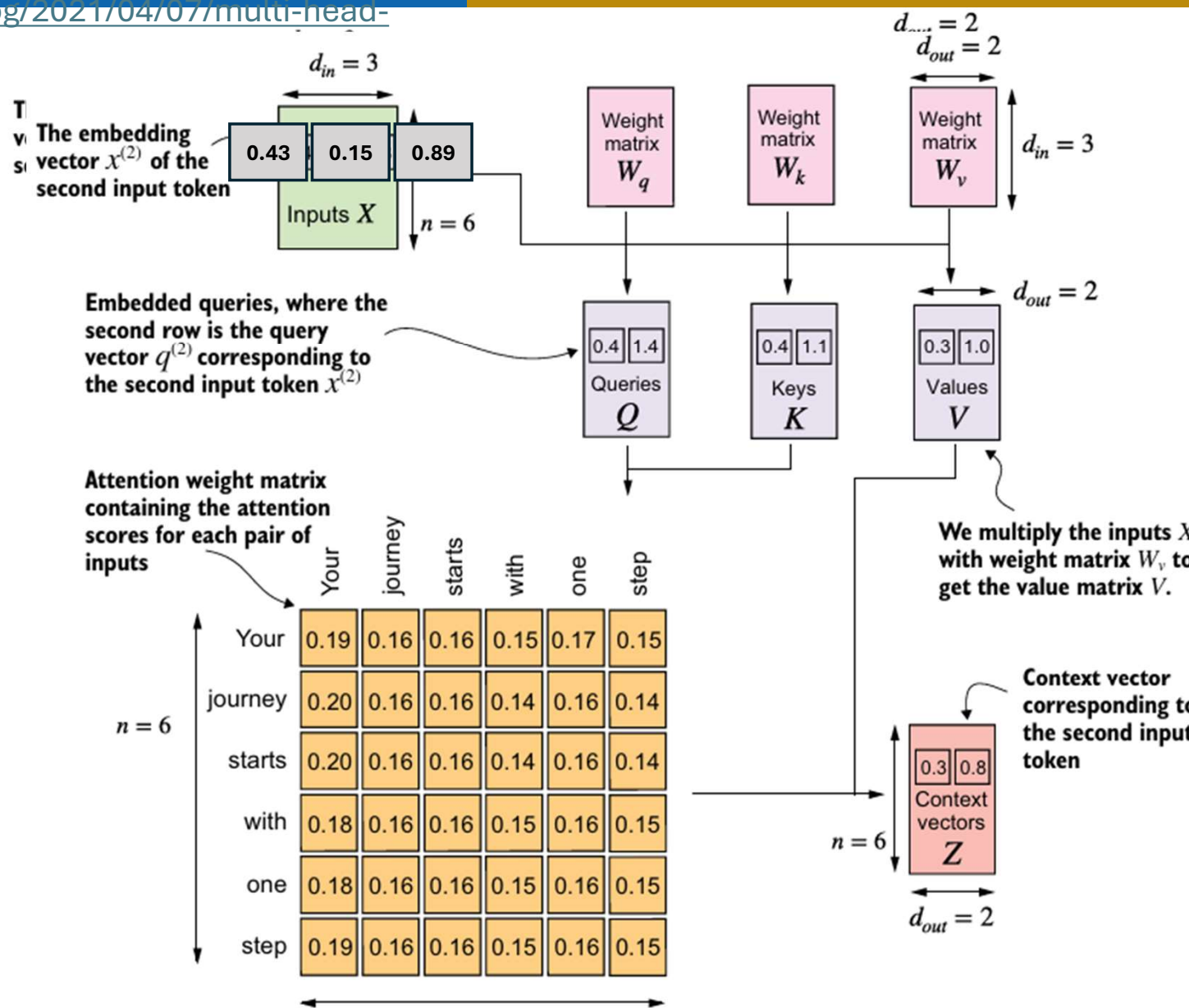


Why query, key, and value?

- 让注意力可学习：权重矩阵 W_q 、 W_k 和 W_v 让模型能学到最佳的 Query、Key、Value，而不是固定计算方式。
- 适应不同任务：不同任务需要不同的关注方式，直接用词向量不够灵活。
- 支持多头注意力：多个 让模型能同时关注输入的不同方面。
- 提高表达能力：矩阵变换可以优化 Query 和 Key 的匹配，让注意力计算更精准。

如果直接用词向量，注意力权重永远保持不变，
注意力机制就会失去学习能力，无法适应不同任务。

自注意力机制



自注意力机制



I_1

I



I_2

ate



I_3

an



I_4

apple

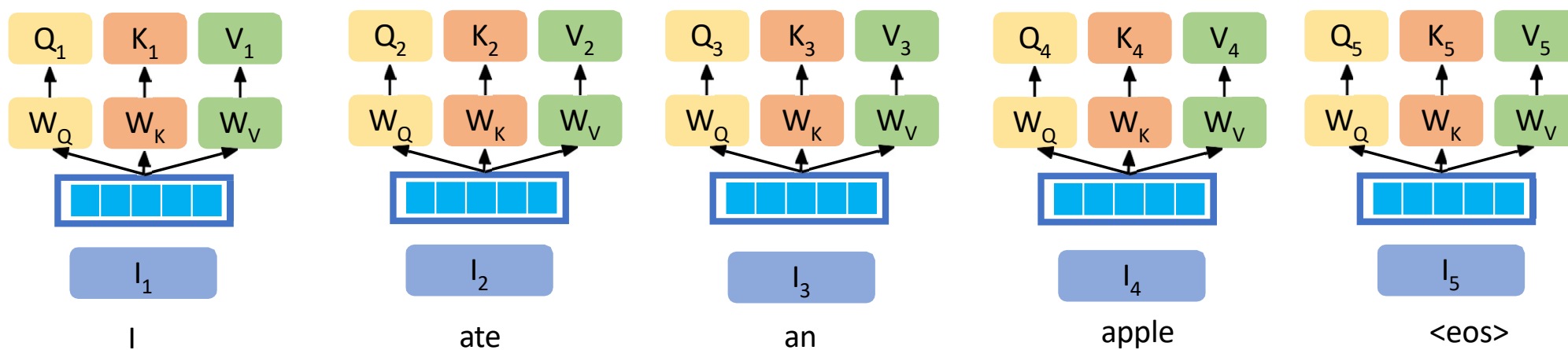


I_5

<eos>

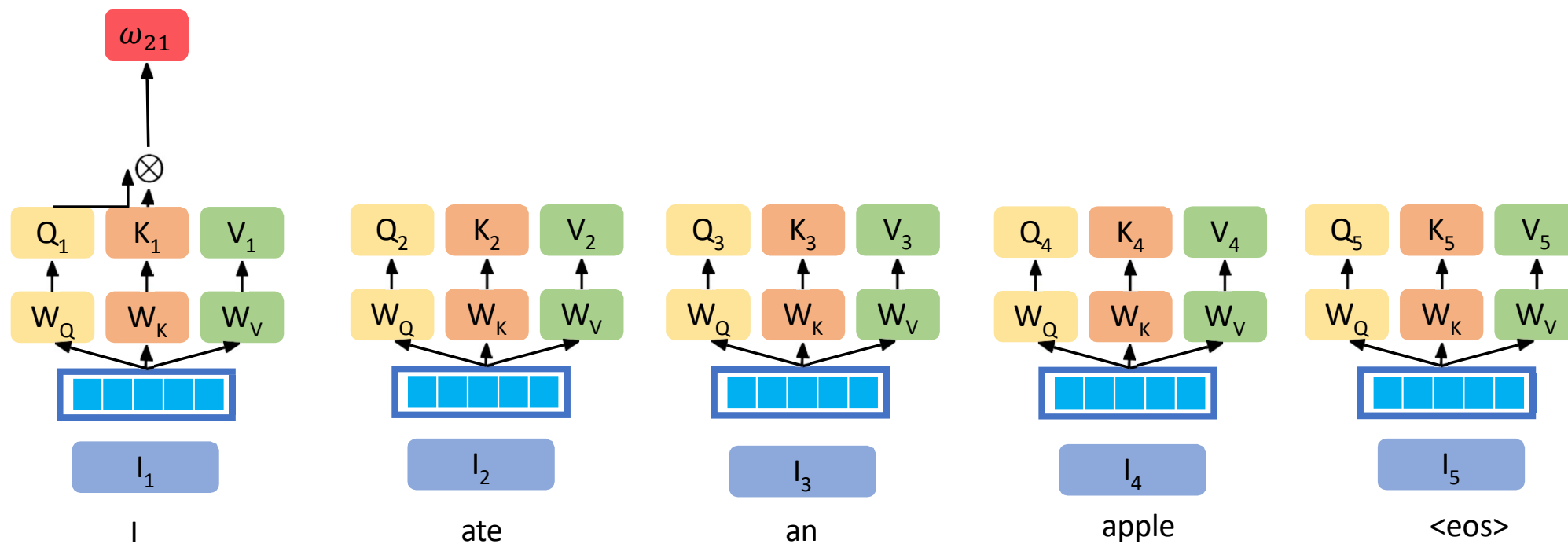
Dimensions across QKV have been dropped for brevity

注意力机制



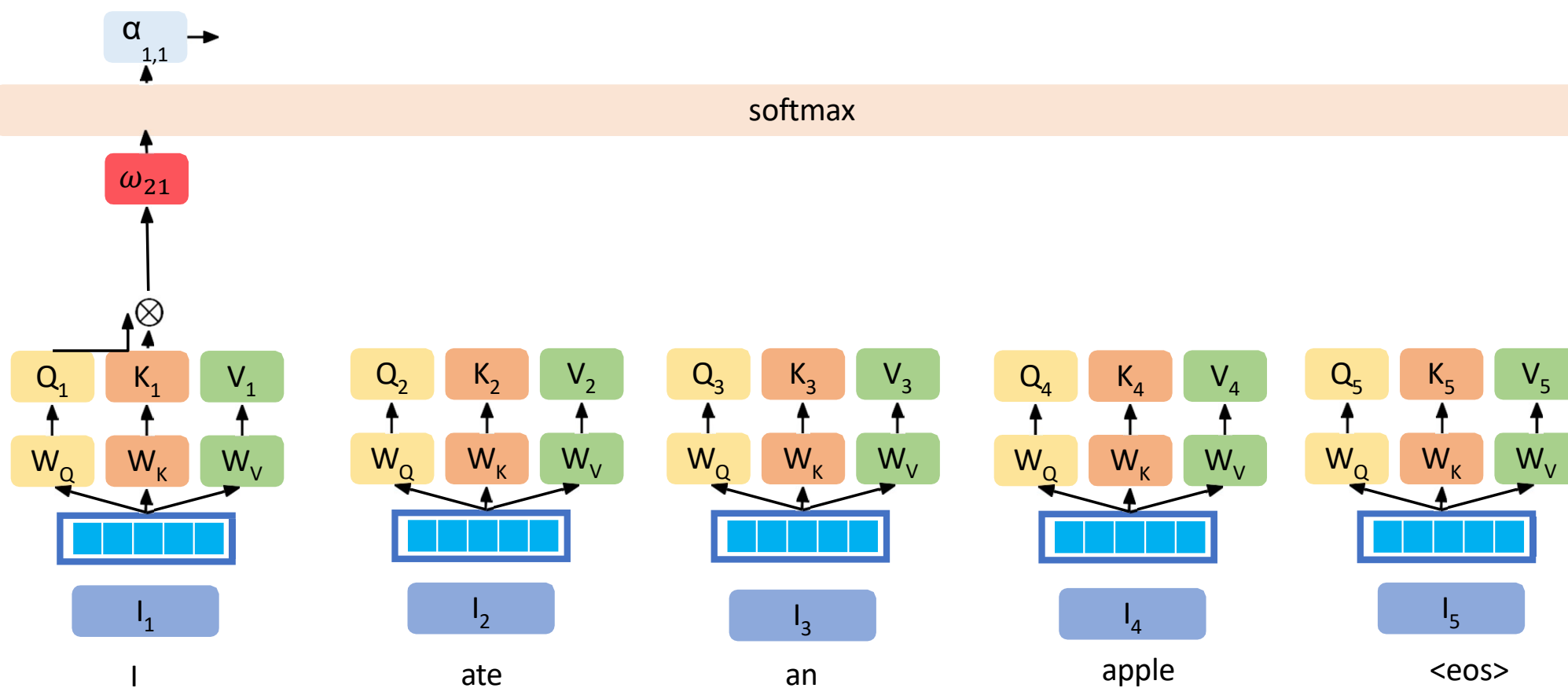
Dimensions across QKV have been dropped for brevity

注意力机制



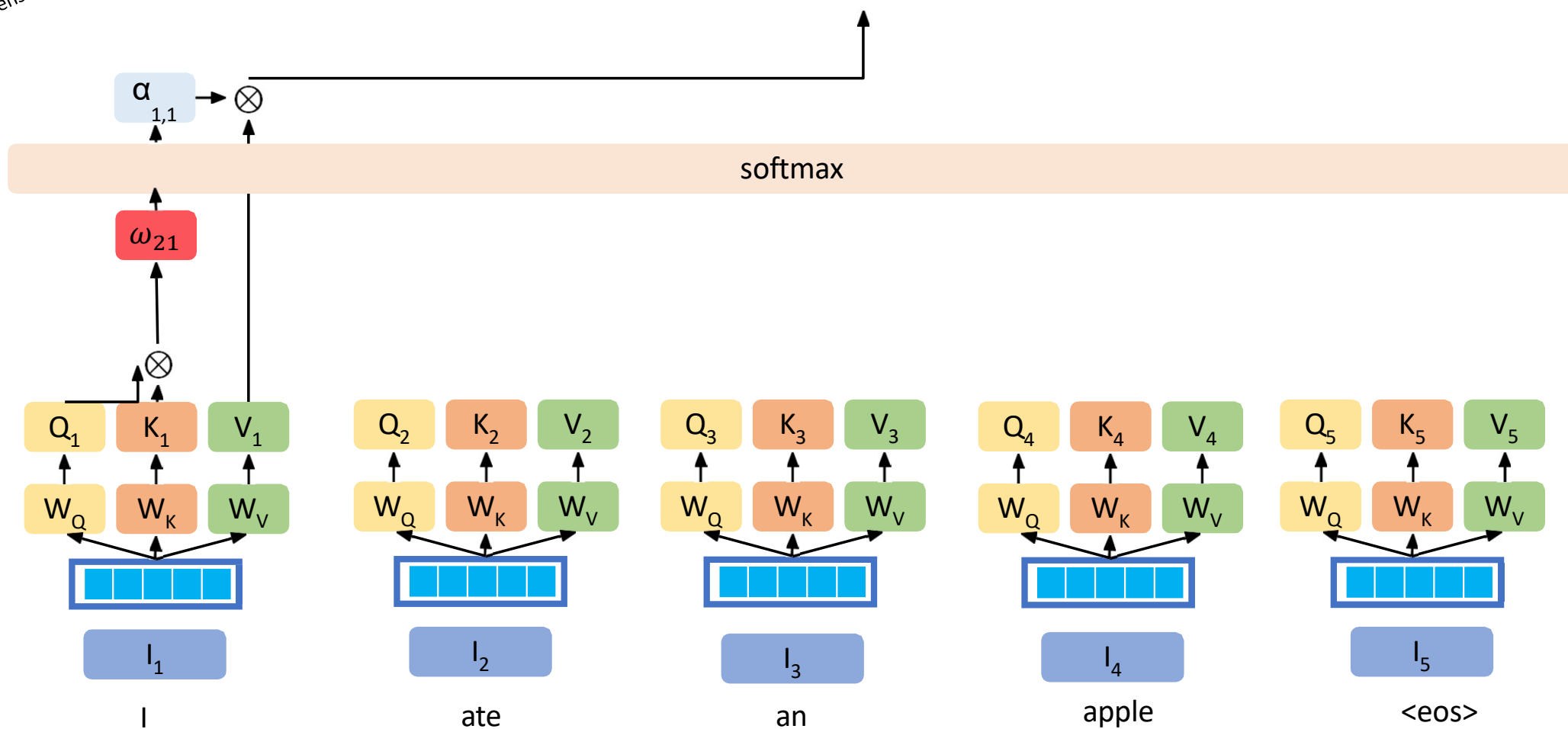
注意力机制

Dimensions across QKV have been dropped for brevity



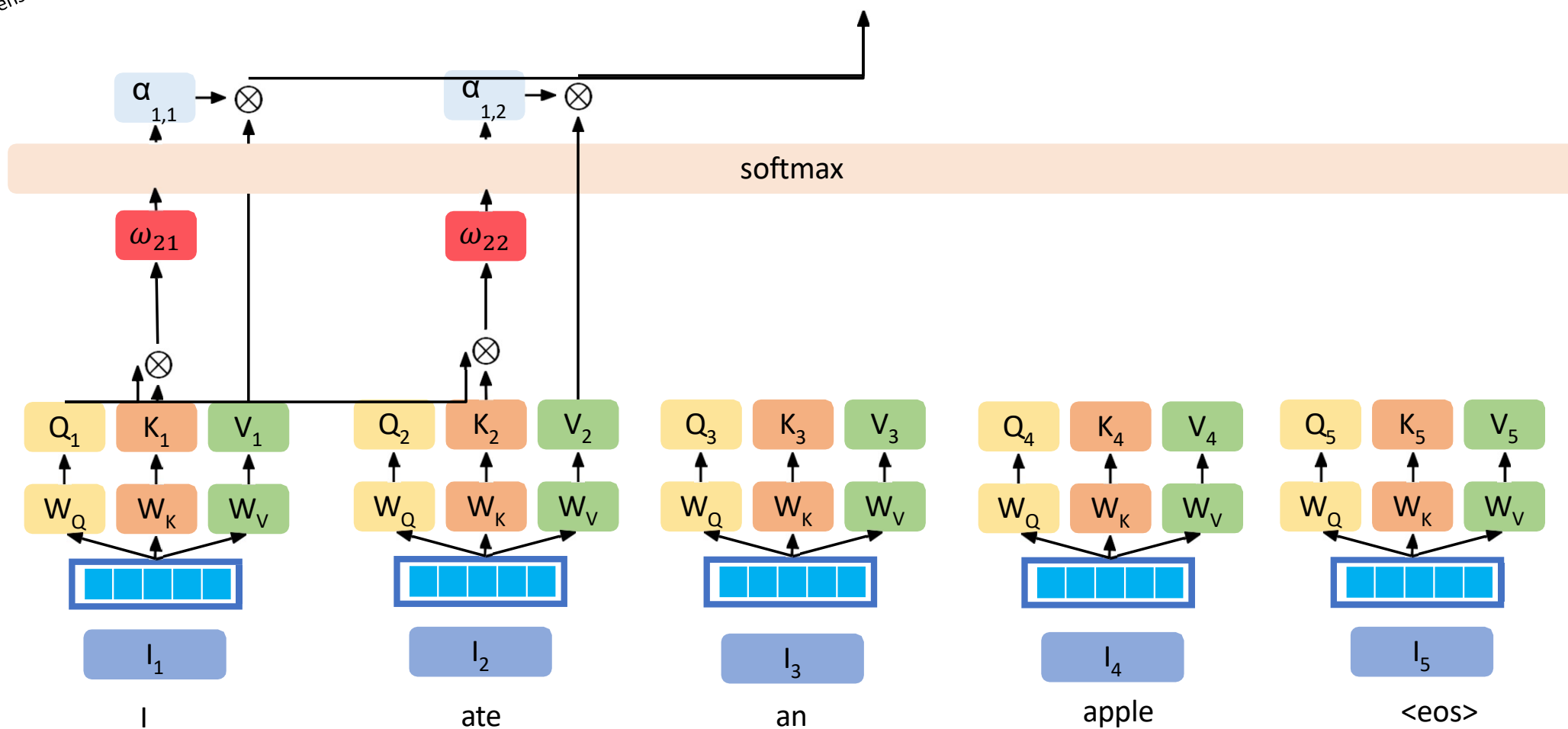
注意力机制

Dimensions across QKV have been dropped for brevity



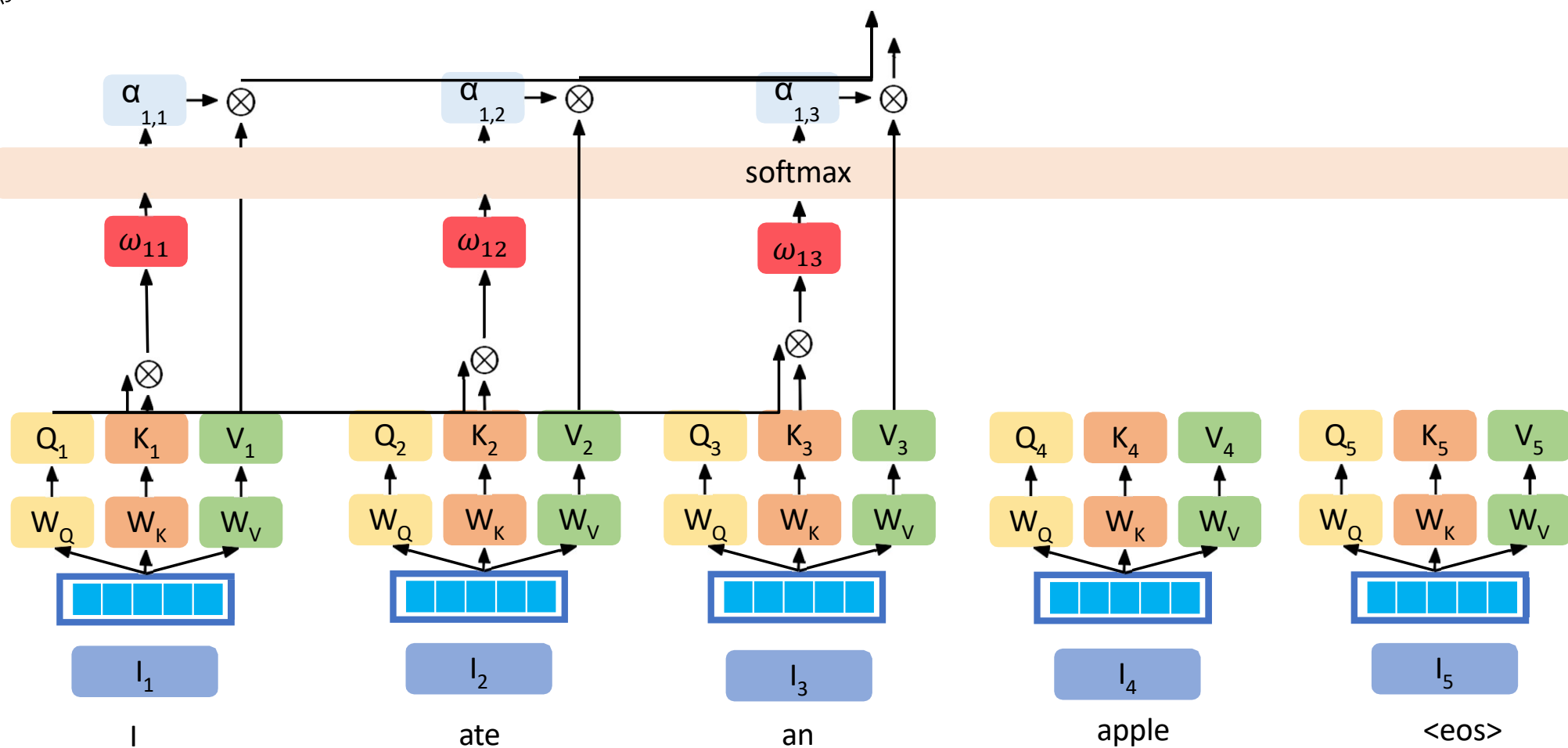
注意力机制

Dimensions across QKV have been dropped for brevity



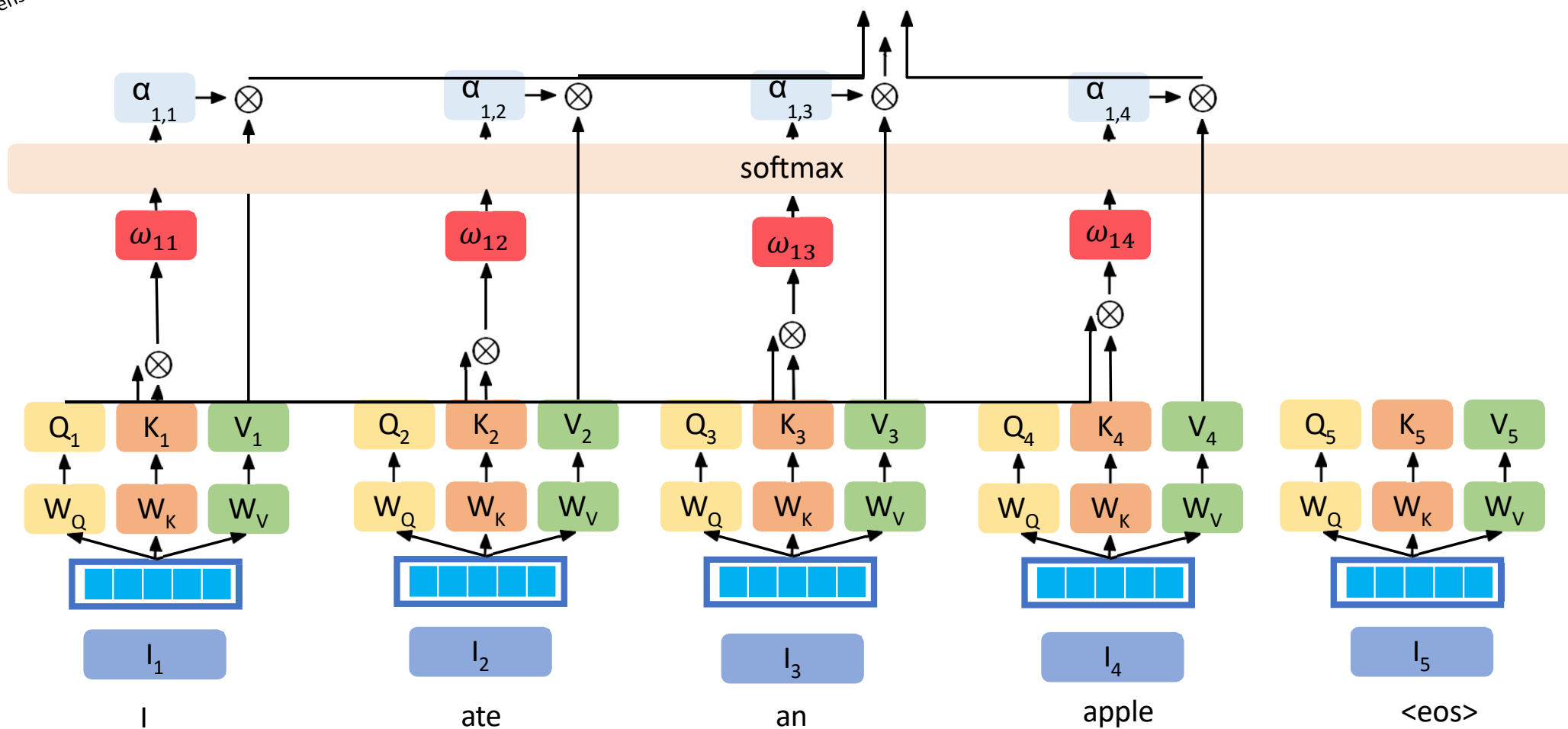
注意力机制

Dimensions across QKV have been dropped for brevity



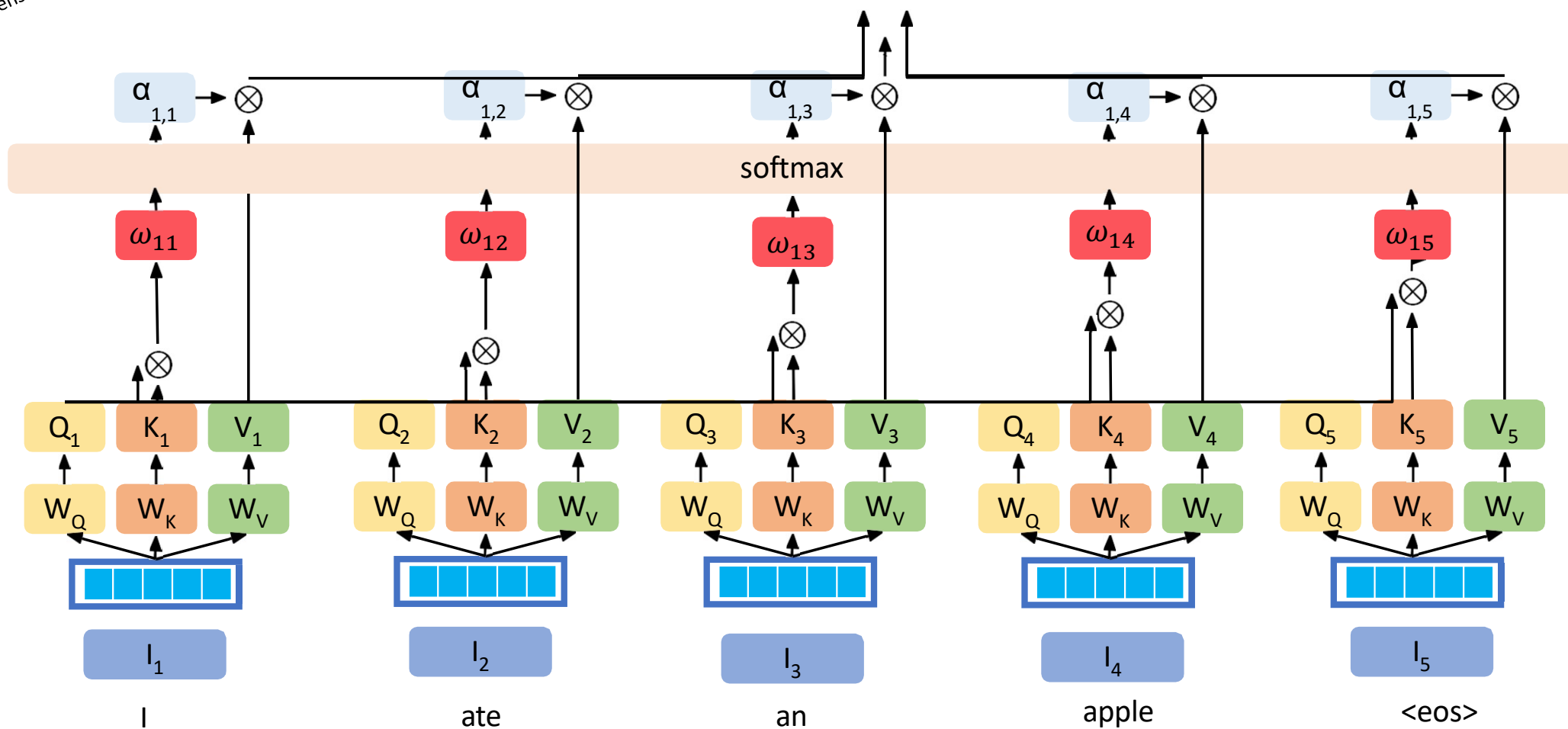
注意力机制

Dimensions across QKV have been dropped for brevity



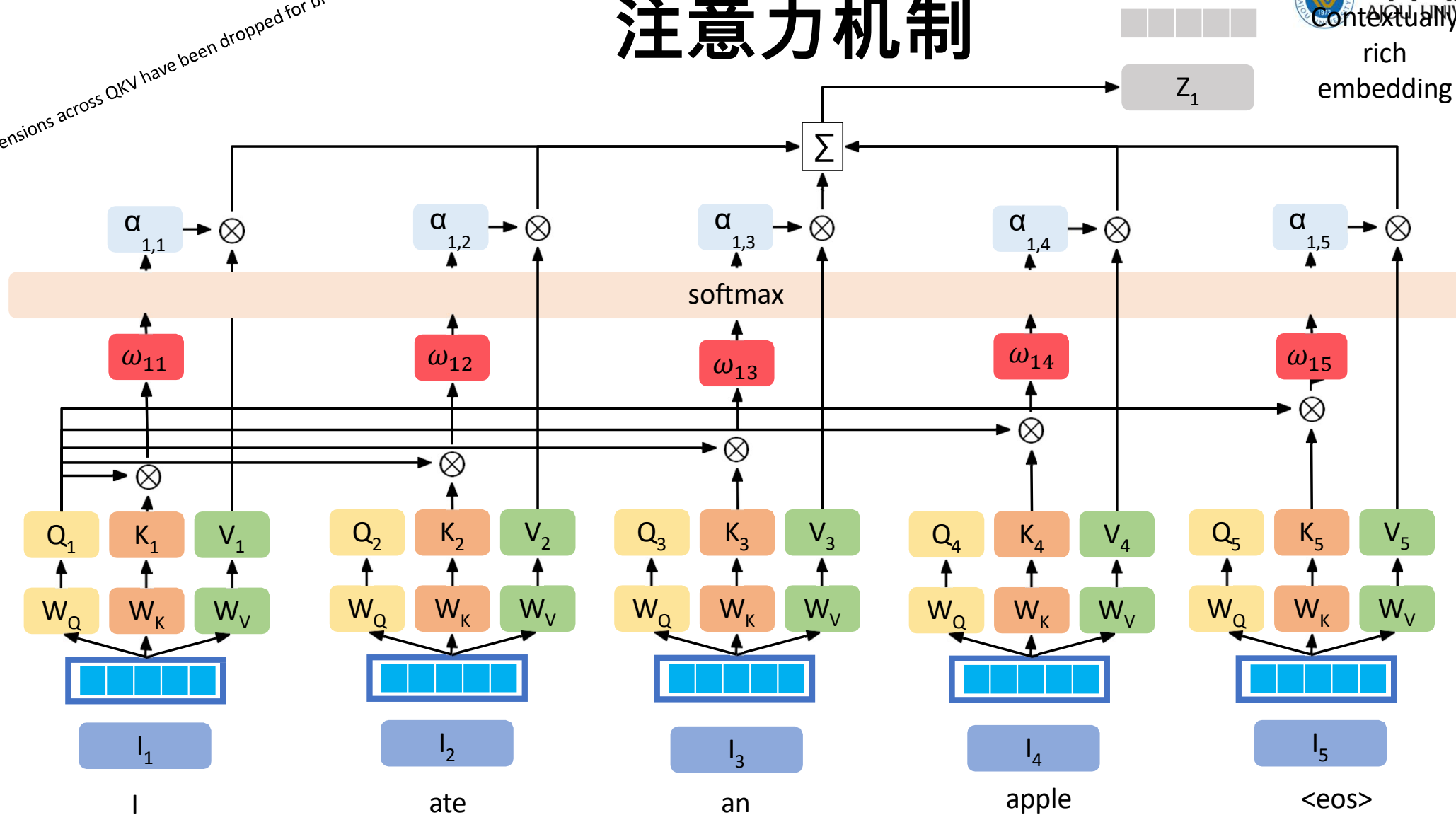
注意力机制

Dimensions across QKV have been dropped for brevity



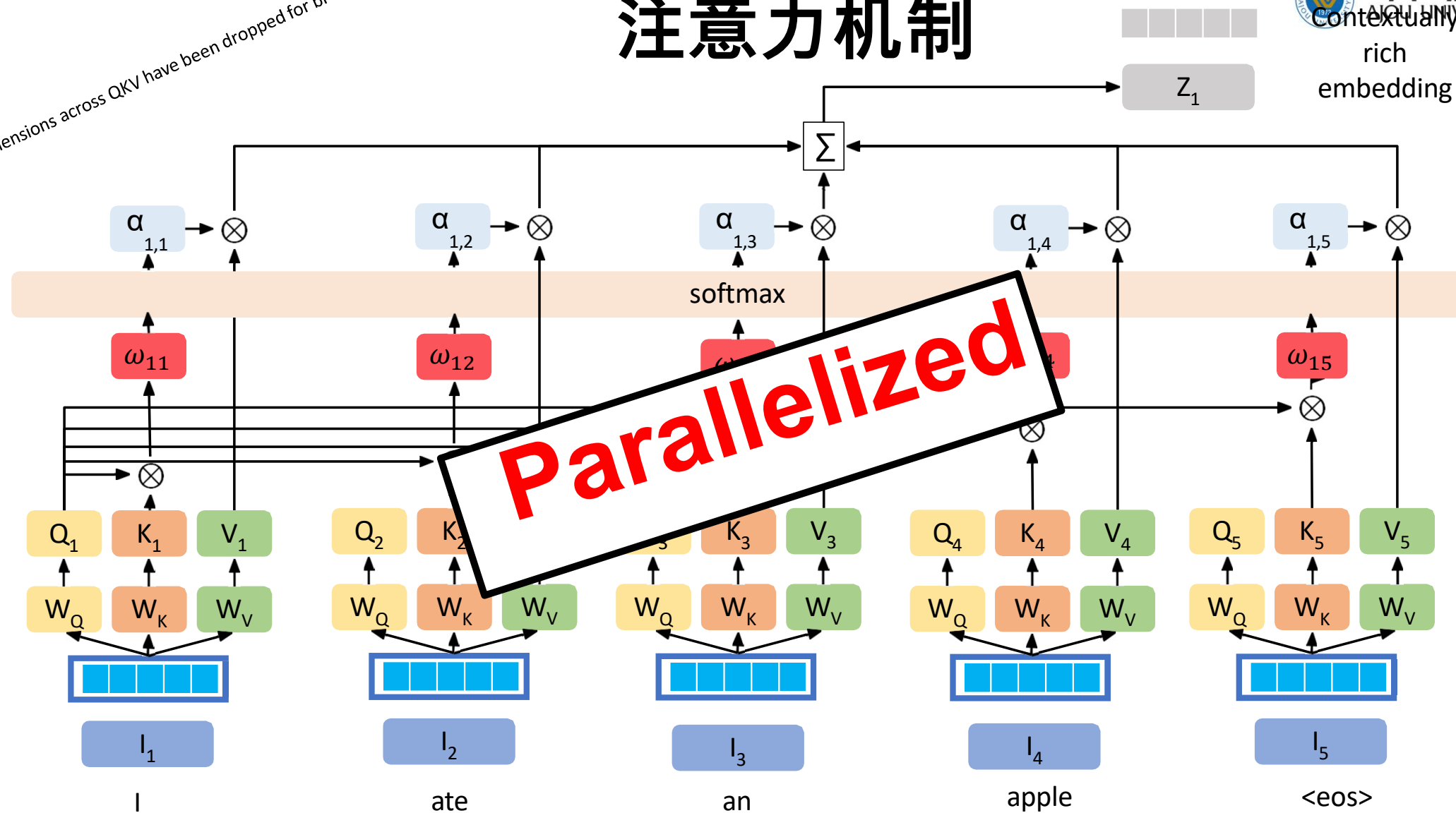
注意力机制

Dimensions across QKV have been dropped for brevity



注意力机制

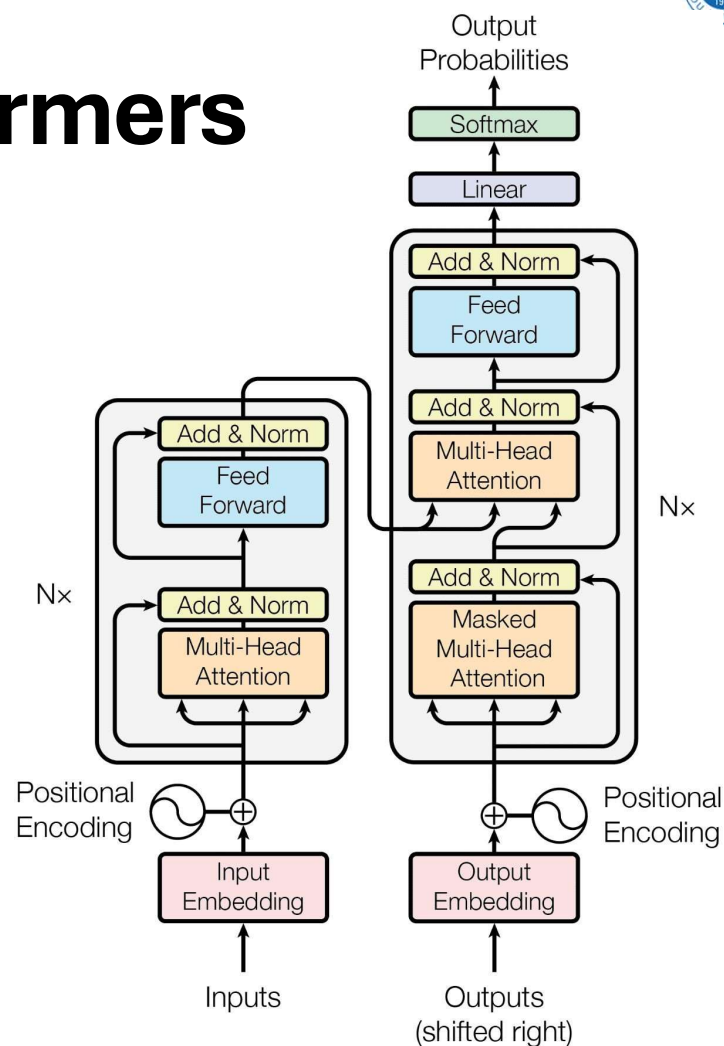
Dimensions across QKV have been dropped for brevity



Transformers

- ✓ Tokenization
- ✓ 输入嵌入
- ✓ 位置编码
- ✓ Query, Key, & Value
- ✓ 注意力机制
 - 自注意力
 - 多头注意力
 - Feed Forward
 - Add & Norm
 - 编码器s

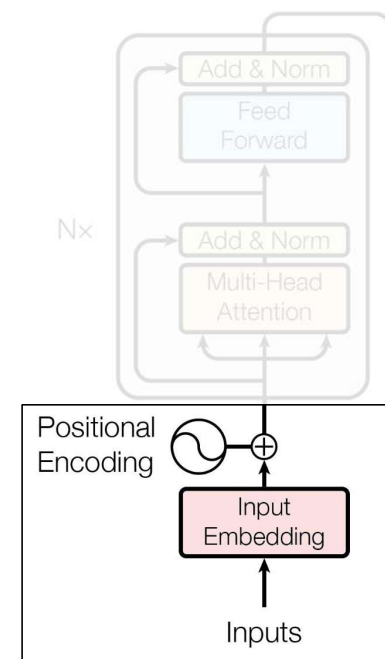
- Masked 注意力机制
- 编码器-解码器注意力机制
- Linear
- Softmax
- 解码器s
- 编码器-解码器 Models



Poll 1 - @1581

关于注意力机制，下列哪些说法是正确的？

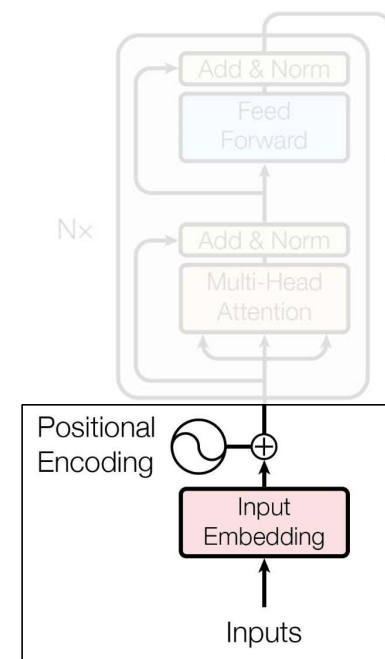
- a. 对于输入 I_2 ，计算注意力权重时，应使用 **key k_2** 和所有的 **query**（查询向量）
- b. 对于输入 I_2 ，计算注意力权重时，应使用 **query q_2** 和所有的 **key**（键向量）
- c. 我们对 QK^T 的结果进行缩放，是为了将注意力权重限制在 $[0, 1]$ 范围内
- d. 我们对 QK^T 的结果进行缩放，是为了提高数值稳定性



Poll 1 - @1581

关于注意力机制，下列哪些说法是正确的？

- a. 对于输入 I_2 ，计算注意力权重时，应使用 **key k_2** 和所有的 **query**（查询向量）
- b. 对于输入 I_2 ，计算注意力权重时，应使用 **query q_2** 和所有的 **key**（键向量）
- c. 我们对 QK^T 的结果进行缩放，是为了将注意力权重限制在 $[0, 1]$ 范围内
- d. 我们对 QK^T 的结果进行缩放，是为了提高数值稳定性



谢谢！ 欢迎提问！

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr



大语言模型及应用

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr

自注意力机制

2 0 0 0 2 1

x_1

I

0 1 2 0 0 0

x_2

ate

0 0 1 1 0 1

x_3

an

2 0 0 1 0 1

x_4

apple

注意力机制

W_Q

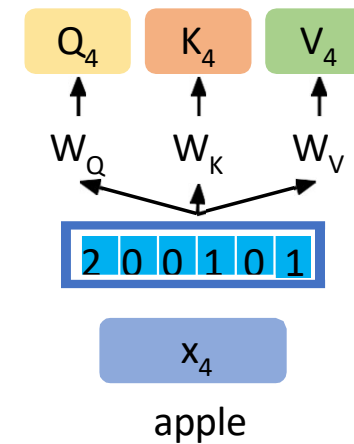
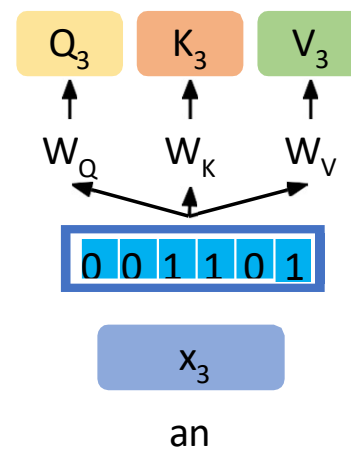
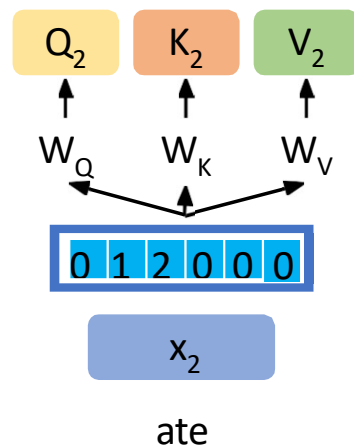
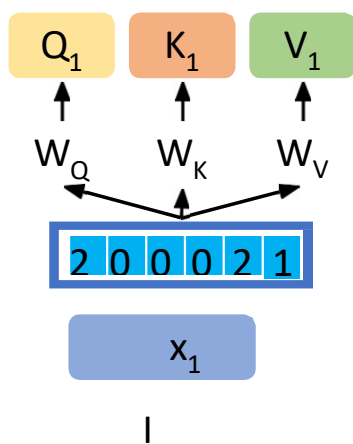
1	1	0	0	0	0
0	1	0	1	0	0
0	0	1	0	1	1

W_K

0	0	1	0	0	0
0	1	0	0	0	0
1	0	0	0	0	-1

W_V

10	0	0	0	0	0
0	0	0	10	0	0
0	10	0	0	0	0



练习

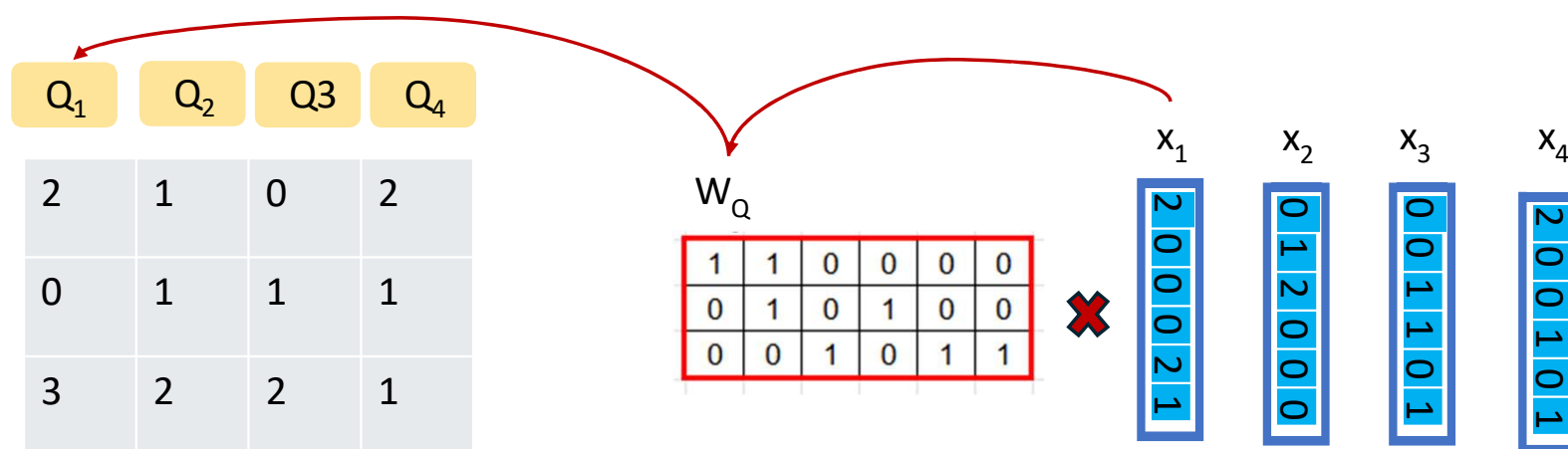
3人一组，例如组员A,B,C

A 根据 W_Q ，分别计算出每个词的Q,即 Q_1, Q_2, Q_3, Q_4 .

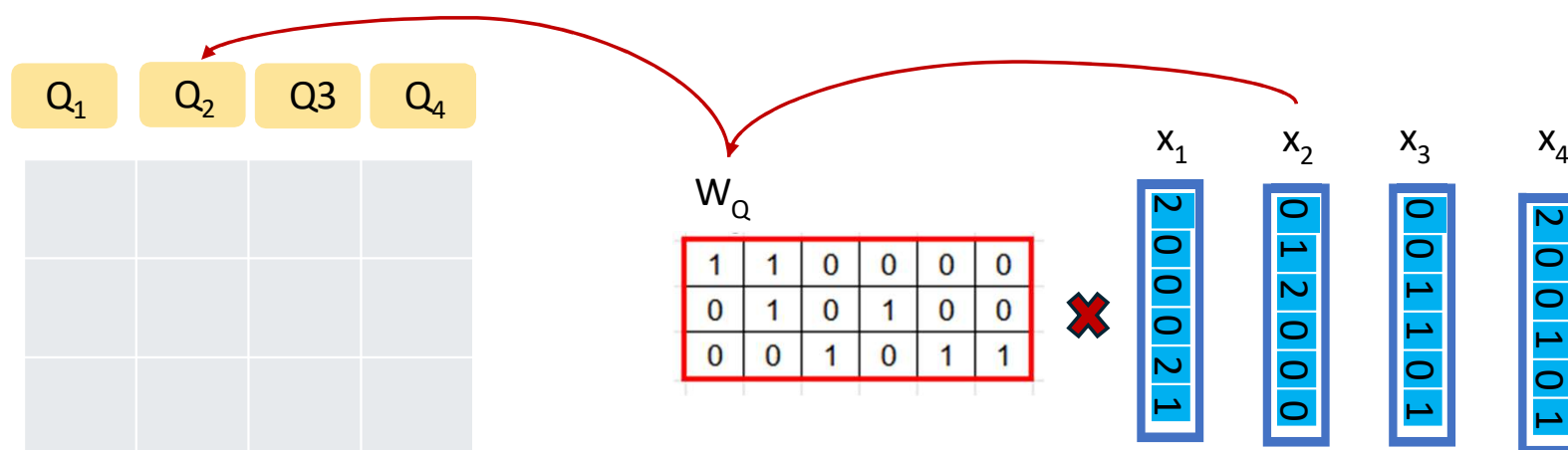
B根据 W_K ，分别计算出每个词的K,即 K_1, K_2, K_3, K_4 .

C 根据 W_Q ，分别计算出每个词的V,即 V_1, V_2, V_3, V_4 .

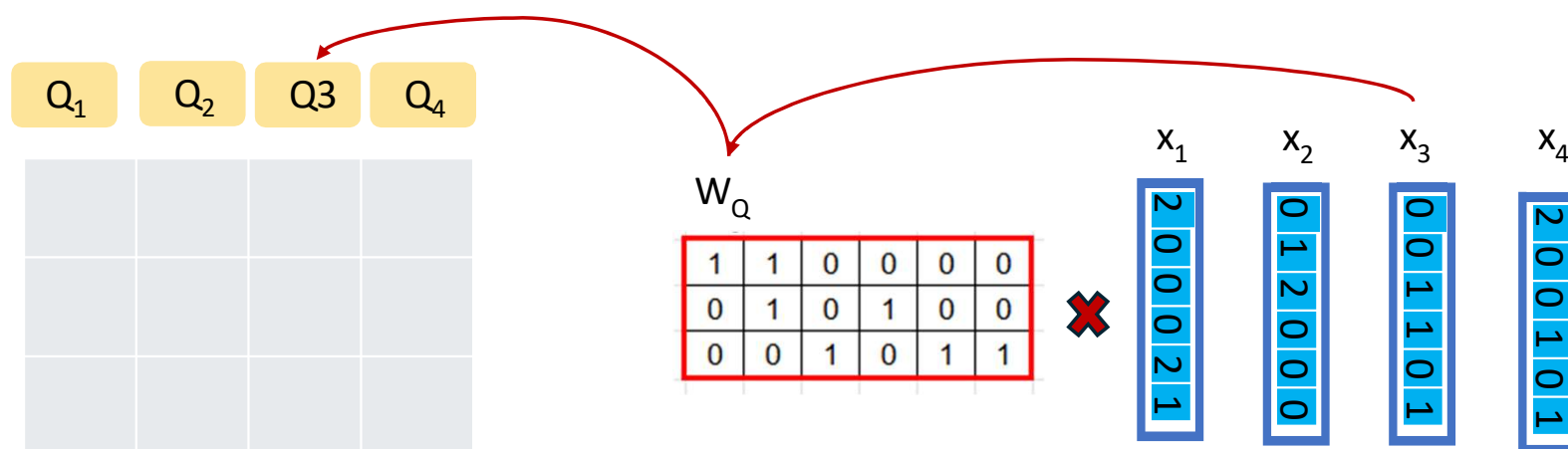
- 在下表中写出每个词的Q,即 Q_1 , Q_2 , Q_3 , Q_4 .



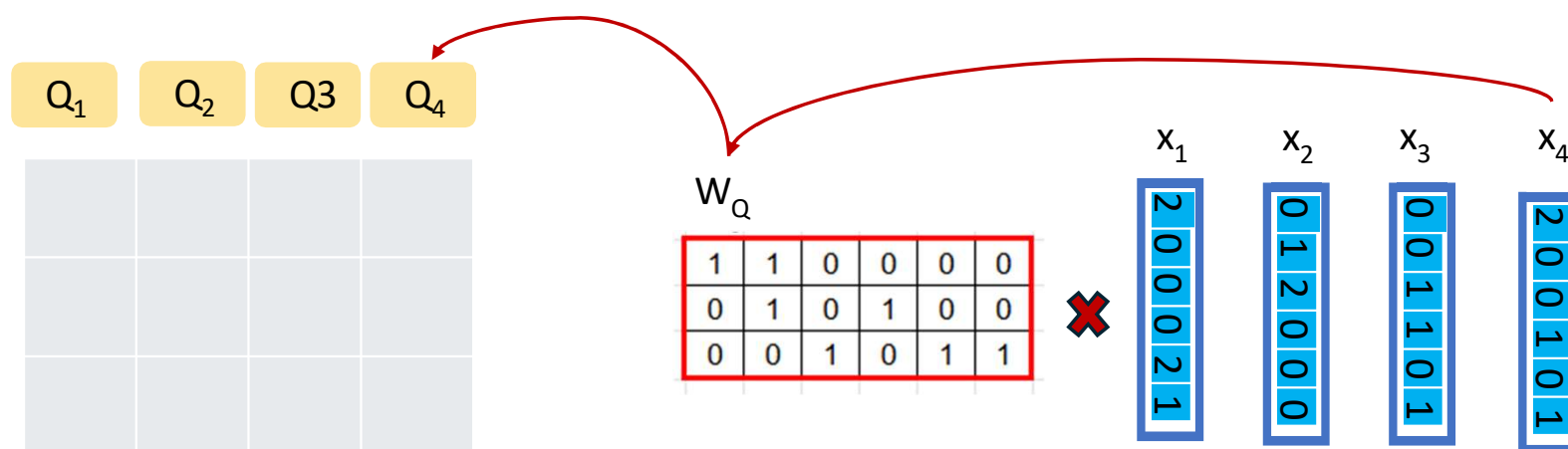
- 在下表中写出每个词的Q,即 Q_1 , Q_2 , Q_3 , Q_4 .



- 在下表中写出每个词的Q,即 Q_1 , Q_2 , Q_3 , Q_4 .



- 在下表中写出每个词的Q, 即 Q_1, Q_2, Q_3, Q_4 .



- 在下表中写出每个词的Q,即 Q_1 , Q_2 , Q_3 , Q_4 .

Q_1	Q_2	Q_3	Q_4

$$W_Q \times \begin{matrix} x_1 & x_2 & x_3 & x_4 \\ \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \\ 2 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} & \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} \end{matrix}$$

- 在下表中写出每个词的Q,即 Q_1 , Q_2 , Q_3 , Q_4 .

Q_1	Q_2	Q_3	Q_4
2	1	0	2
0	1	1	1
3	2	2	1

$$W_Q \times \begin{matrix} x_1 & x_2 & x_3 & x_4 \\ \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \\ 2 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} & \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} \end{matrix}$$

- 在下表中写出每个词的K,即 K_1 , K_2 , K_3 , K_4 .

K_1	K_2	K_3	K_4
0	2	1	0
0	1	0	0
1	0	-1	1

W_K

0	0	1	0	0	0
0	1	0	0	0	0
1	0	0	0	0	-1

×

x_1	x_2	x_3	x_4
2	0	0	1
0	1	1	0
0	2	1	0
0	0	0	1
2	0	0	1
1	0	0	1

- 在下表中写出每个词的K,即 K_1 , K_2 , K_3 , K_4 .

K_1	K_2	K_3	K_4
0	2	1	0
0	1	0	0
1	0	-1	1

W_K

0	0	1	0	0	0
0	1	0	0	0	0
1	0	0	0	0	-1

×

x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	1
0	0	1	0
2	0	0	1
1	0	1	1

- 在下表中写出每个词的V,即 V_1 , V_2 , V_3 , V_4 .

V_1	V_2	V_3	V_4
20	0	0	20
0	0	10	10
0	10	0	0

W_V

10	0	0	0	0	0
0	0	0	10	0	0
0	10	0	0	0	0

×

x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	0
0	0	1	1
2	0	0	1
1	0	1	1

- 在下表中写出每个词的V,即 V_1 , V_2 , V_3 , V_4 .

V_1	V_2	V_3	V_4
20	0	0	20
0	0	10	10
0	10	0	0

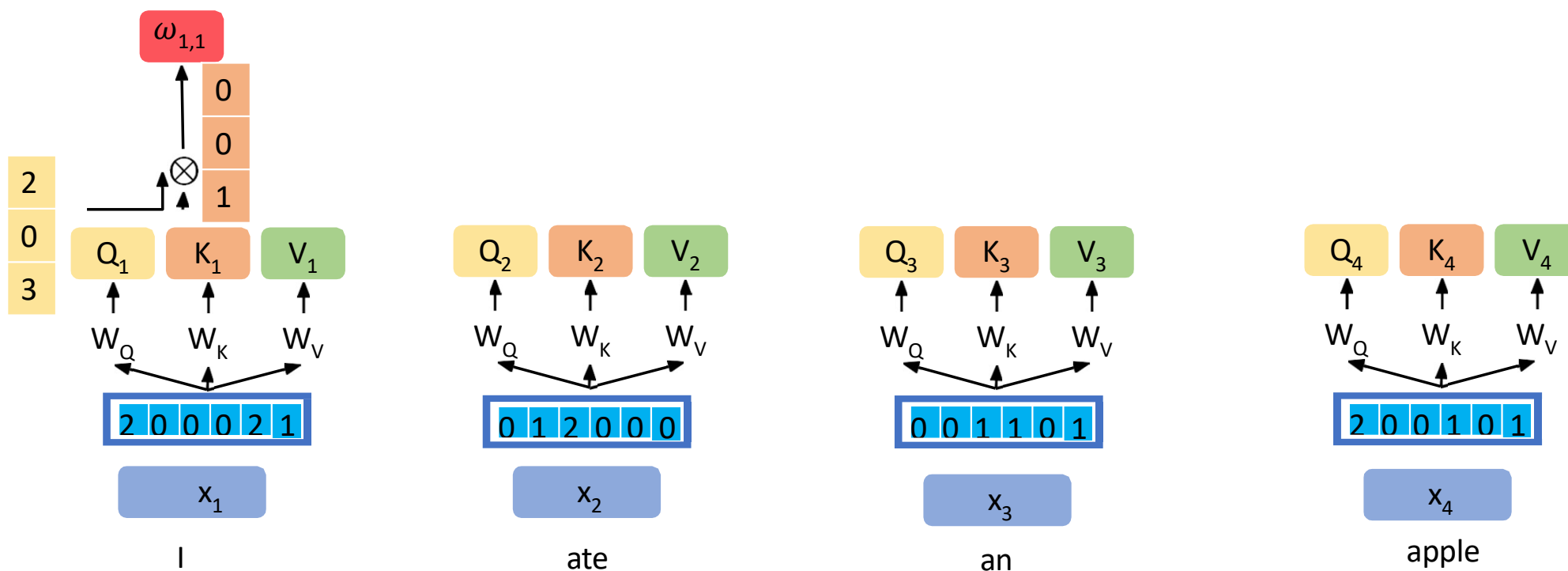
W_V

10	0	0	0	0	0
0	0	0	10	0	0
0	10	0	0	0	0

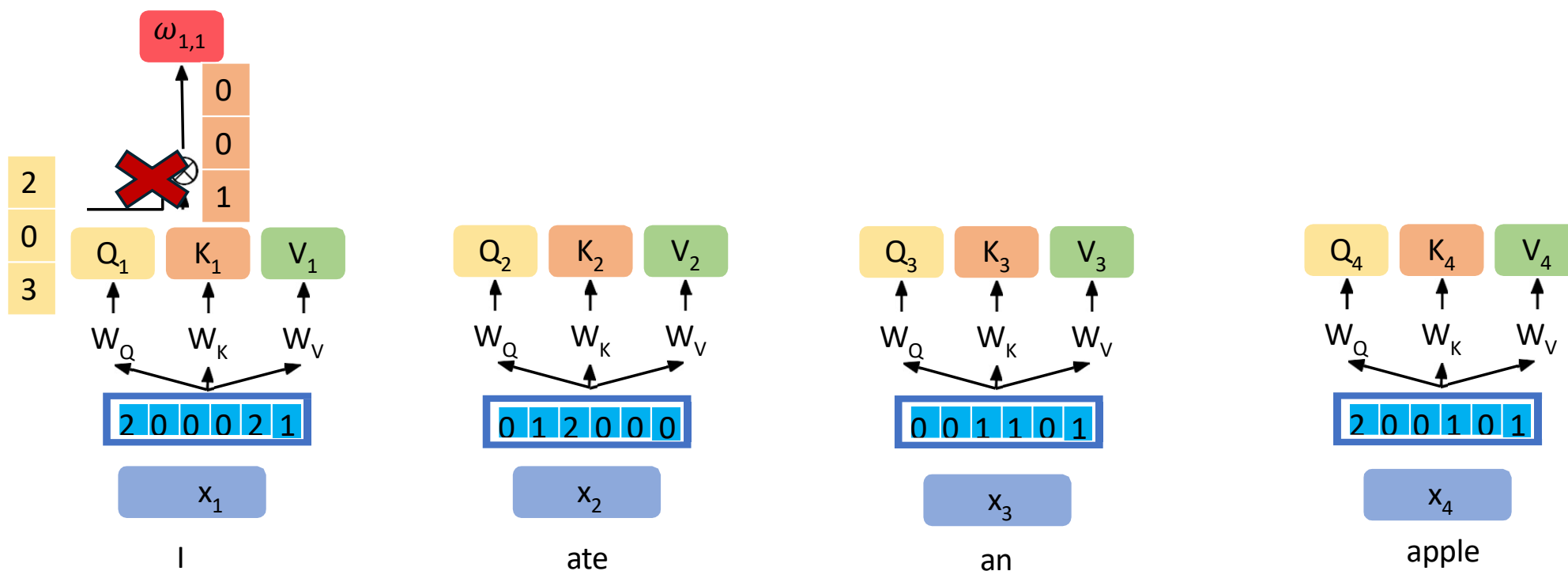
×

x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	0
0	0	1	1
2	0	0	1
1	0	1	1

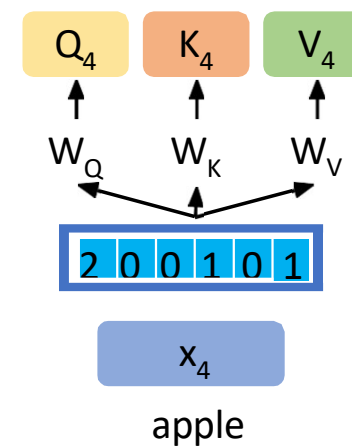
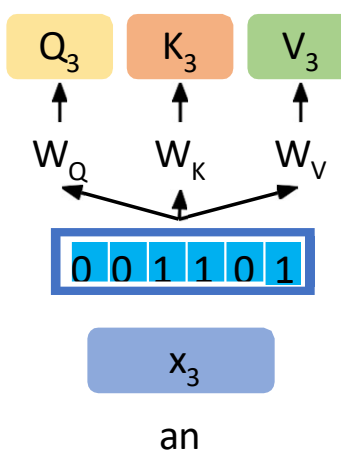
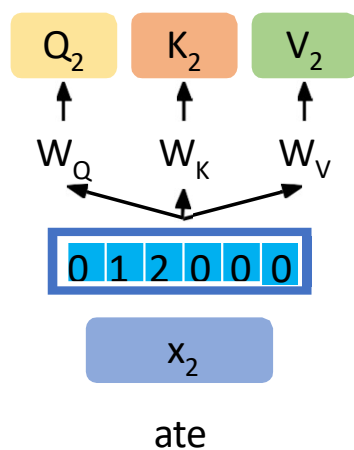
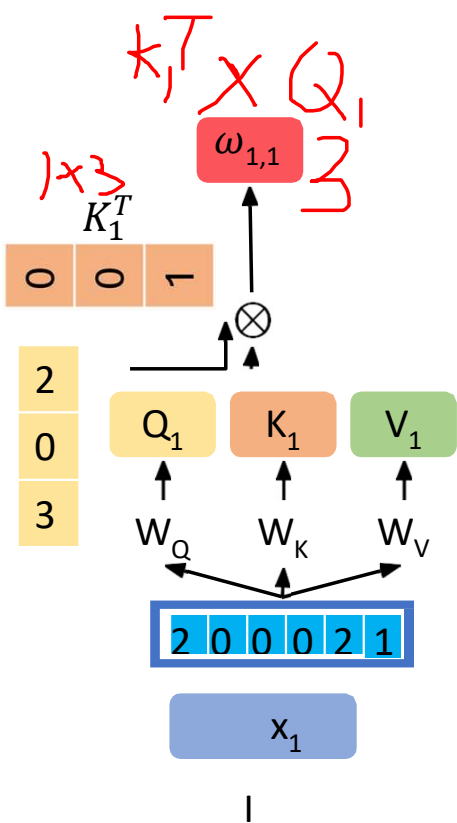
注意力机制



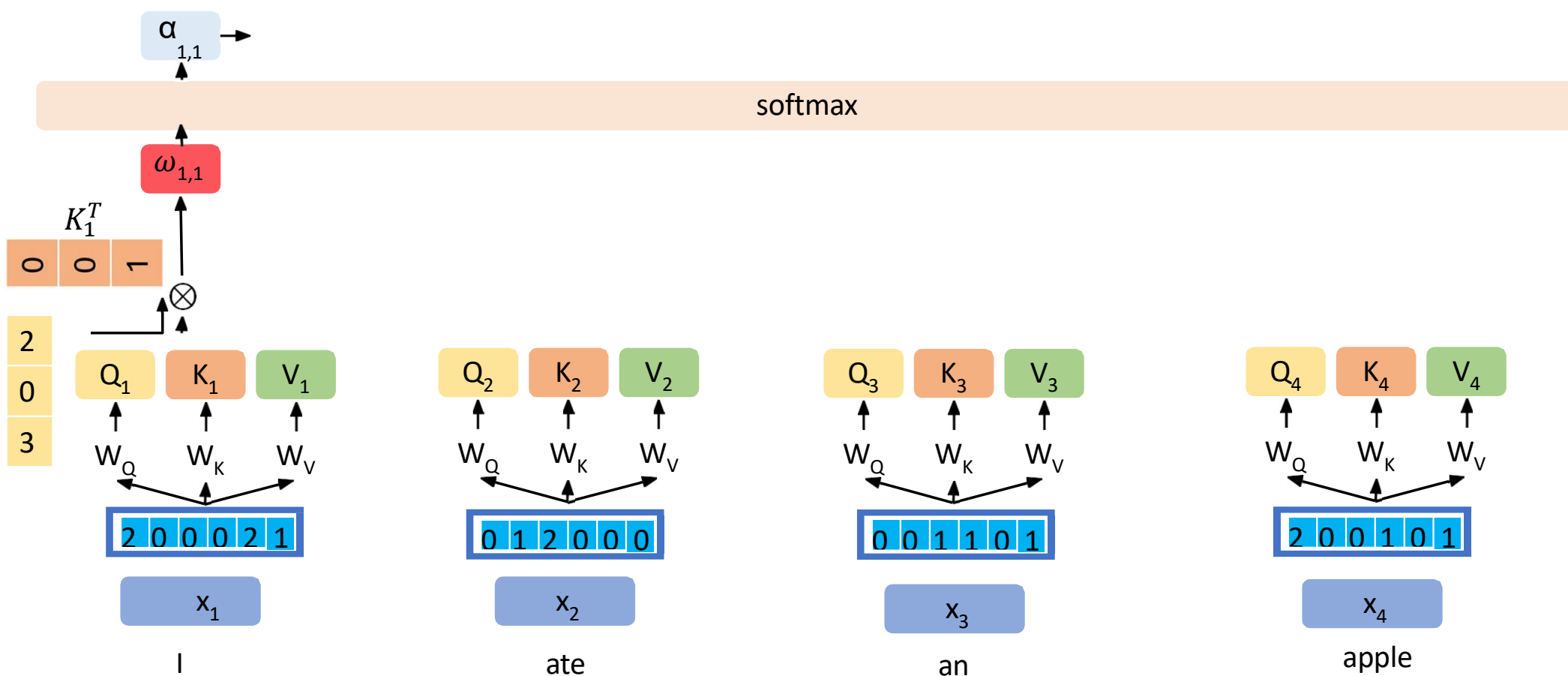
注意力机制



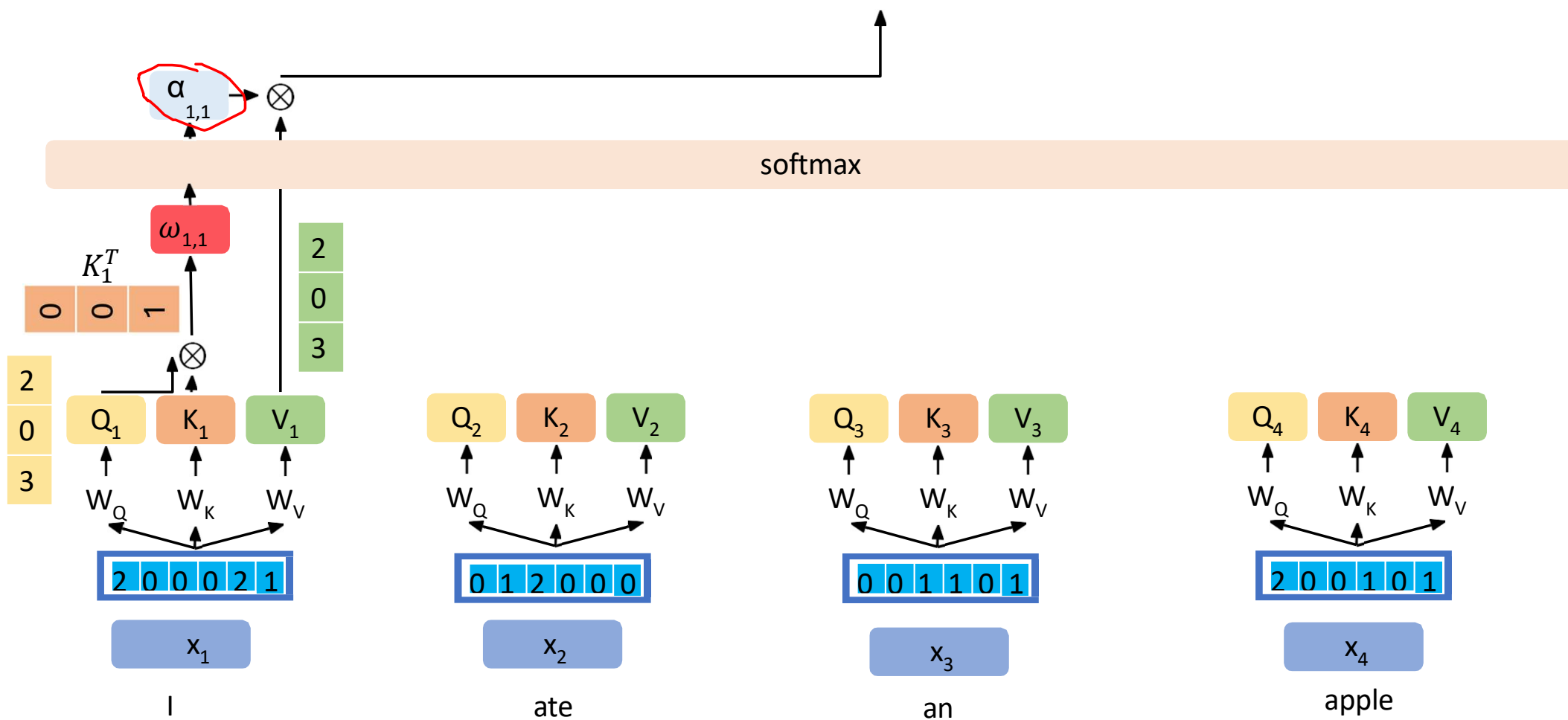
注意力机制



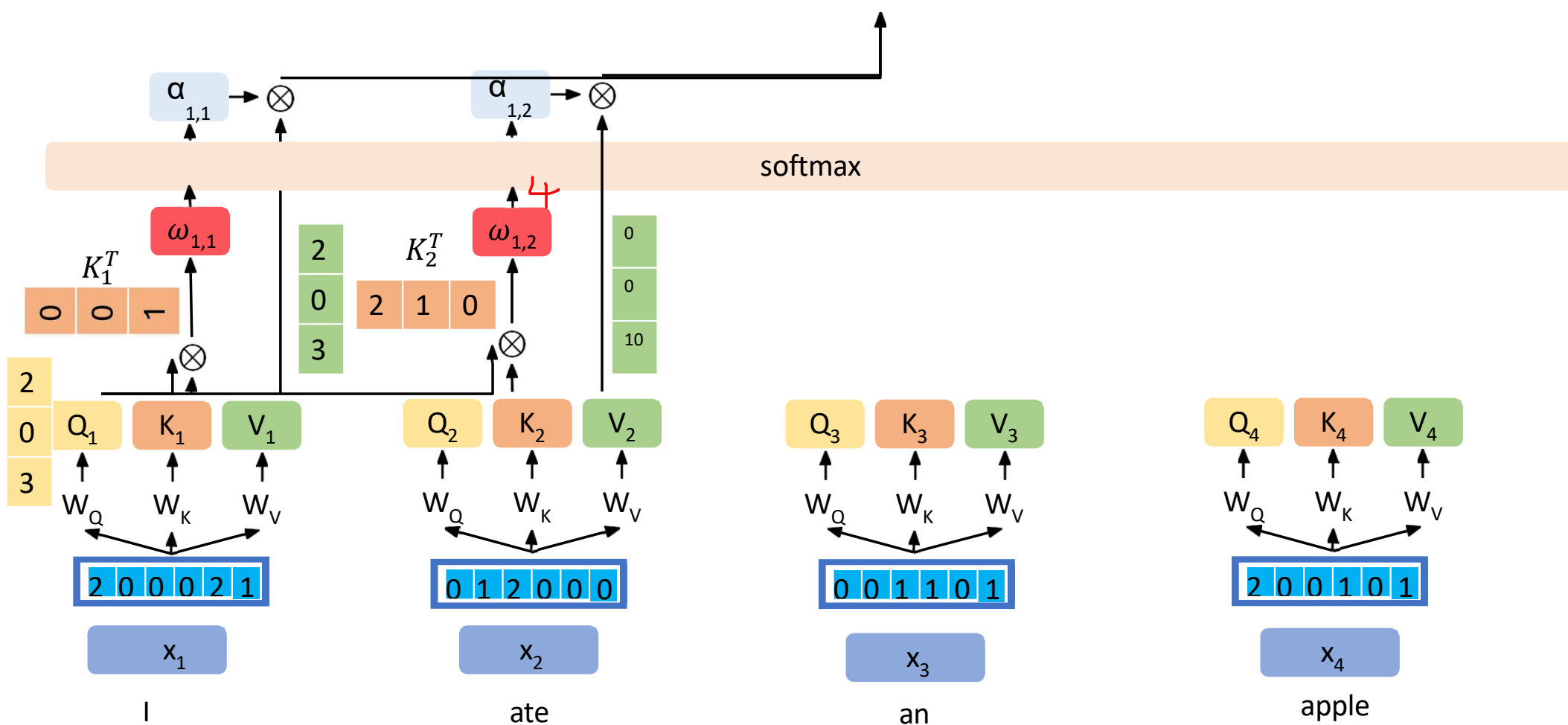
注意力机制



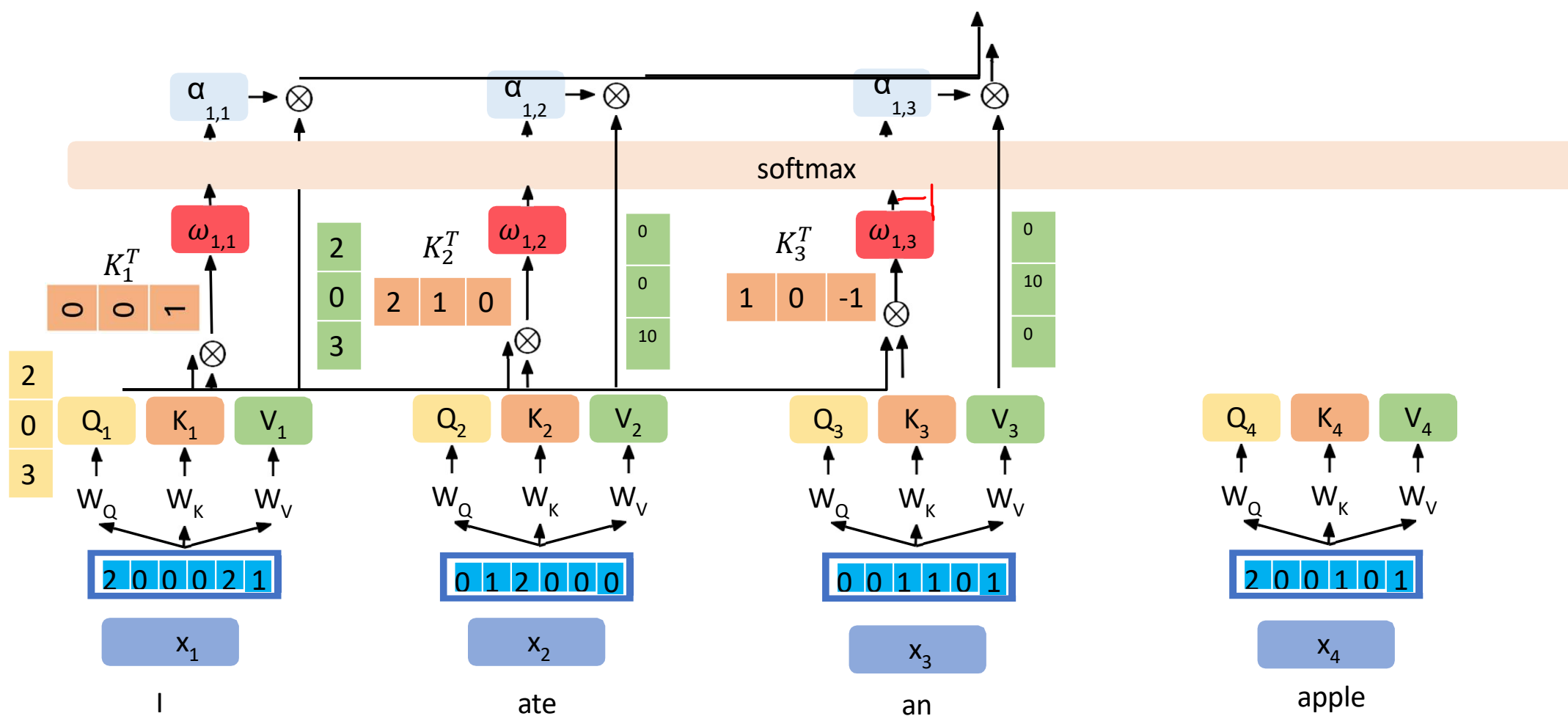
注意力机制



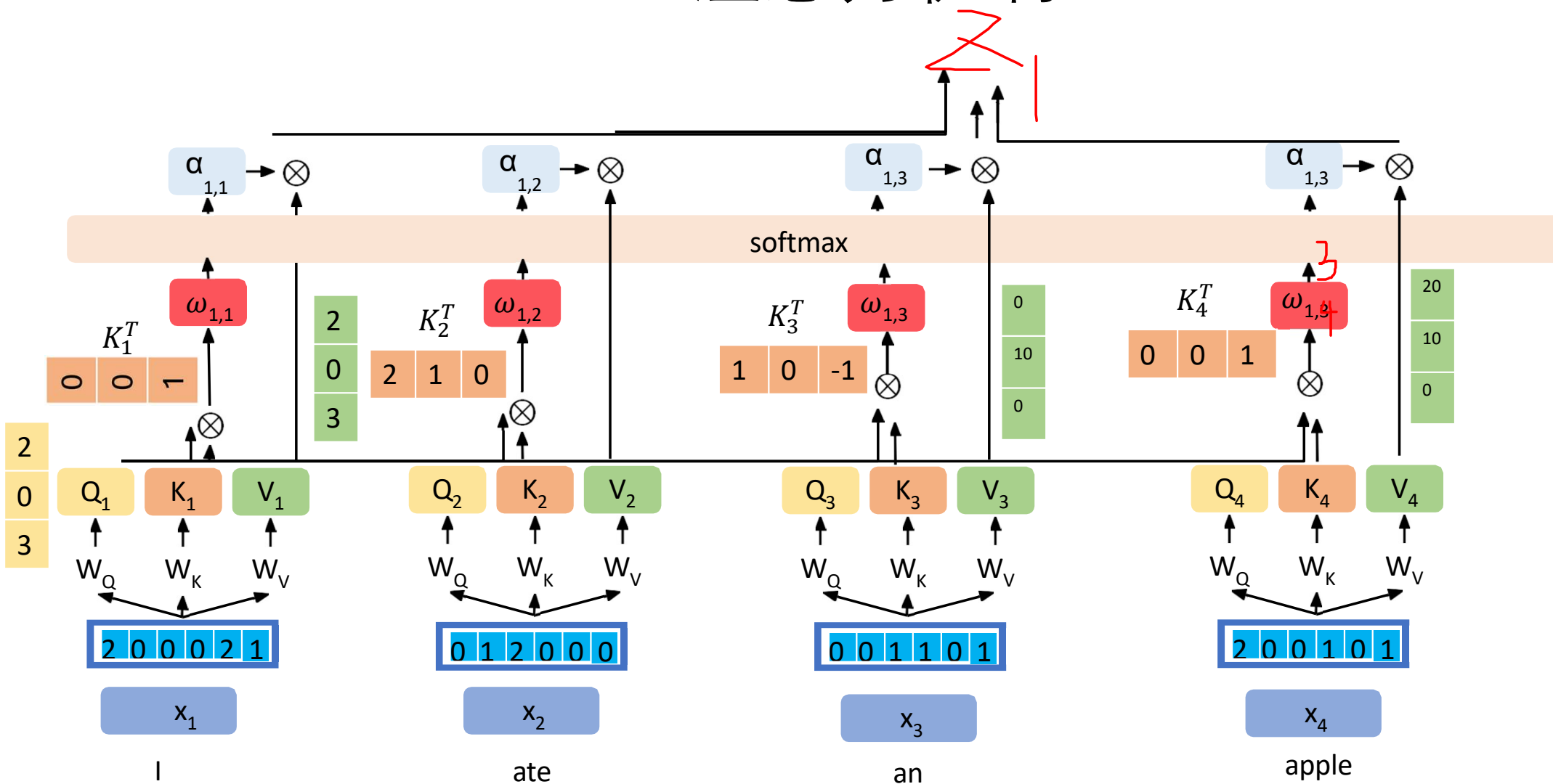
注意力机制



注意力机制

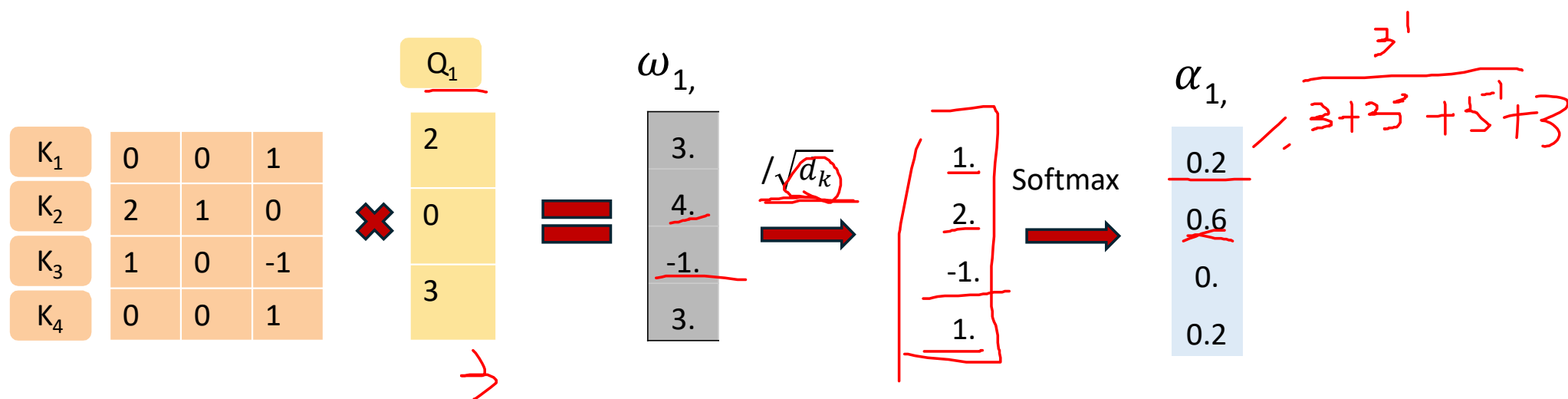


注意力机制

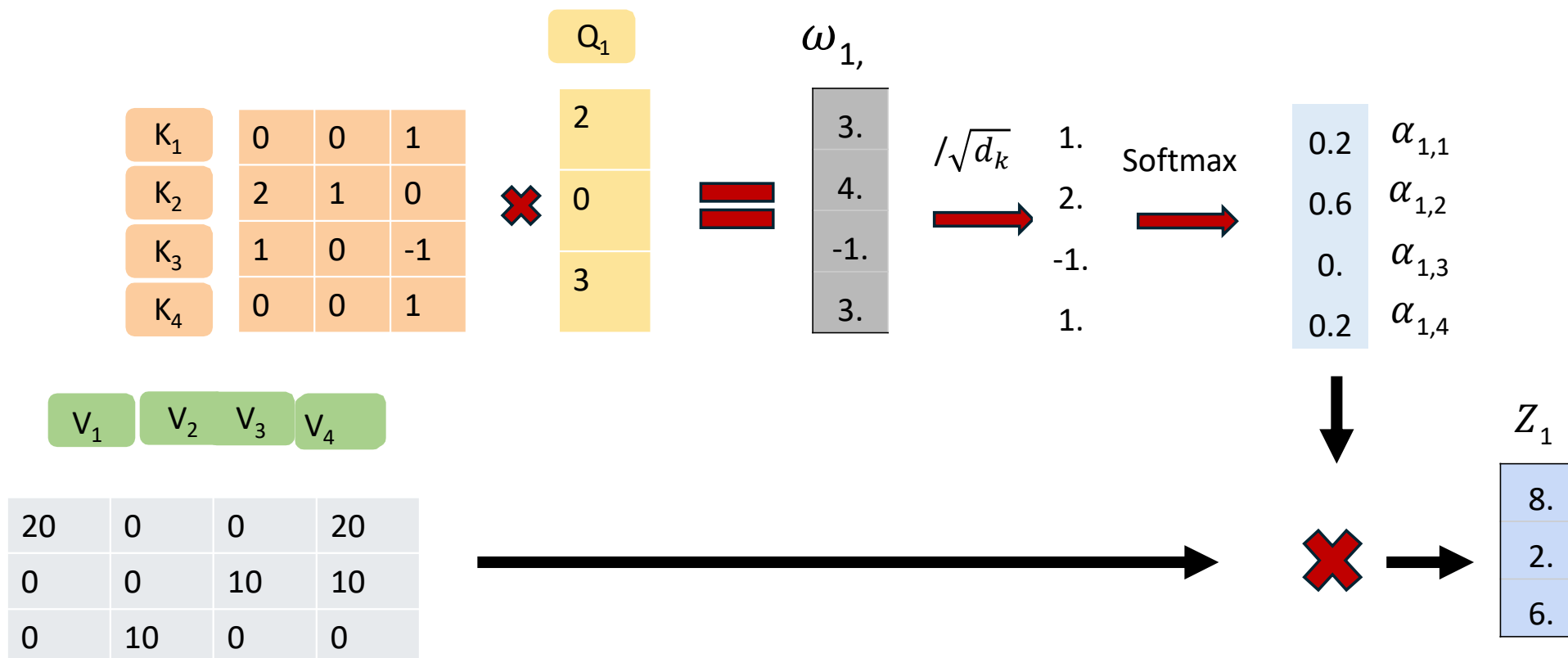


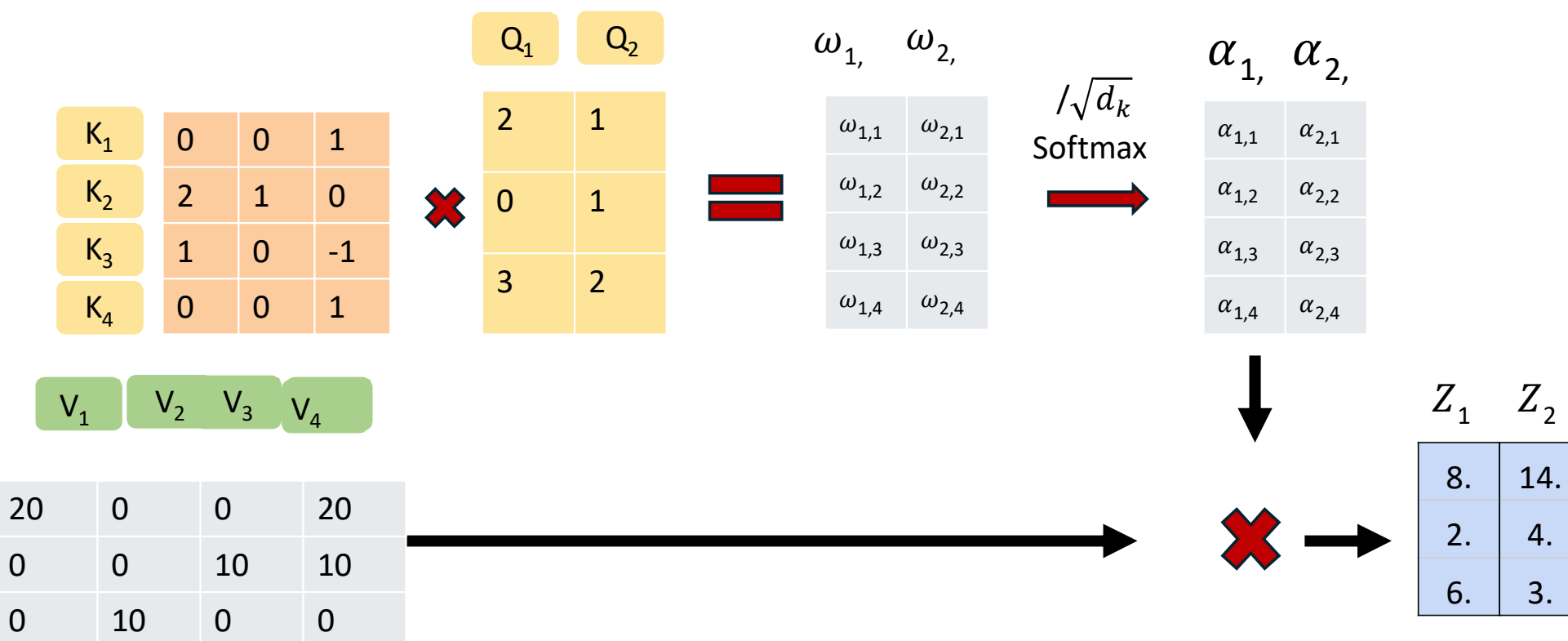
				Q_1			
K_1	0	0	1	$\times$	$=$	$\omega_{1,1}$	
K_2	2	1	0			2	$\omega_{1,2}$
K_3	1	0	-1			0	$\omega_{1,3}$
K_4	0	0	1			3	$\omega_{1,4}$

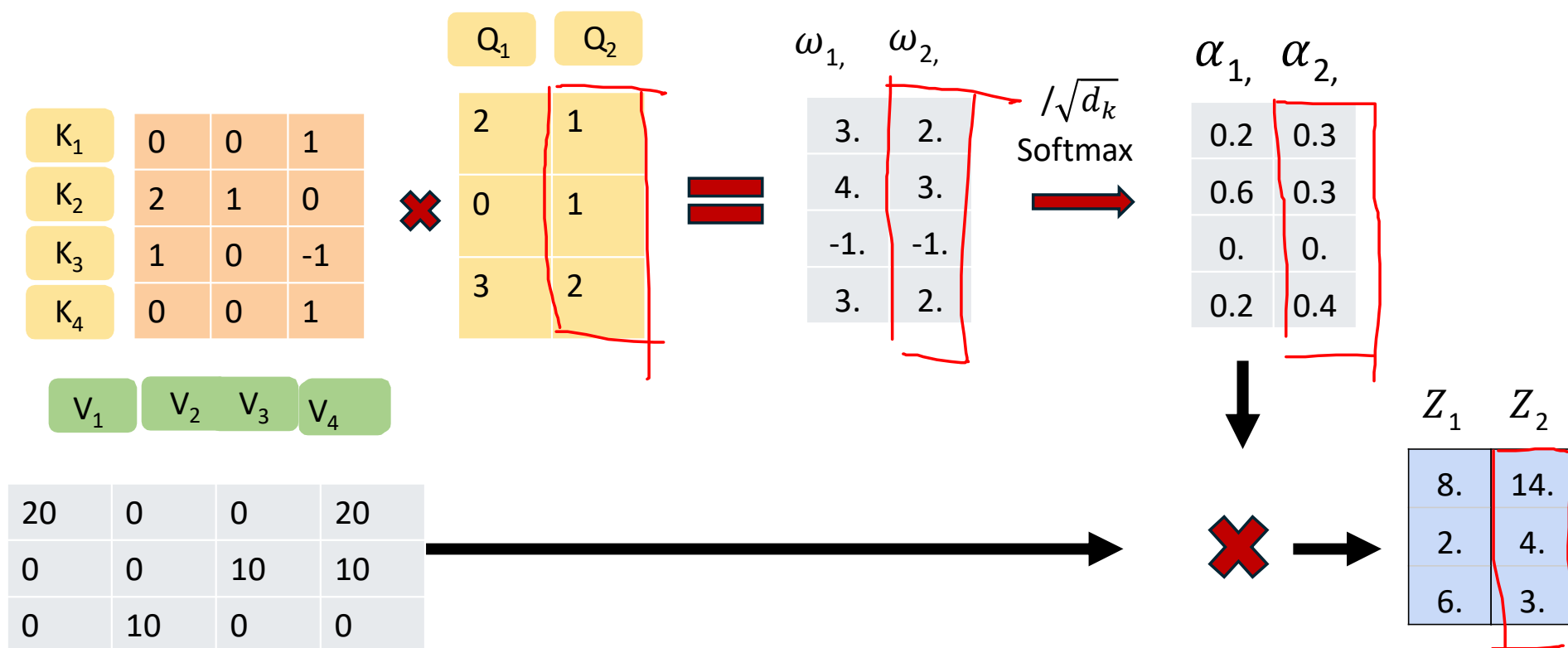
4x1

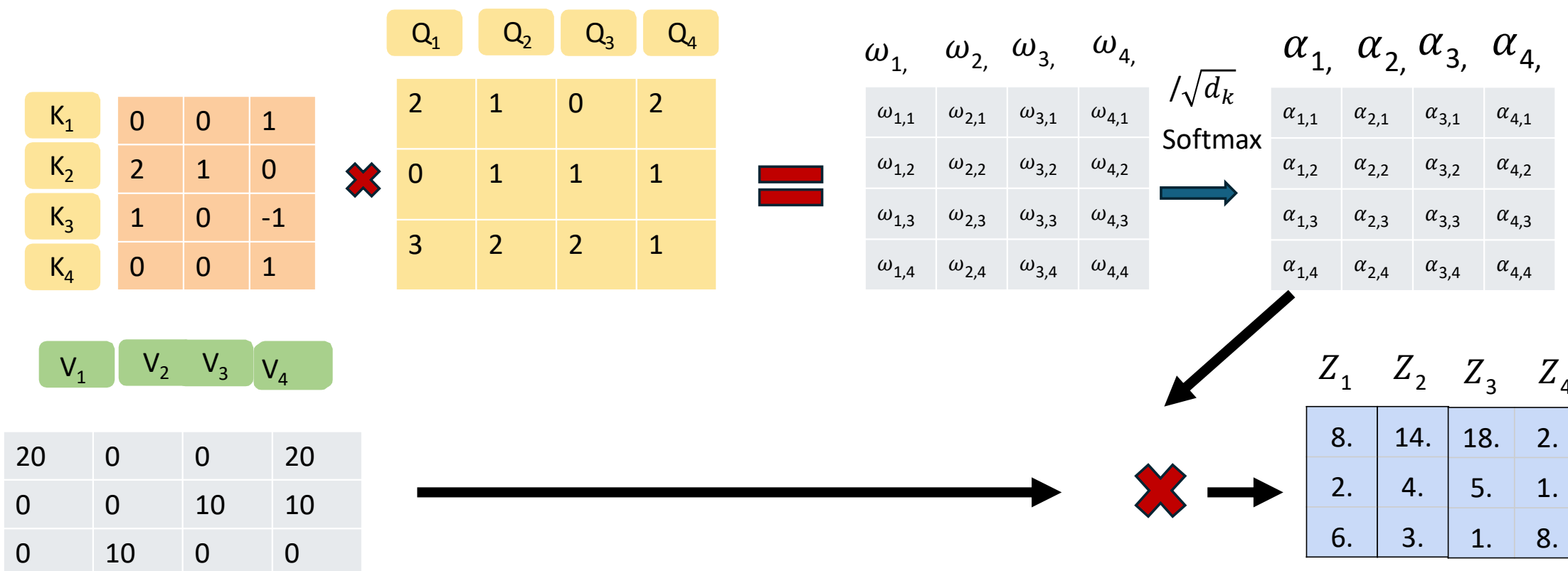


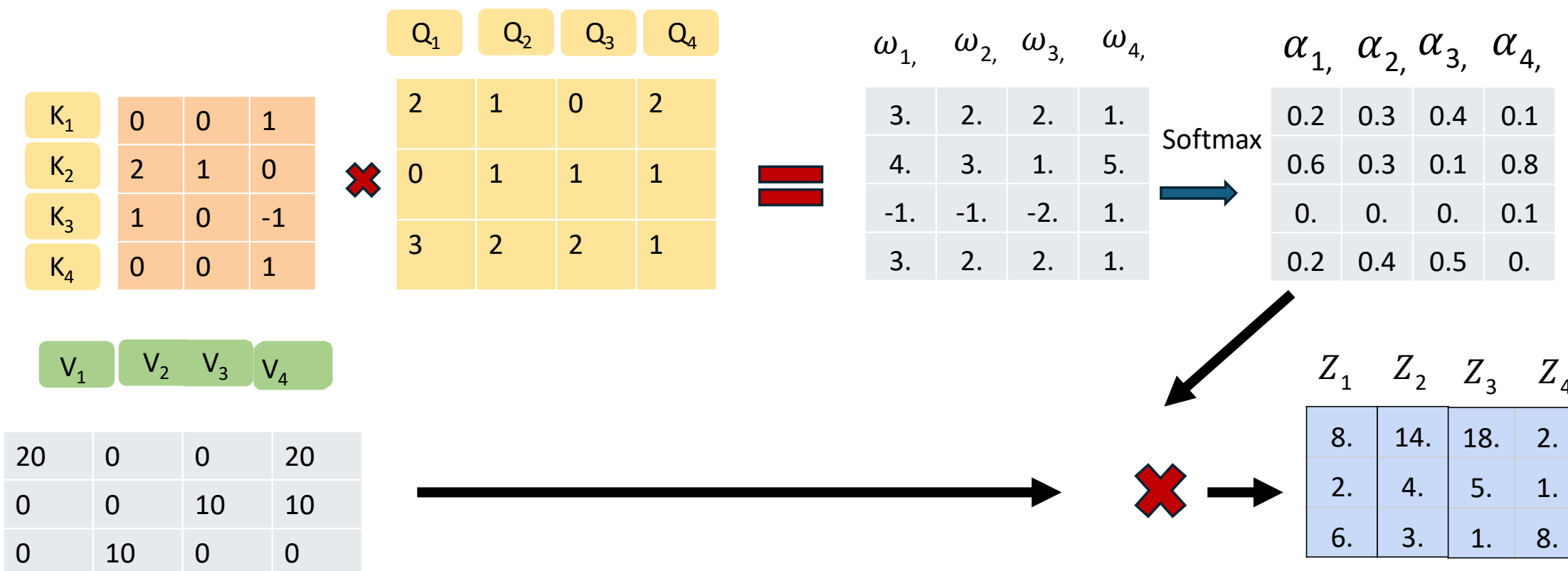
d_k 是Q,K,V向量的维度。这里 $d_k = 3$, $\sqrt{d_k}$ 取近似值2来计算, 计算结果向下取整
Softmax 中的e取近似值3.







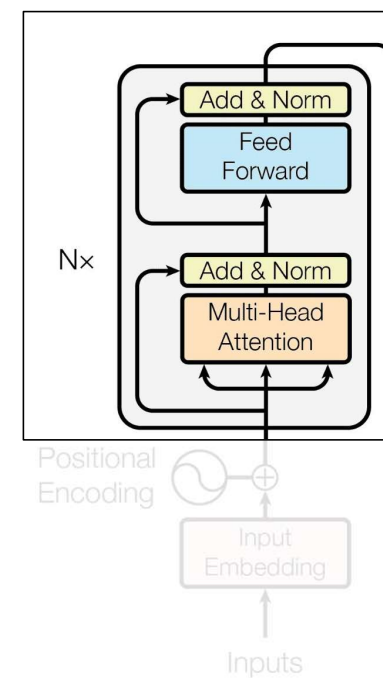




From lecture 18:

自注意力

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$





自注意力

The

animal

didn't

cross

the

street

because

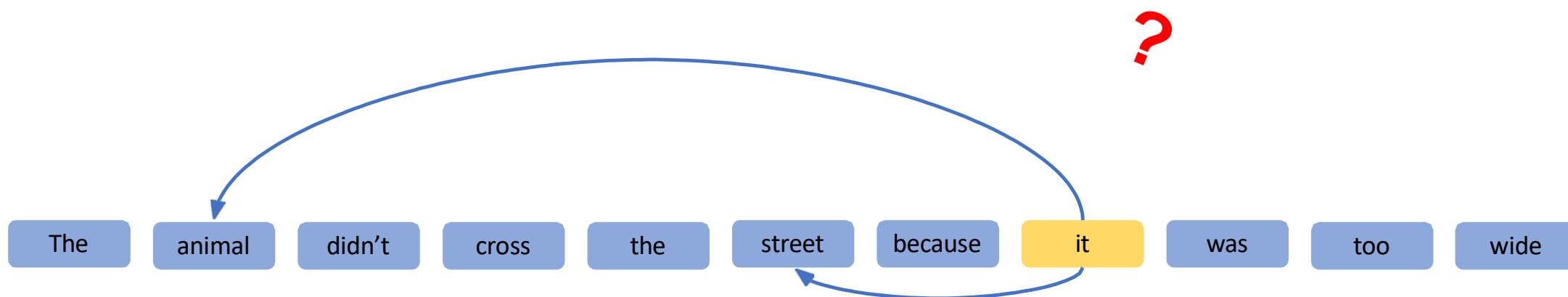
it

was

too

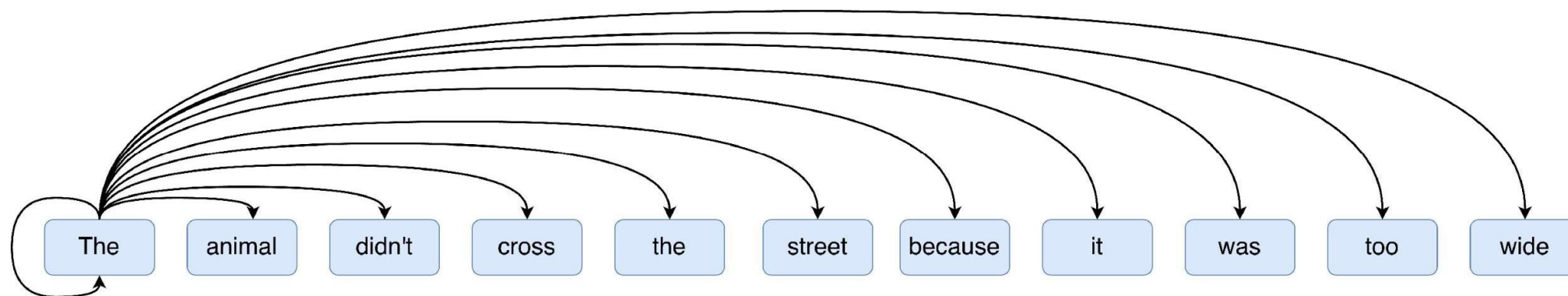
wide

自注意力

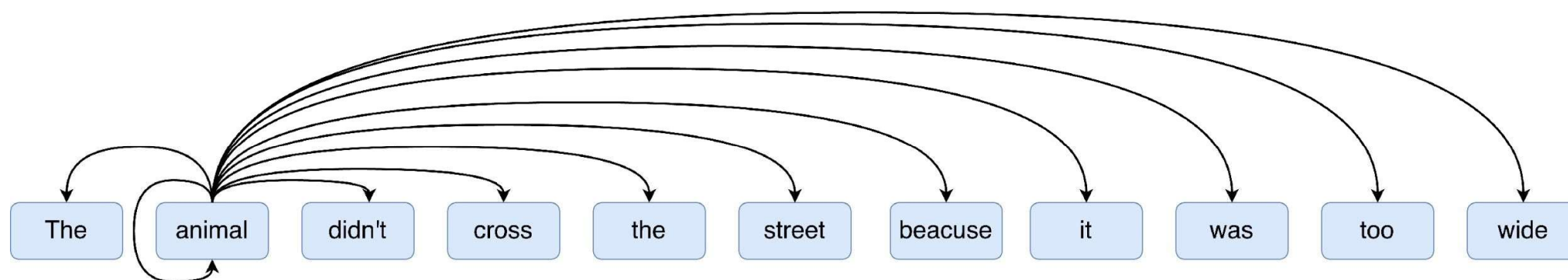


coreference resolution?

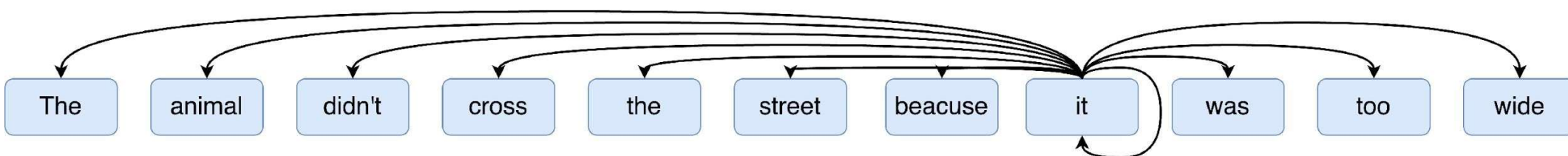
自注意力



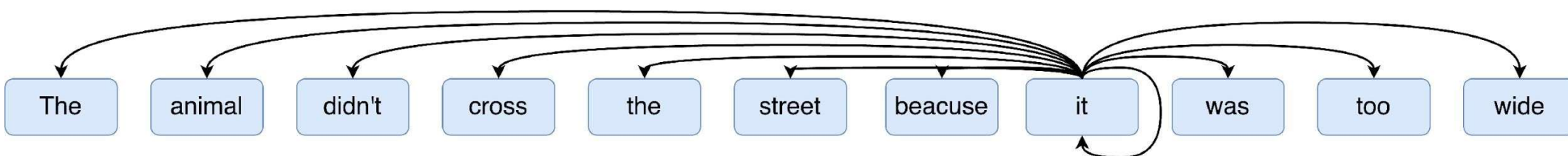
自注意力



自注意力



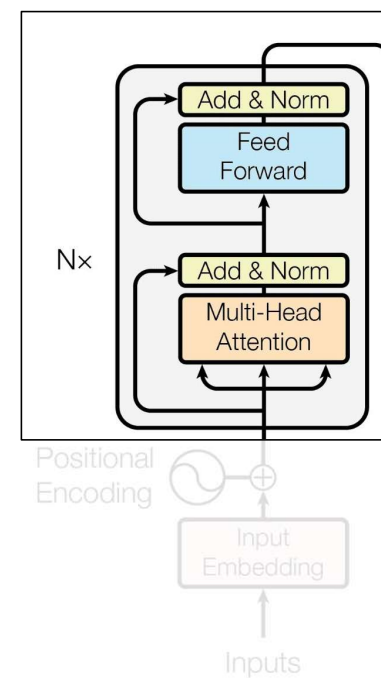
自注意力



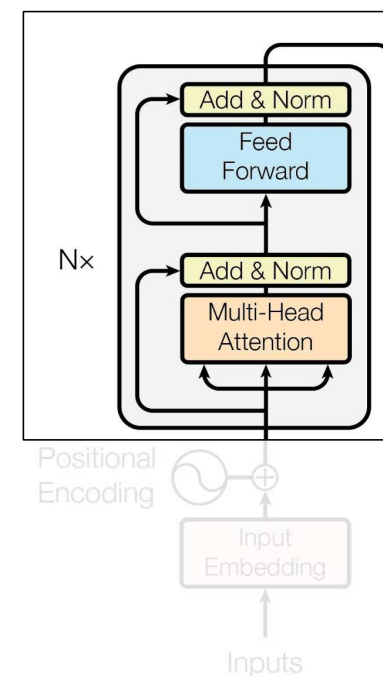
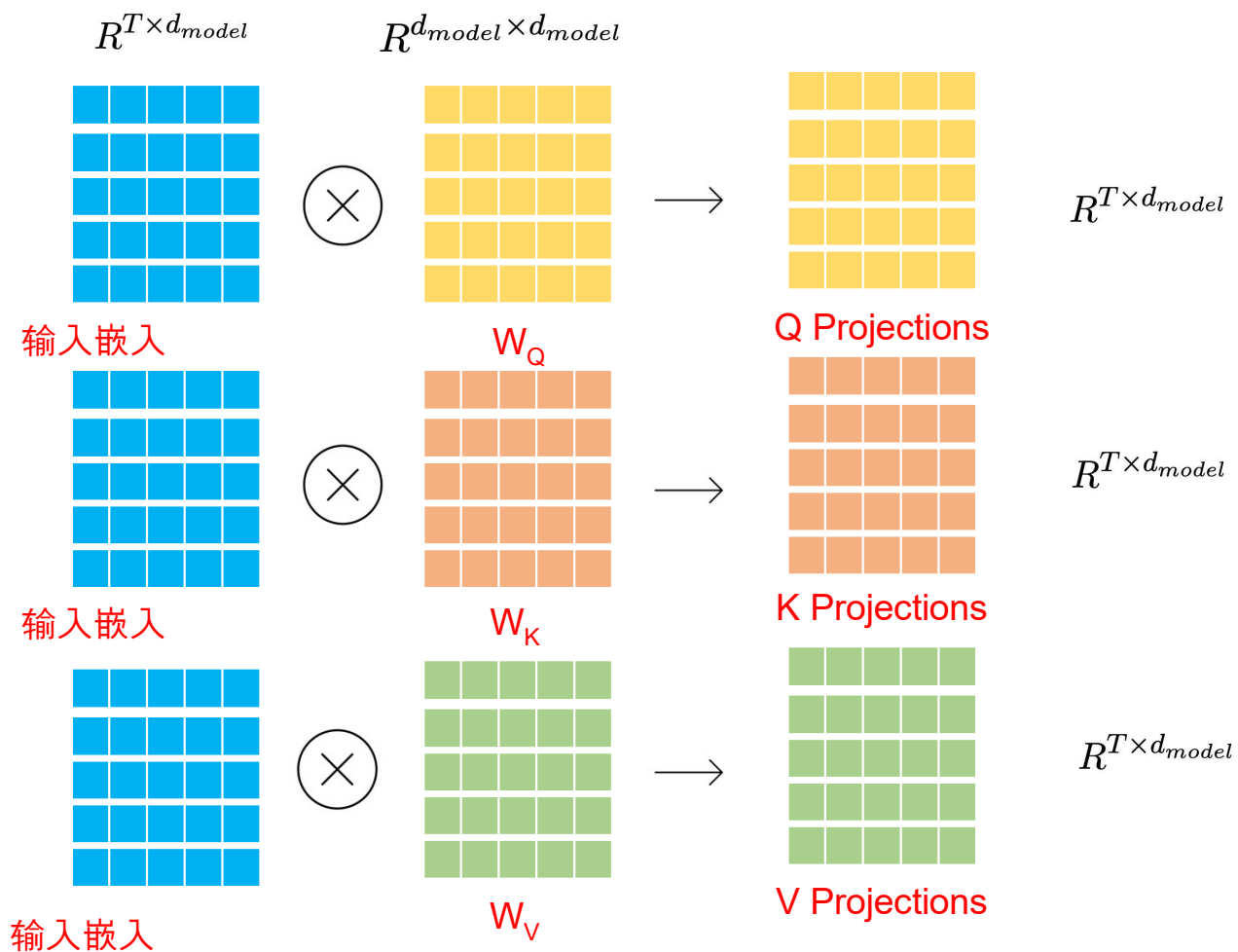
SELF

Query Inputs = Key Inputs = Value Inputs

自注意力

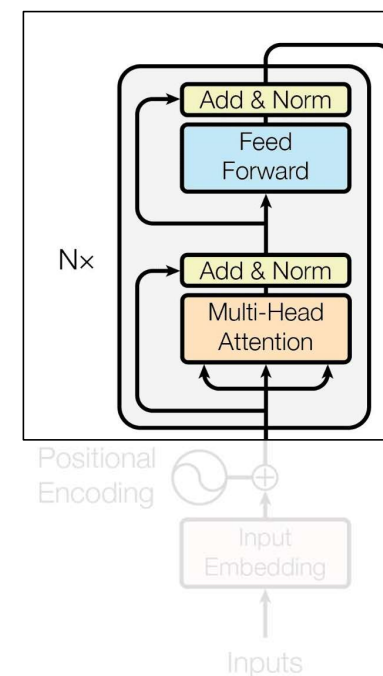
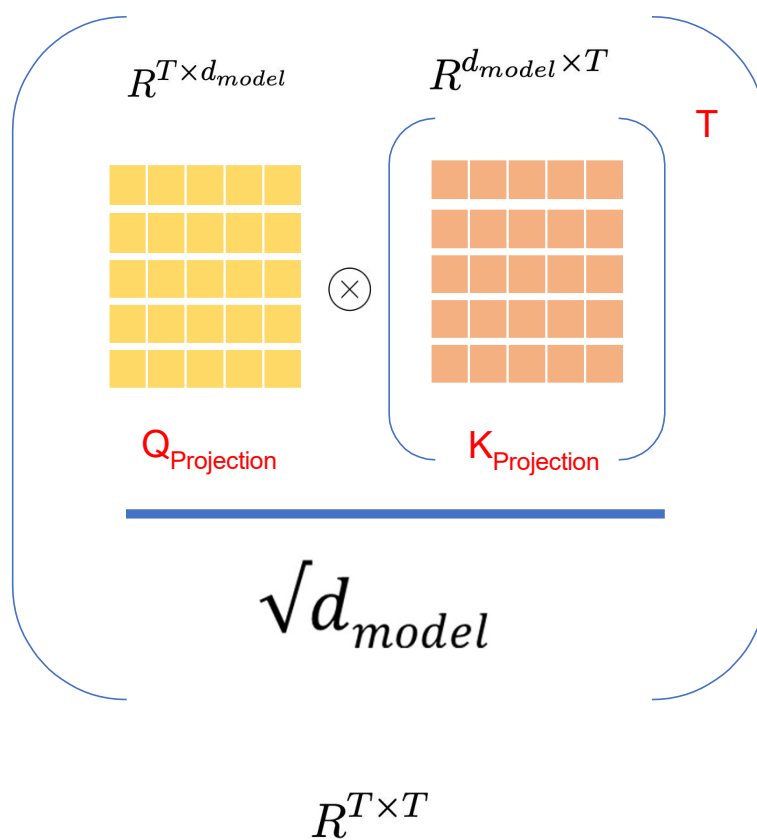


自注意力

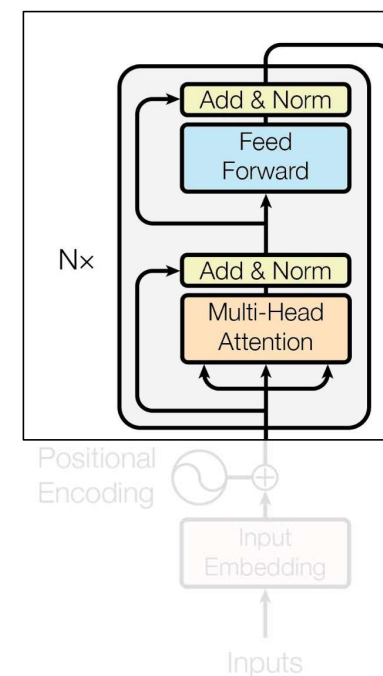
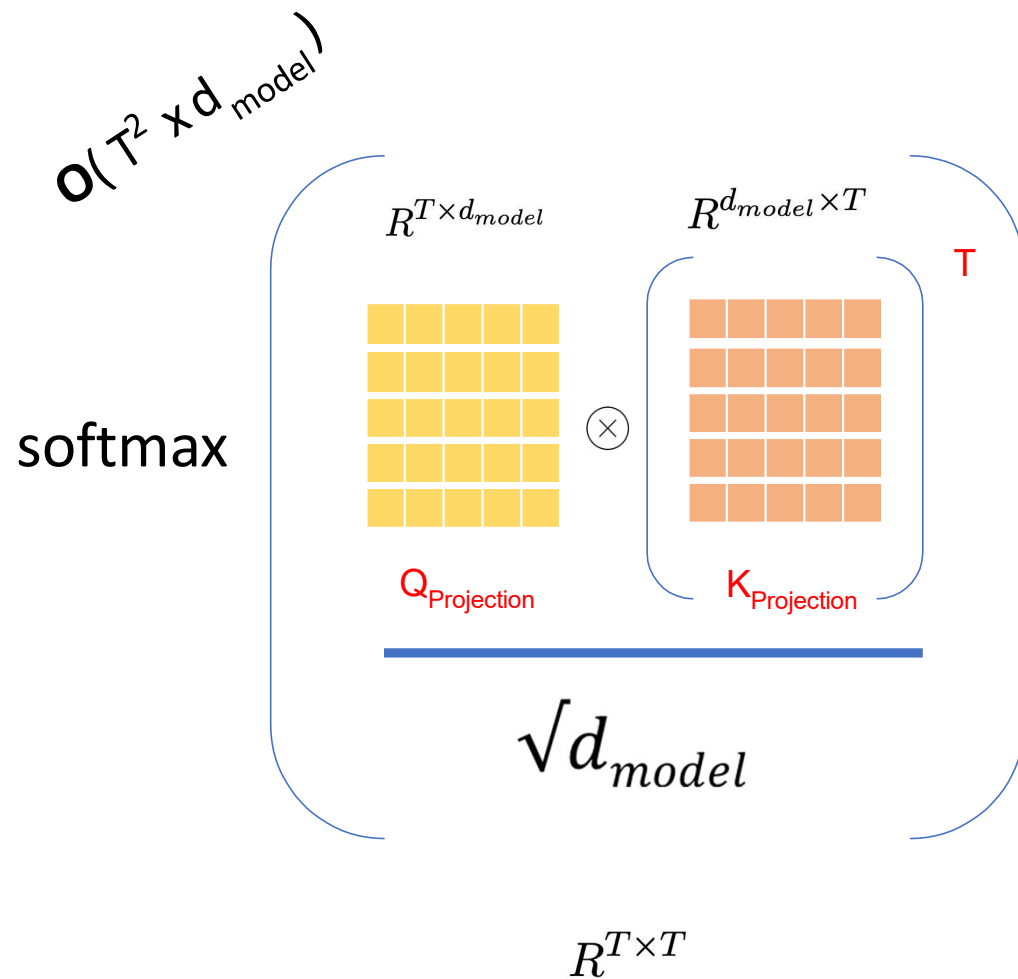


自注意力

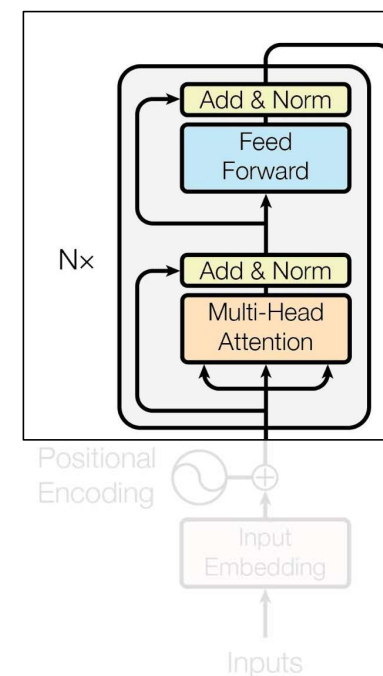
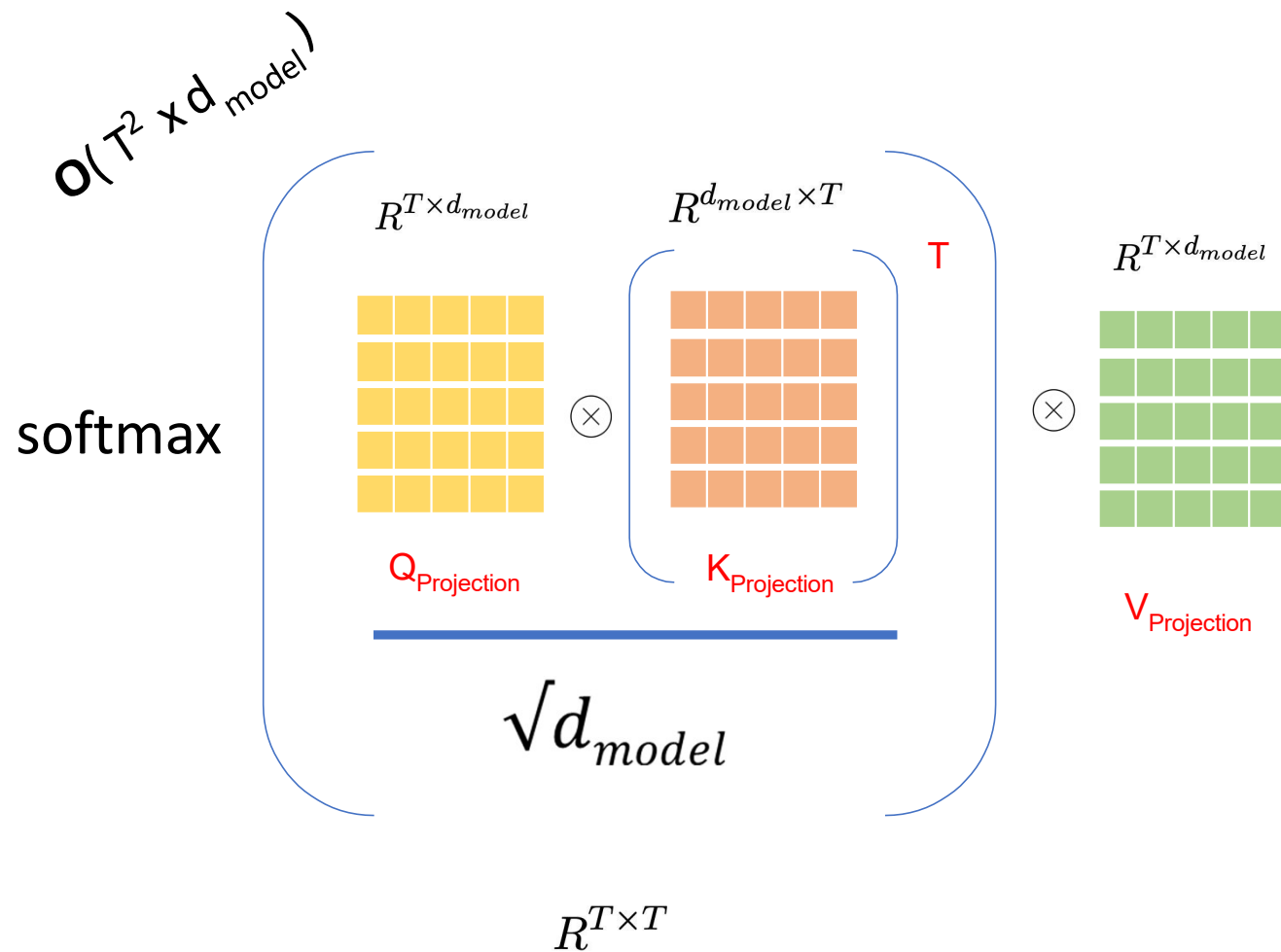
softmax



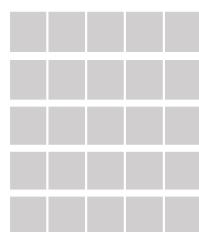
自注意力



自注意力

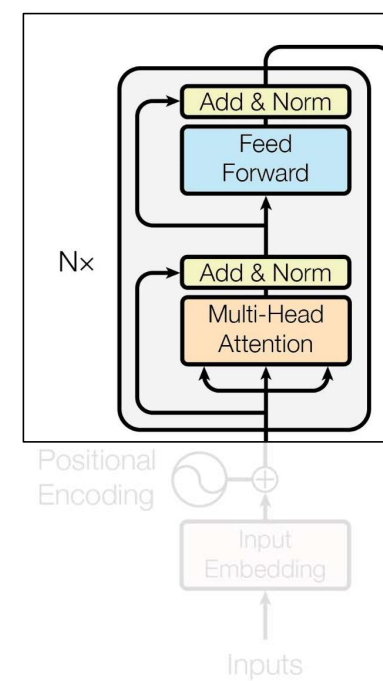


$$R^{T \times d_{model}}$$

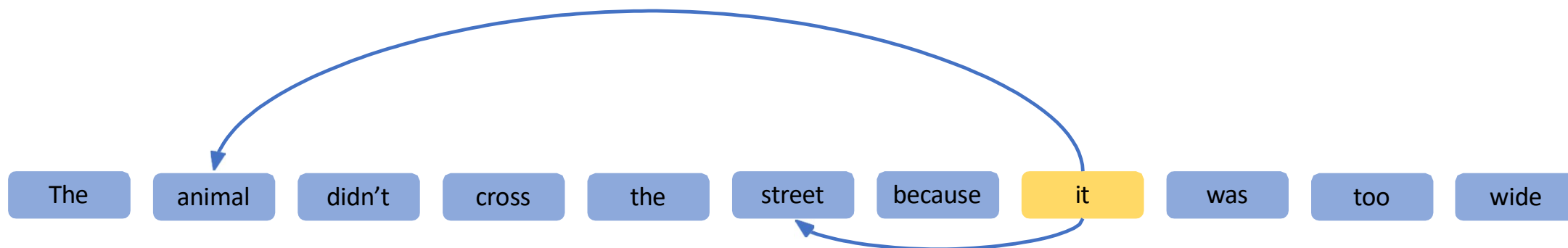


注意力机制: Z

自注意力

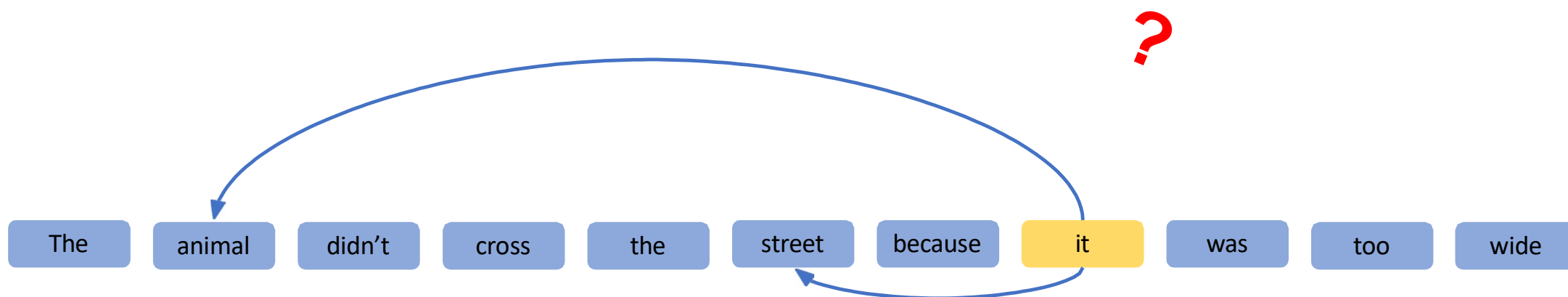


自注意力



Coreference resolution ✓

自注意力



Sentence boundaries ?

Coreference resolution ✓

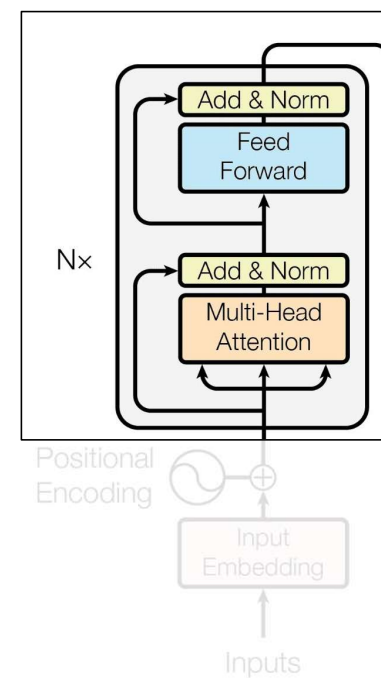
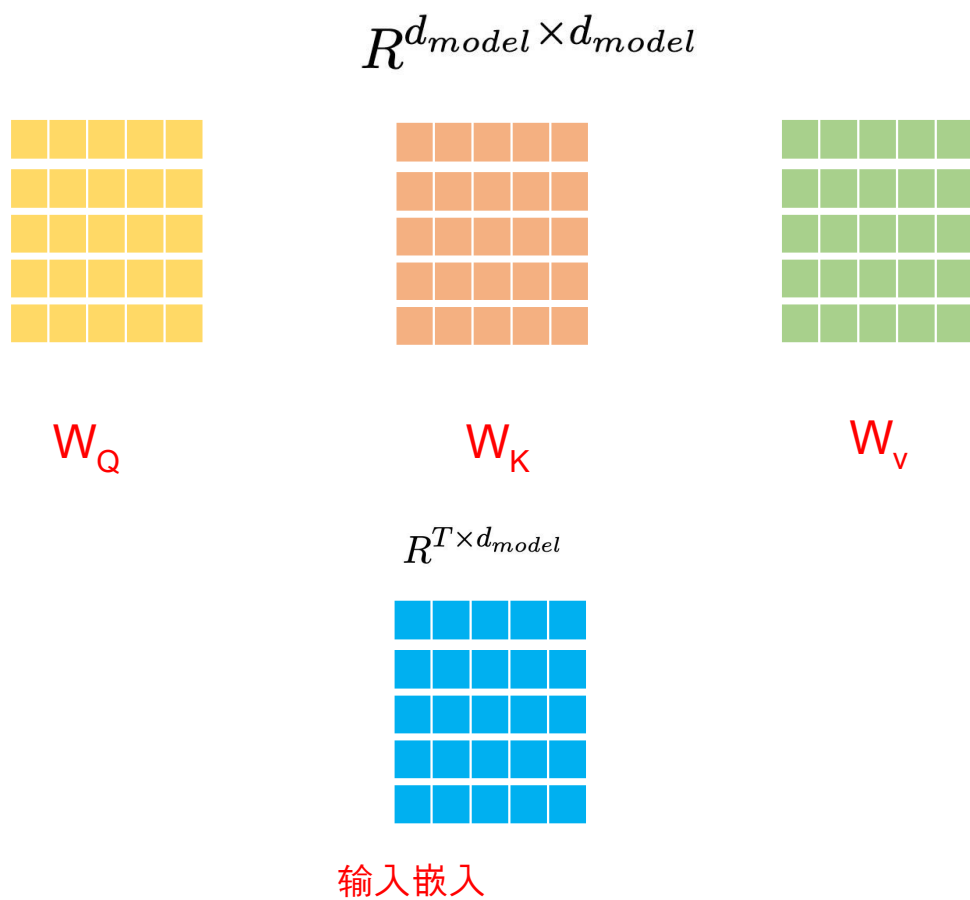
Context ?

Semantic relationships ?

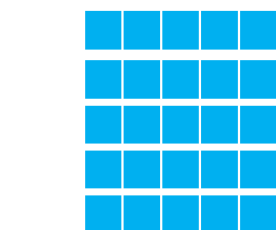
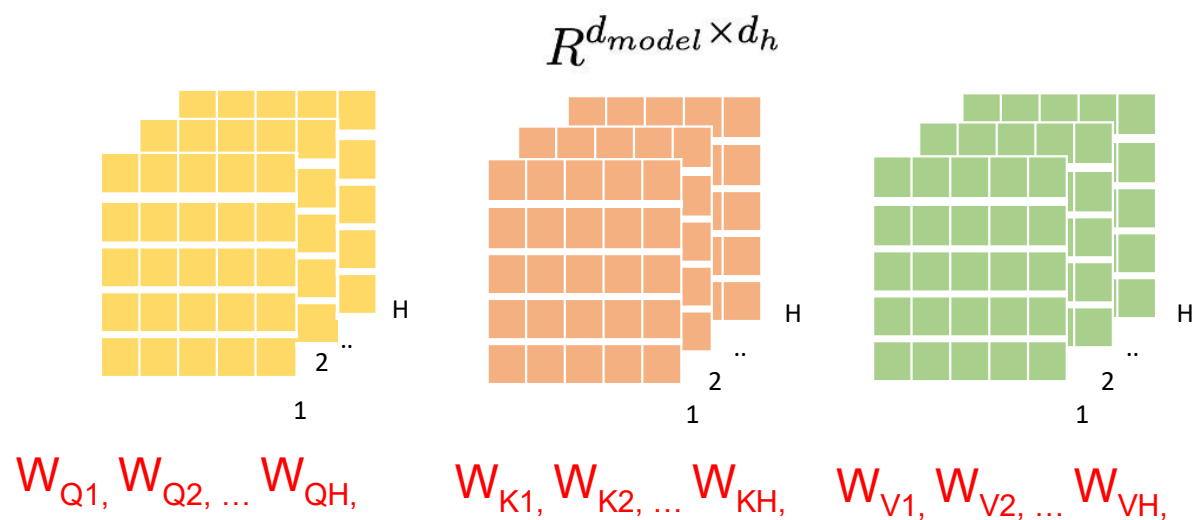
Part of Speech ?

Comparisons ?

自注意力



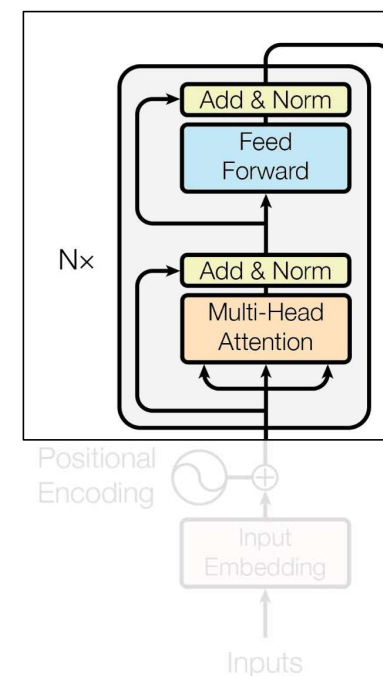
多头注意力



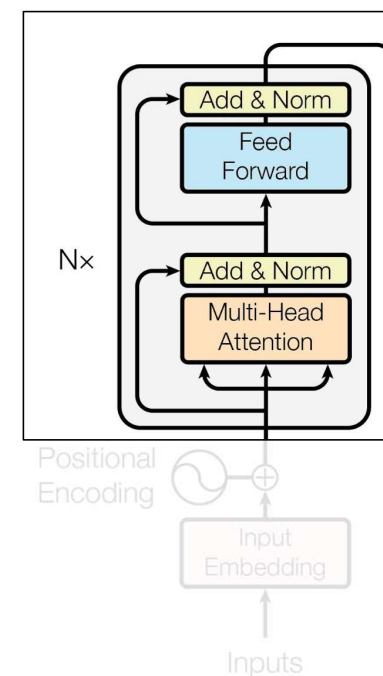
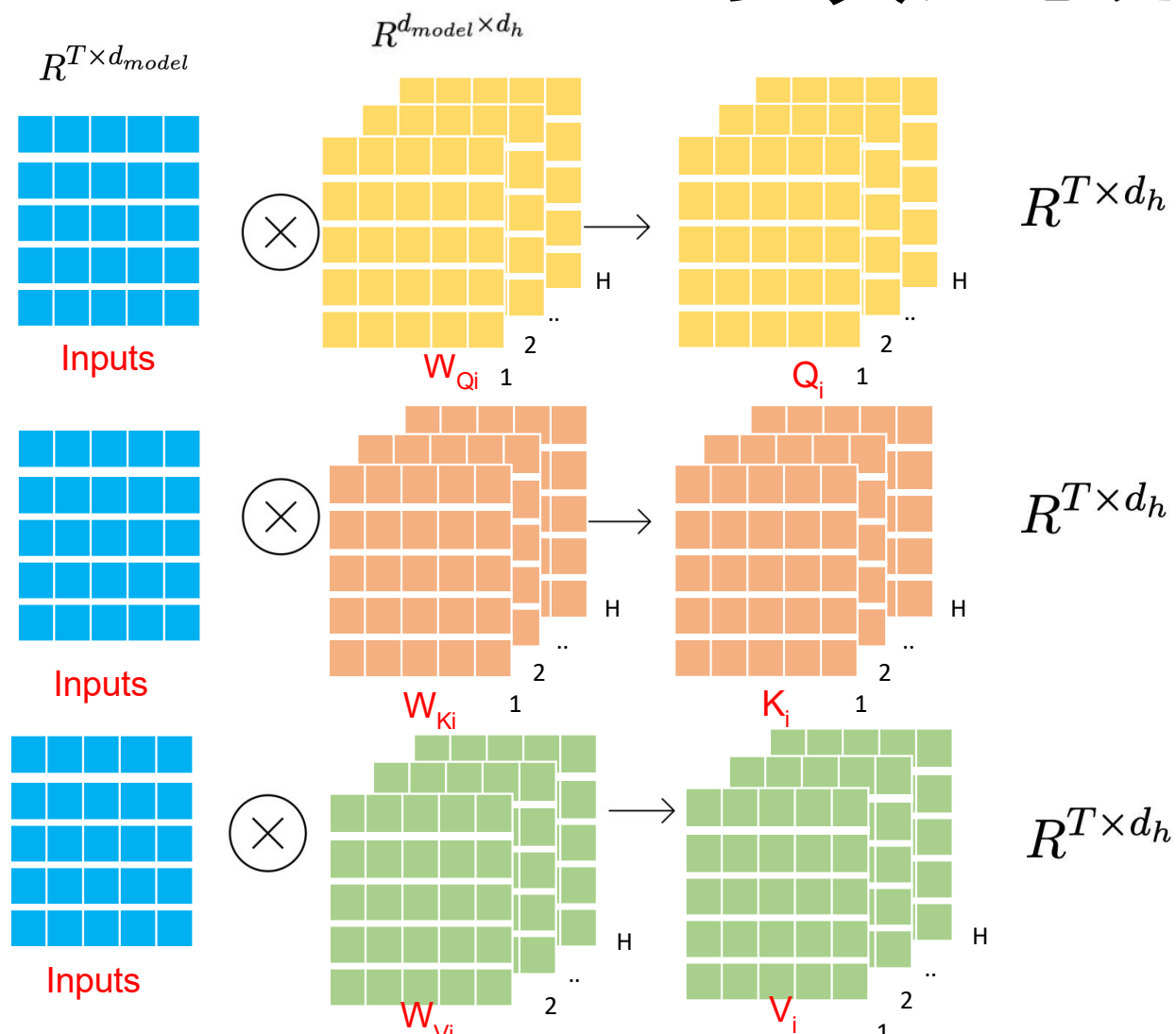
输入嵌入

$$R^{T \times d_{model}}$$

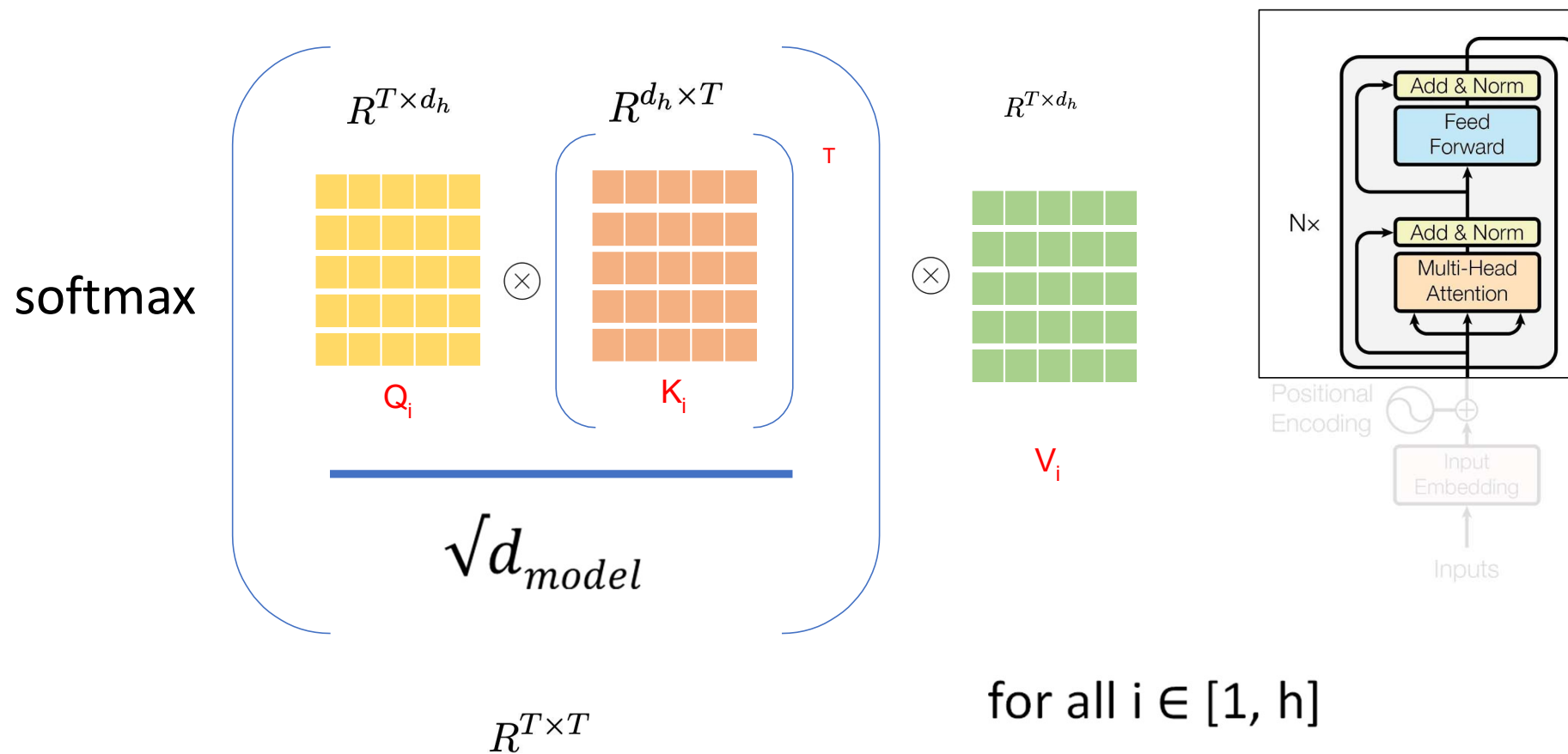
$$d_h = \frac{d_{model}}{h}$$



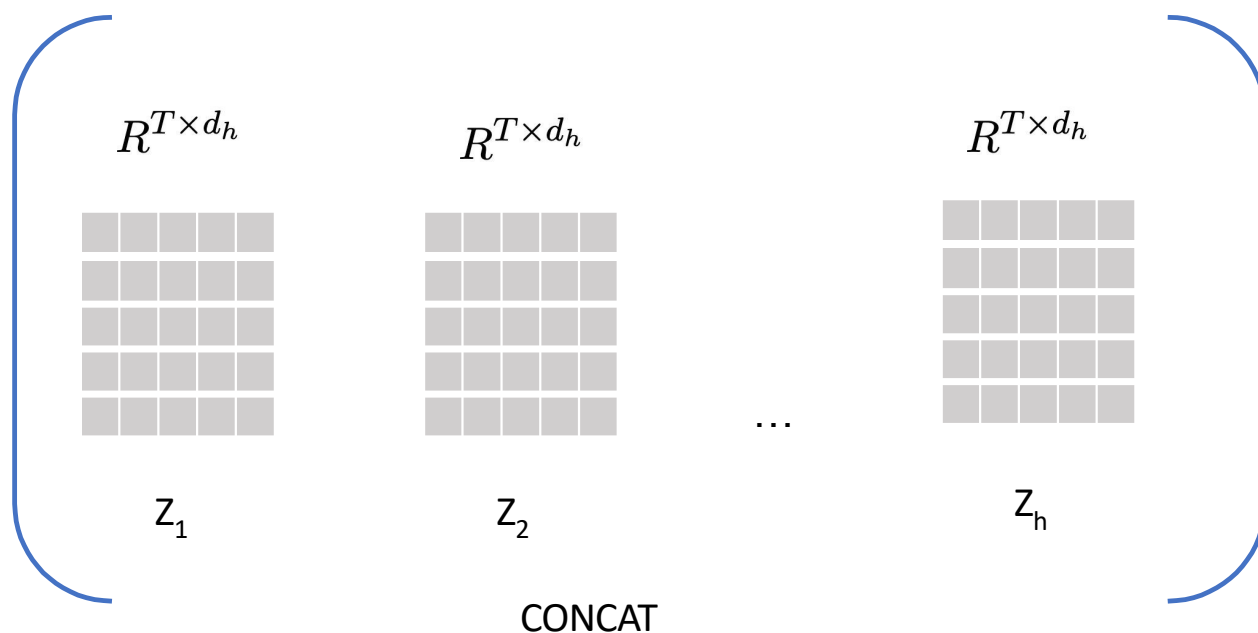
多头注意力



多头注意力



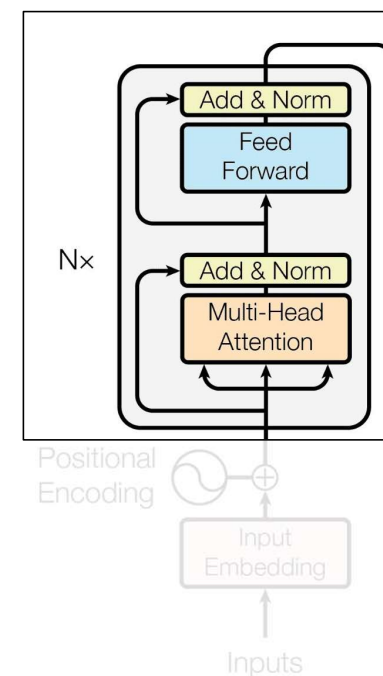
多头注意力



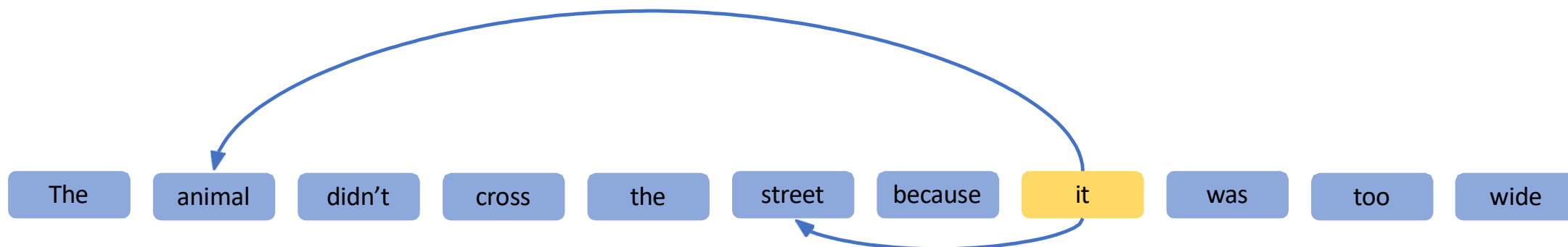
Multi Head 注意力机制
: Z

$$R^{T \times d_{model}}$$

$$d_h = \frac{d_{model}}{h}$$



多头注意力



Sentence boundaries



Coreference resolution



Context



Semantic relationships



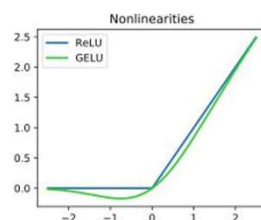
Part of speech



Comparisons

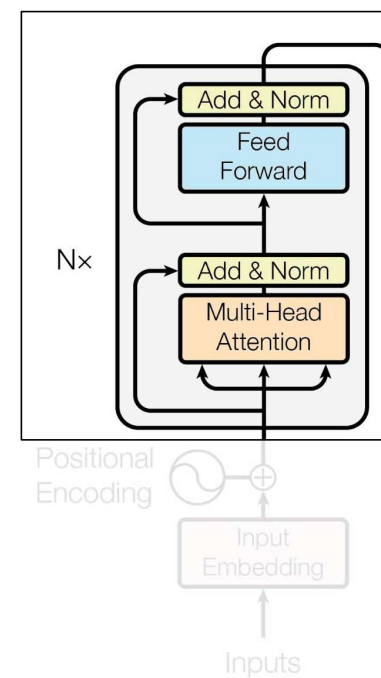


Feed Forward 前馈层

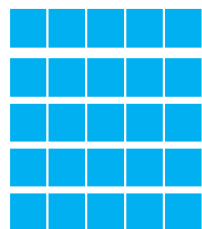


Feed Forward

- Non Linearity
- Complex Relationships
- Learn from each other



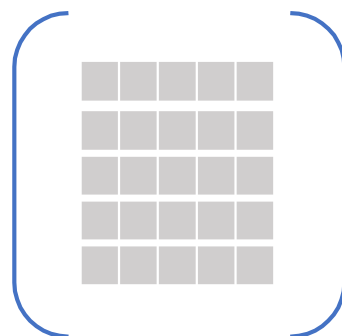
Feed Forward



Input

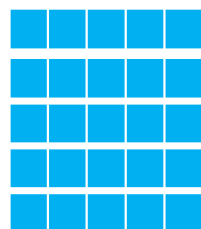


Residuals

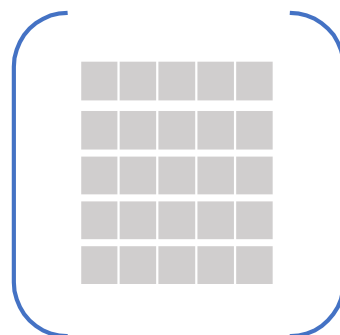


Norm(Z)

Add & Norm



Input



Norm(Z)

Normalization : 归一化 (标准化)

Mean 0, Std dev 1 :

均值为 0, 标准差为 1

Stabilizes training :

使训练过程更加稳定

Regularization effect :

具有正则化效果

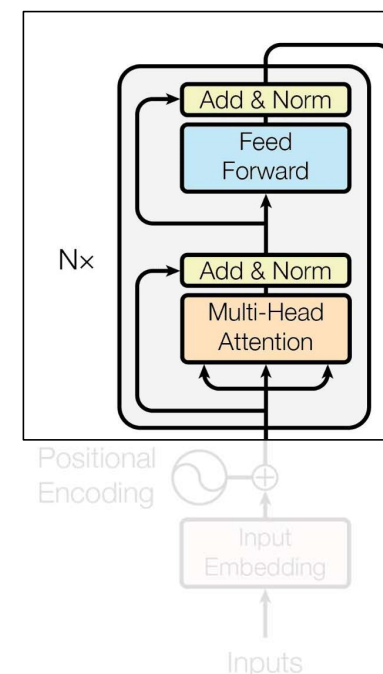
Add Residuals : 引入残差连接

Avoid vanishing gradients :

避免梯度消失

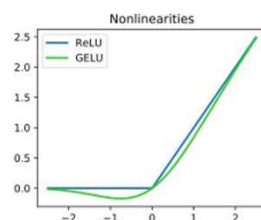
Train deeper networks :

可以训练更深的神经网络

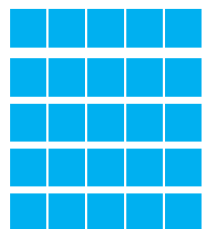


Add & Norm

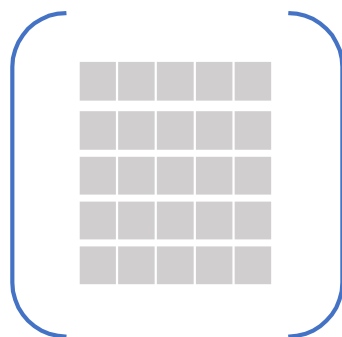
Add & Norm



Feed Forward

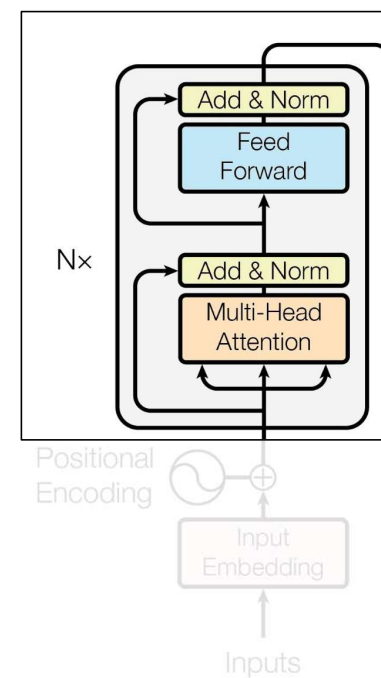


Residuals



Norm(Z)

Input

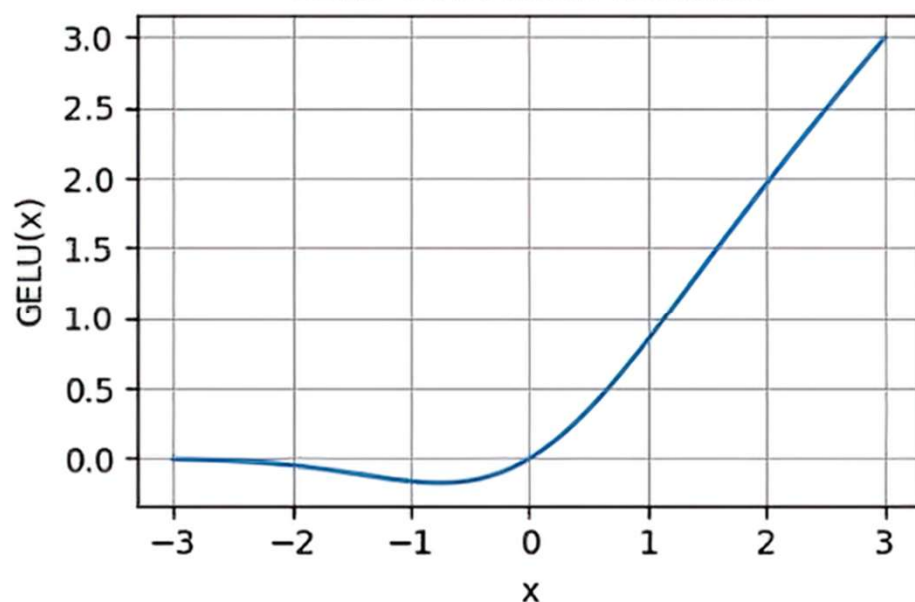


GELU 激活函数

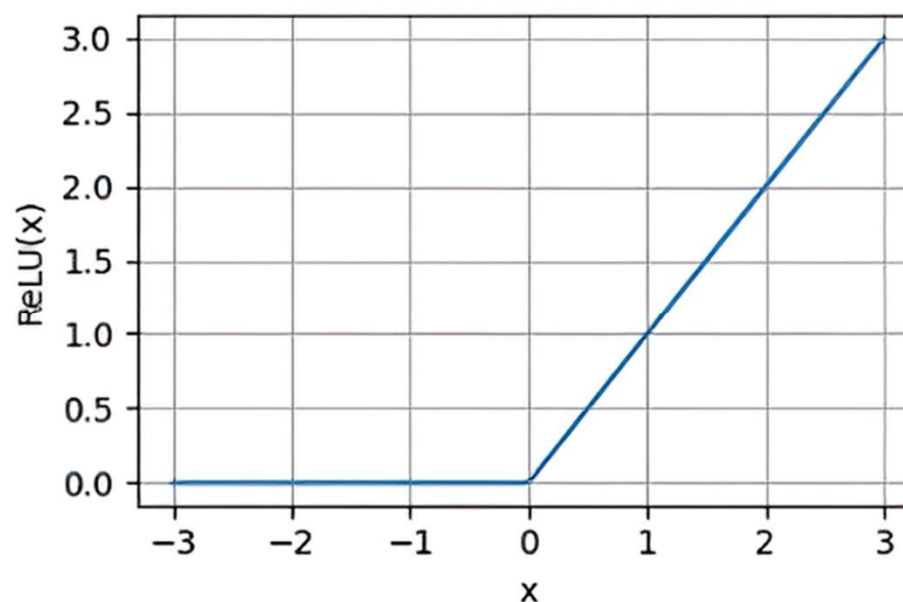
GELU 激活函数更平滑、信息保留更充分(负值), 是适合 GPT 等深层模型的激活函数, 提升了训练稳定性和效果。

$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} \cdot \left(x + 0.044715 \cdot x^3 \right) \right] \right)$$

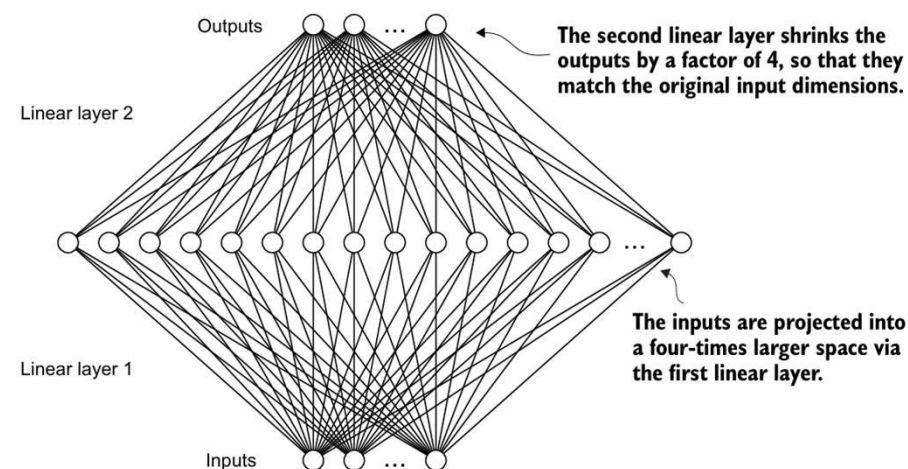
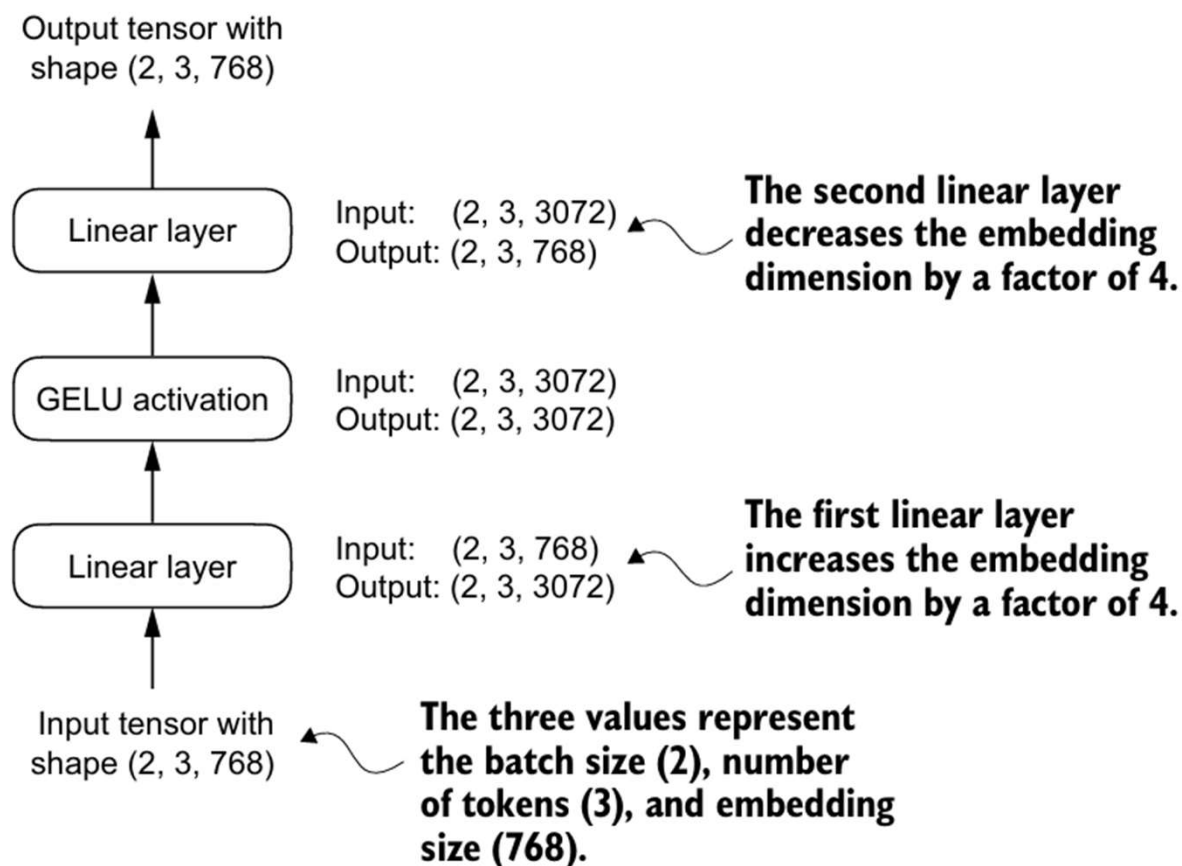
GELU activation function



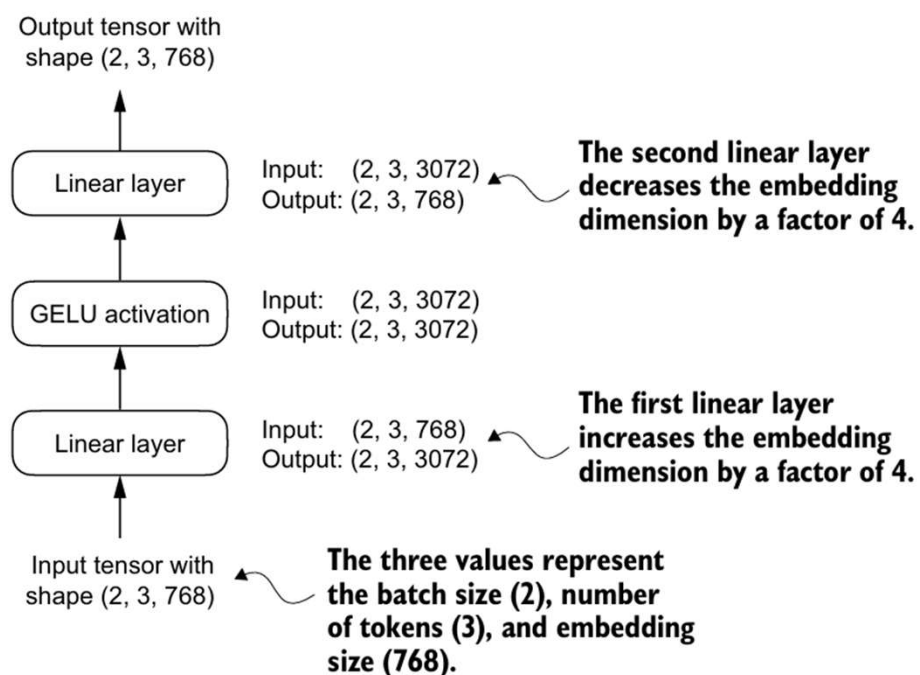
ReLU activation function



前馈层：激活函数为**GELU**的全连接层



前馈层： 激活函数为**GELU**的全连接层



```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
            GELU(),
            nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
        )

    def forward(self, x):
        return self.layers(x)
```

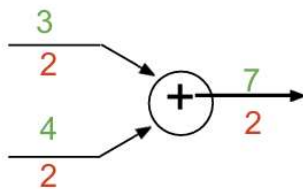
在深度神经网络中添加残差连接

- 残差连接是为了缓解深度网络的梯度消失问题

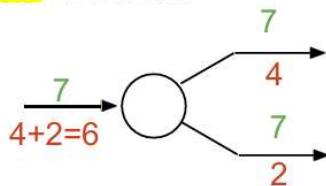
- 梯度消失问题是指梯度在向后传播到各个层时逐渐变小的问题，这使得难以有效地训练前面的层。

梯度流中的模式

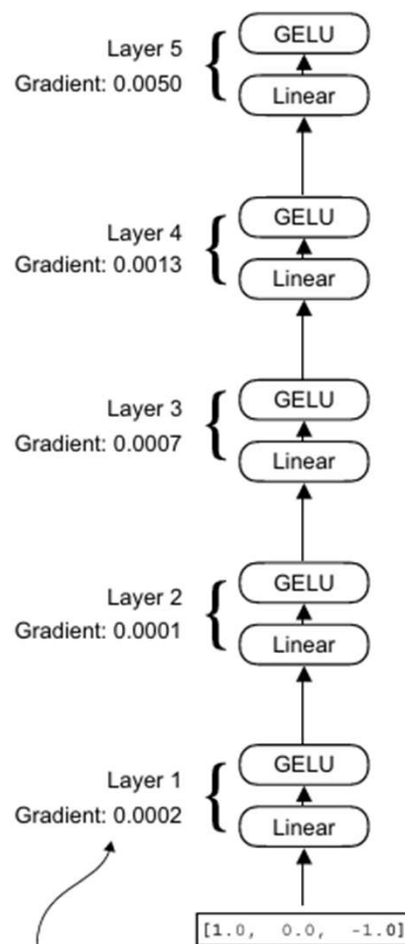
加法：梯度均分



复制：梯度加法

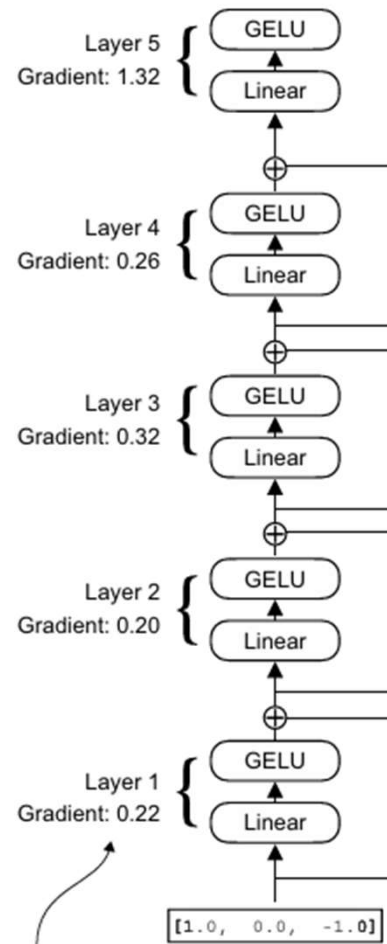


Deep neural network



In very deep networks, the gradient values in early layers become vanishingly small

Deep neural network with shortcut connections



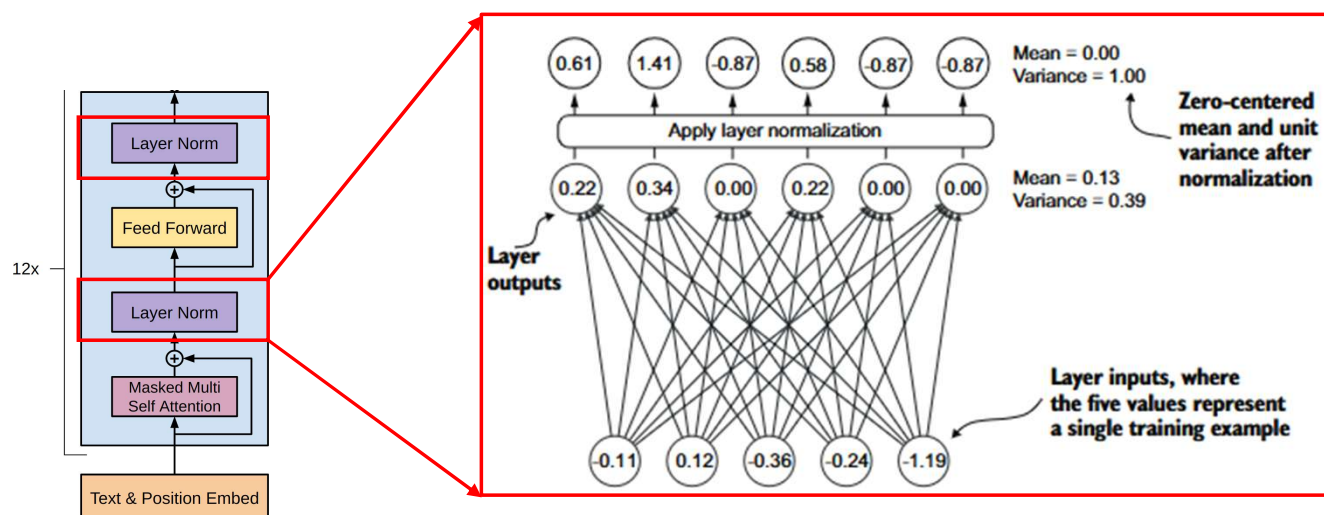
梯度可以原封不动地通过残差连接传回来

Shortcut connection adds input values to the outputs of layer 1

The shortcut connections help with maintaining relatively large gradient values even in early layers

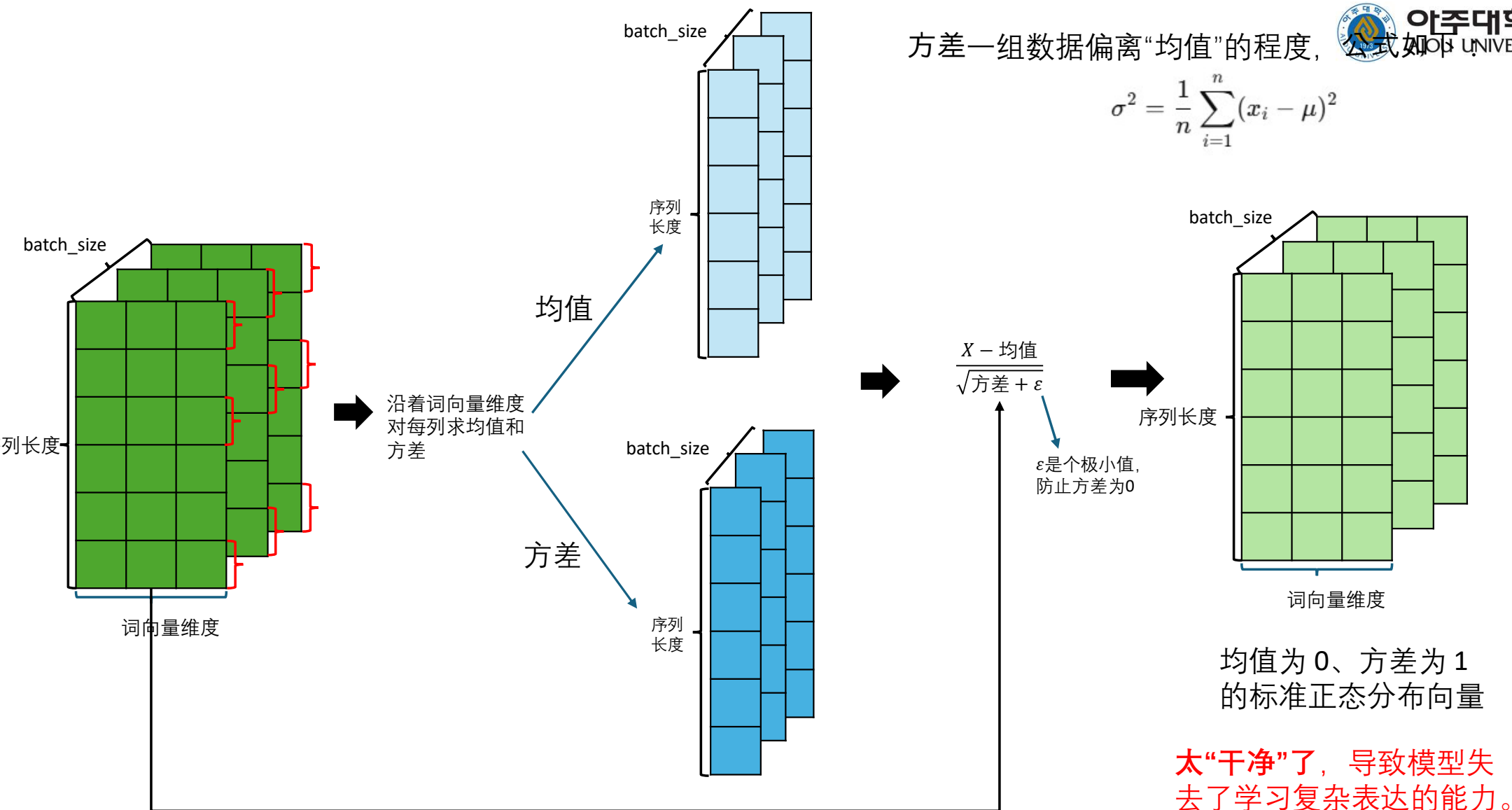
层归一化(Layer Normalization)

- 调整神经网络层的激活（输出），使其均值为 0，方差为 1
- 它通过对每一个输入样本在最后一个维度（通常是特征维度）上做标准化，使激活值的分布更稳定，从而提升训练效果。
- 在 GPT-2 和现代 Transformer 架构中，层归一化通常应用于多头注意模块之前和之后



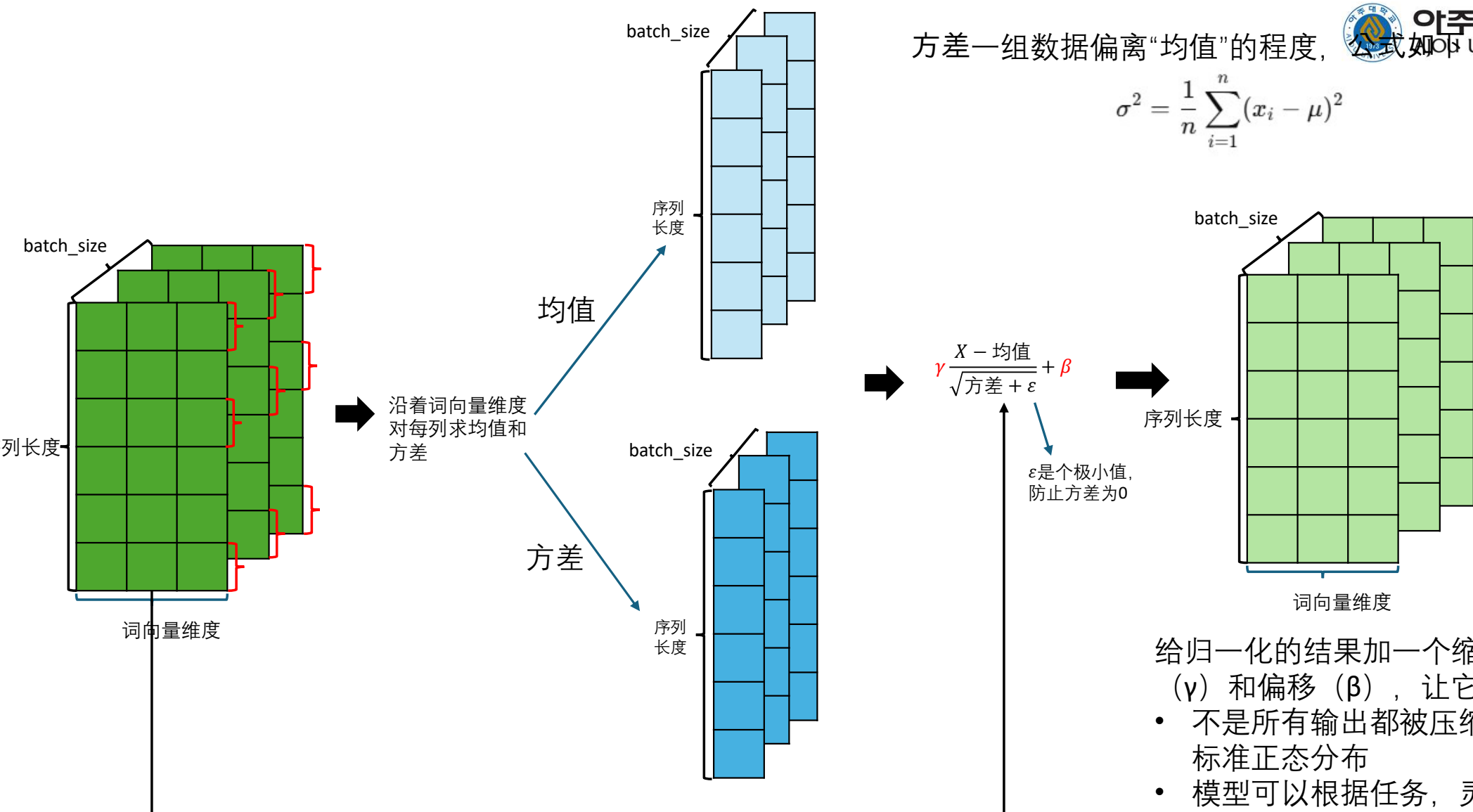
方差一组数据偏离“均值”的程度，

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$



方差一组数据偏离“均值”的程度，

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$



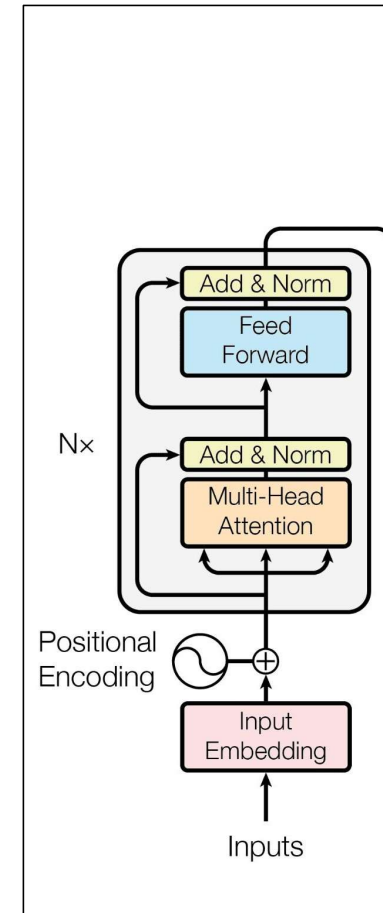
给归一化的结果加一个缩放 (γ) 和偏移 (β), 让它:

- 不是所有输出都被压缩成标准正态分布
- 模型可以根据任务, 灵活地调整激活分布

编码器

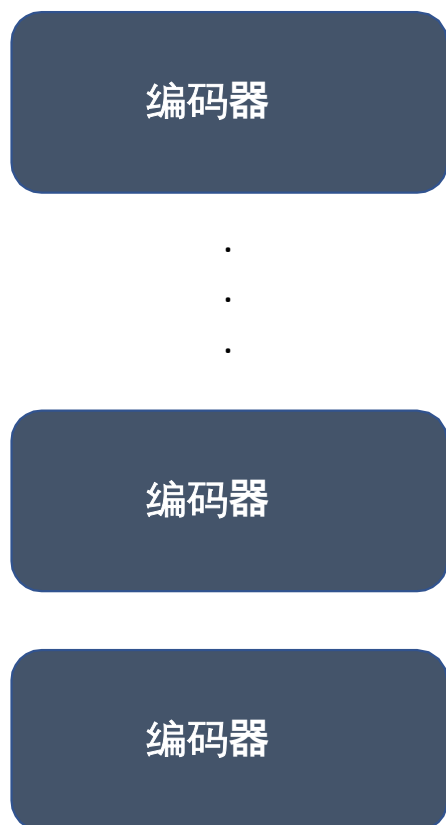
编码器

编码器



编码器

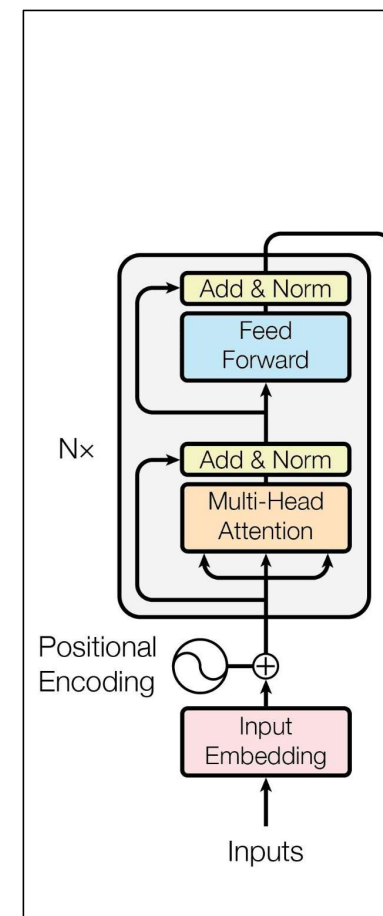
编码器



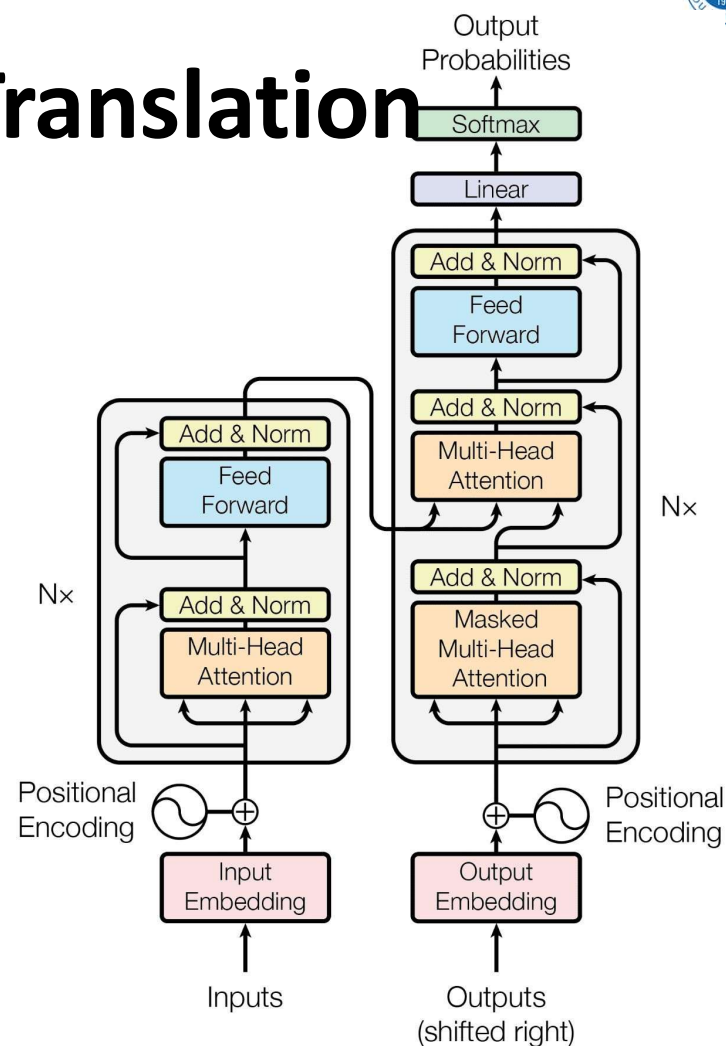
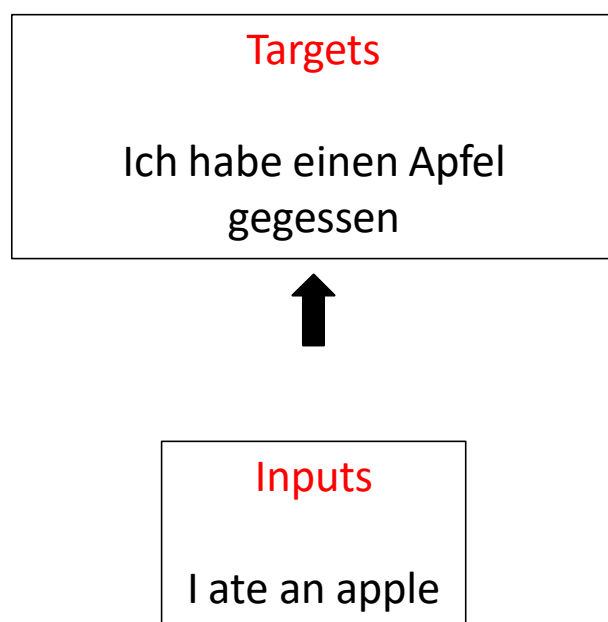
第i+1的编码器的输入



第i个编码器的输出



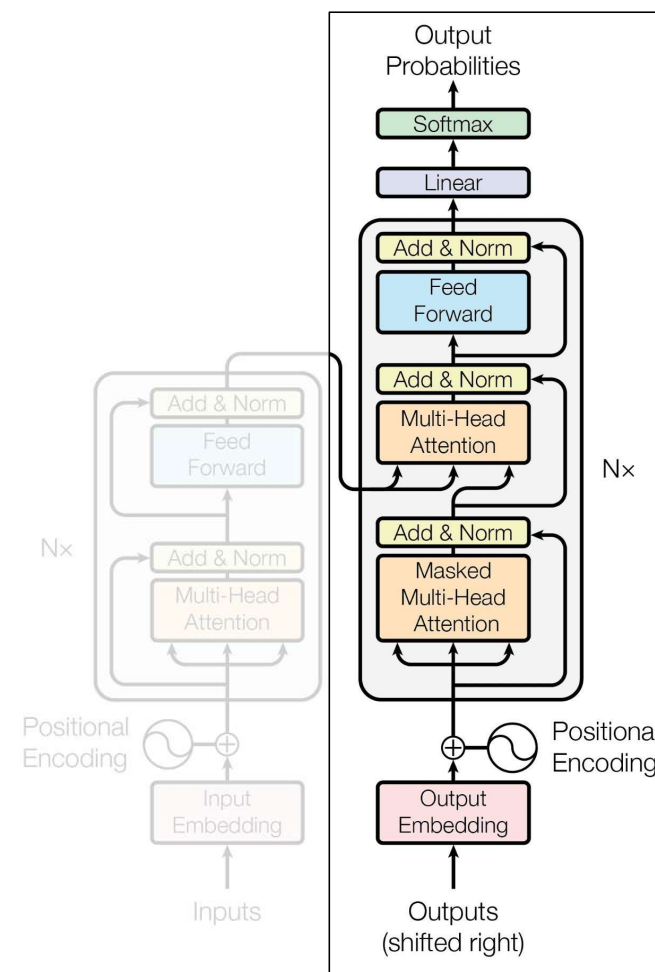
Machine Translation

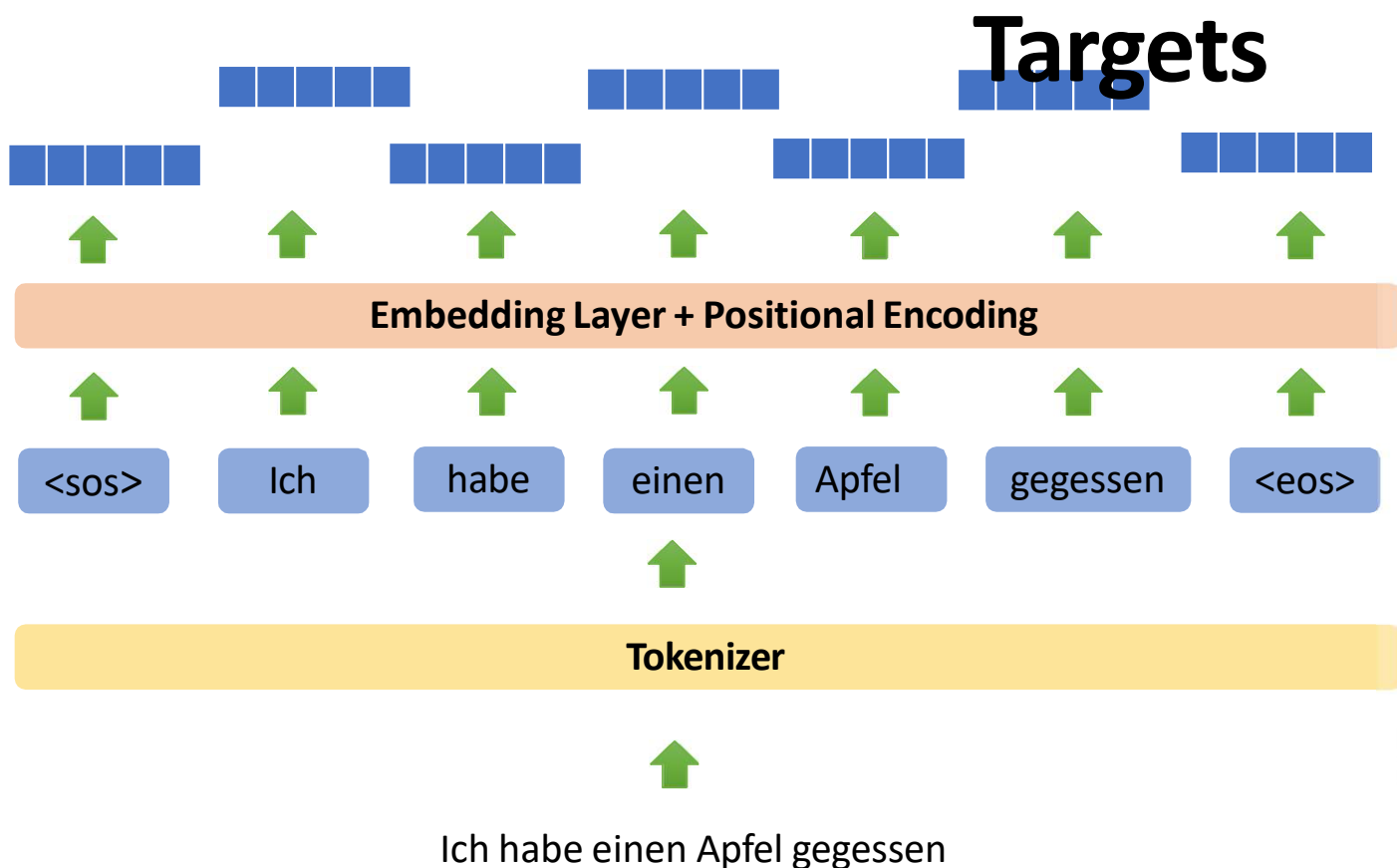


Targets

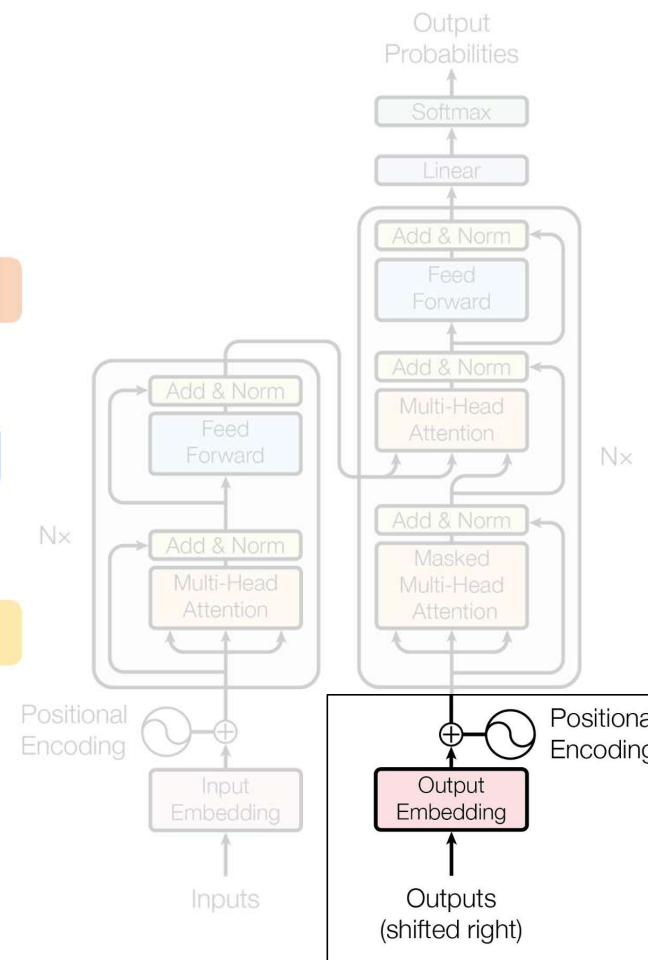
Targets

Ich habe einen Apfel
gegessen

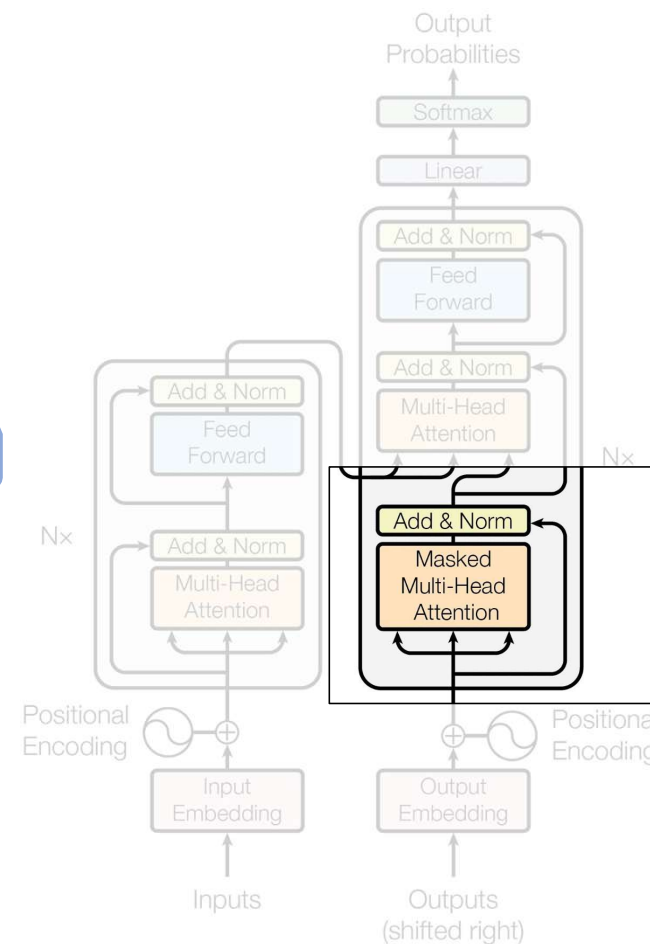
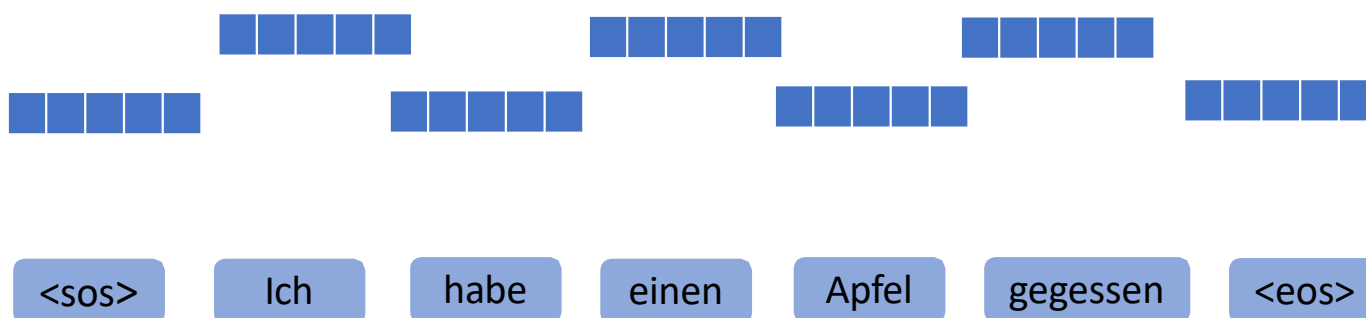




Generate Target Embeddings

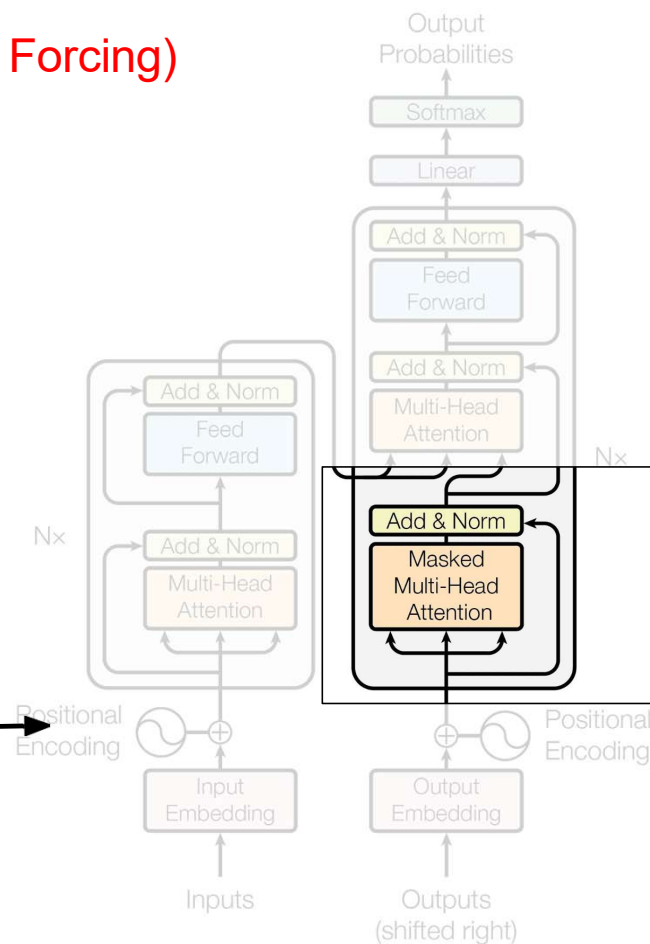
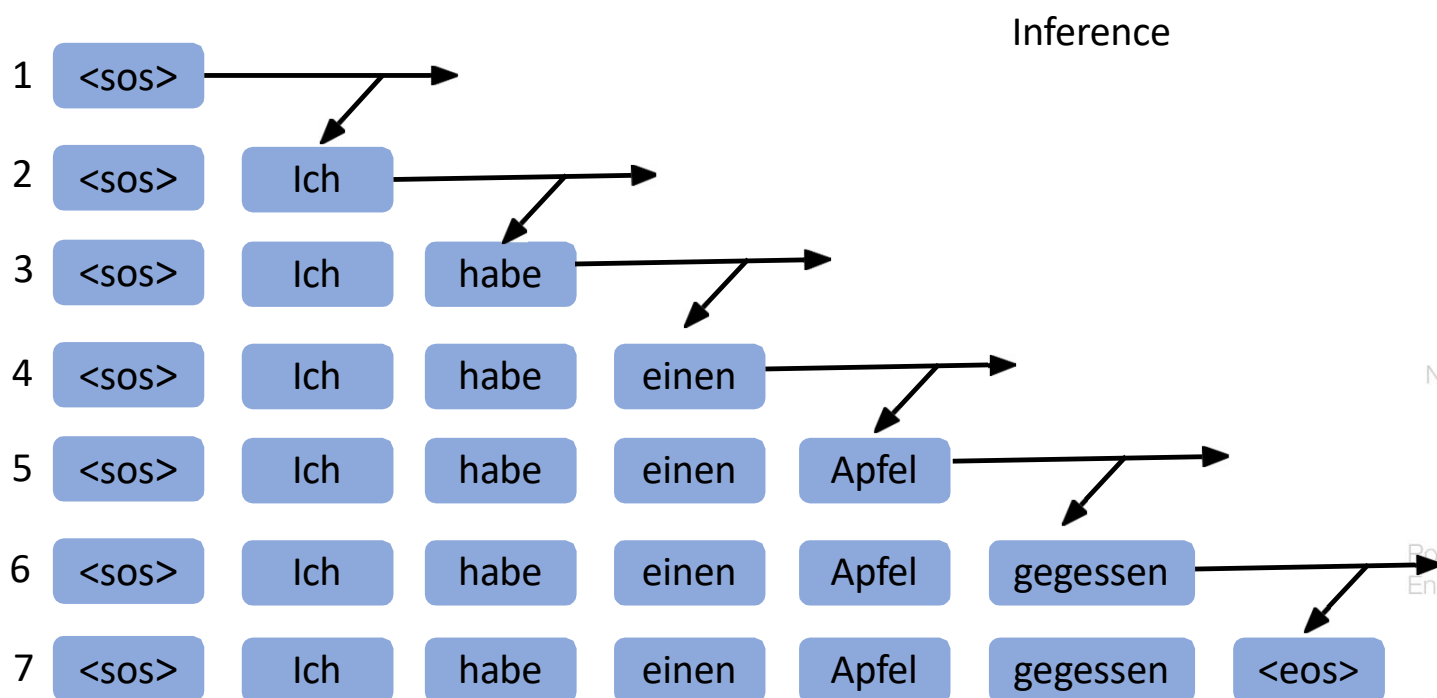


掩码多头注意力



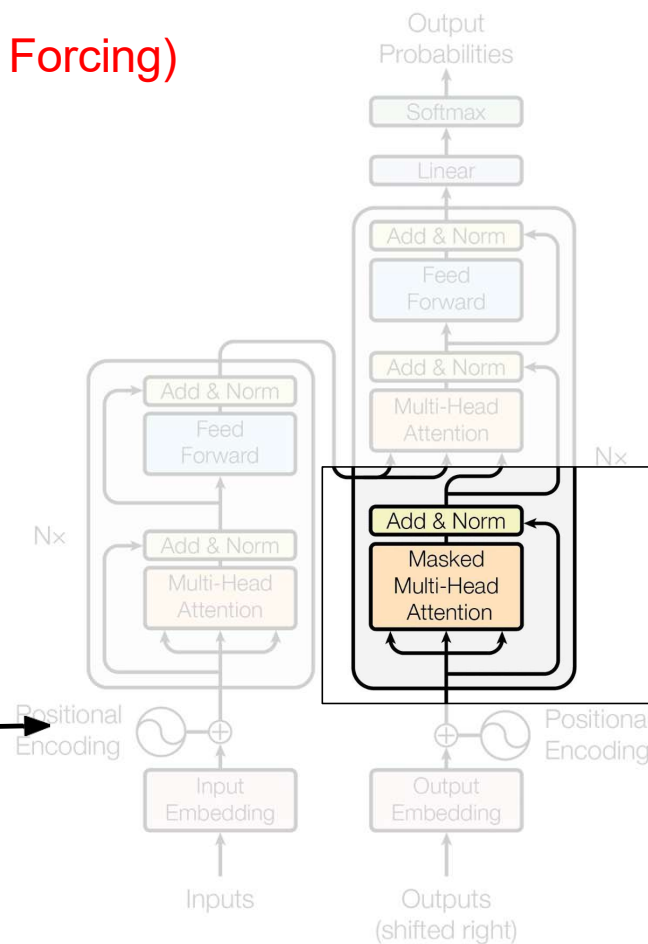
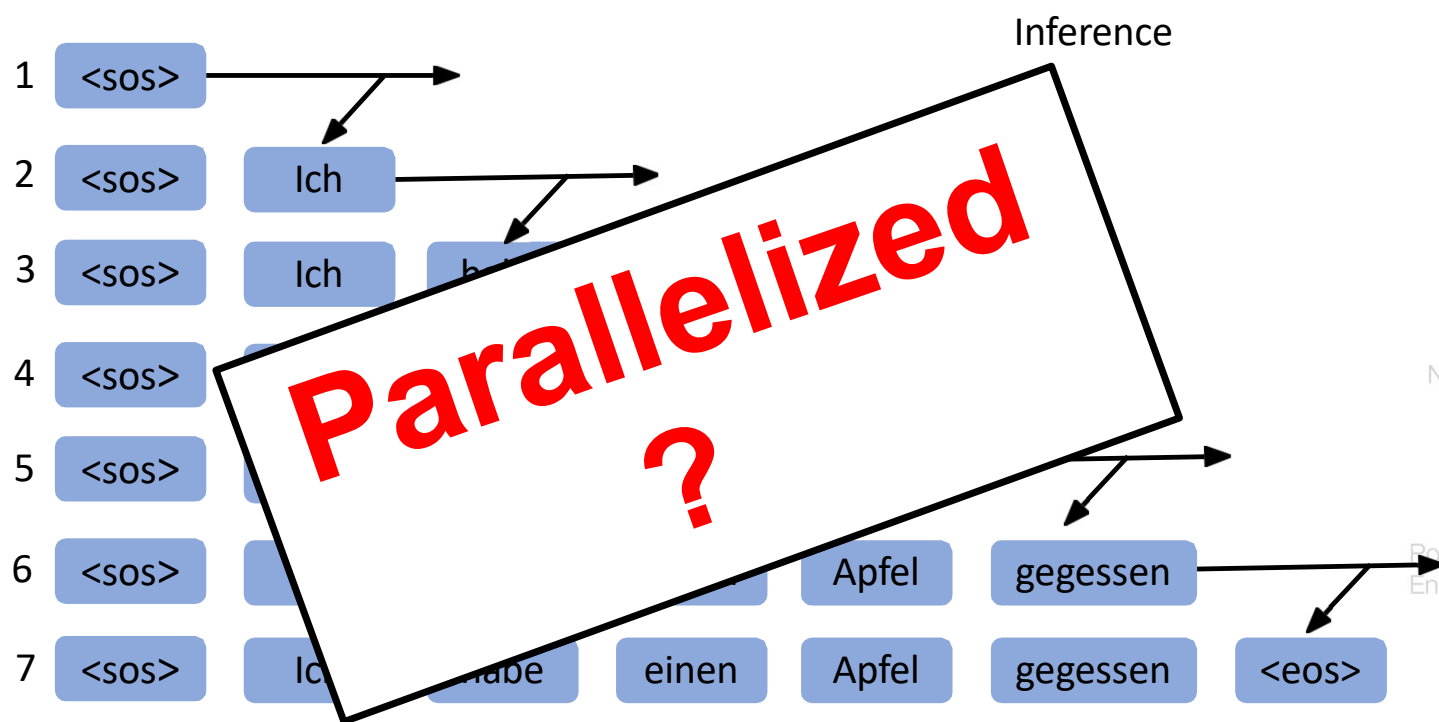
掩码多头注意力

Decoding step by step (using Teacher Forcing)



掩码多头注意力

Decoding step by step (using Teacher Forcing)

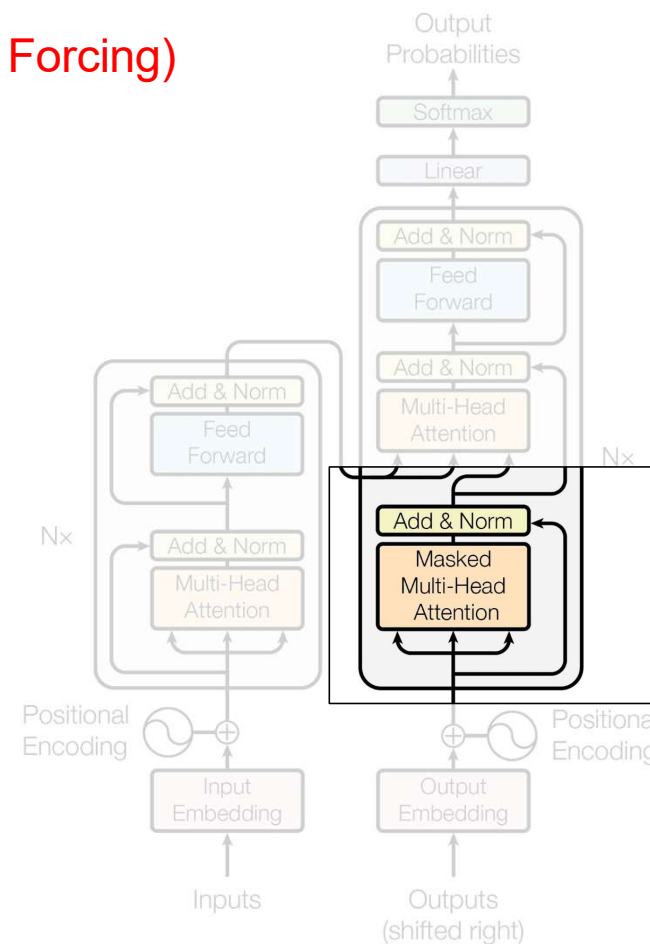


掩码多头注意力

Decoding step by step (using Teacher Forcing)

Training

< sos > Ich habe einen Apfel gegessen < eos >



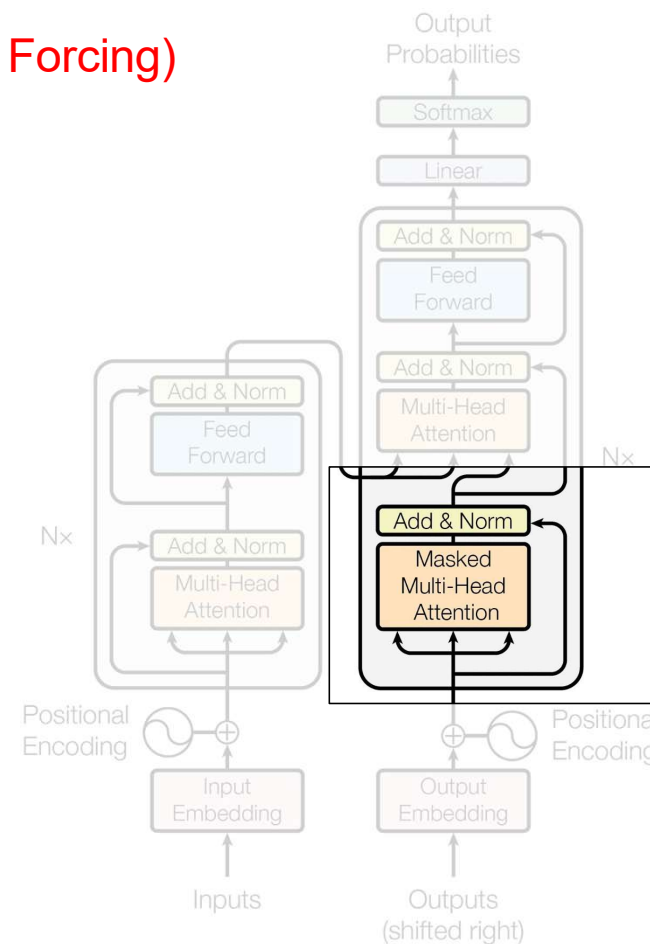
掩码多头注意力

Decoding step by step (using Teacher Forcing)

Training

<sos> Ich habe einen Apfel gegessen <eos>

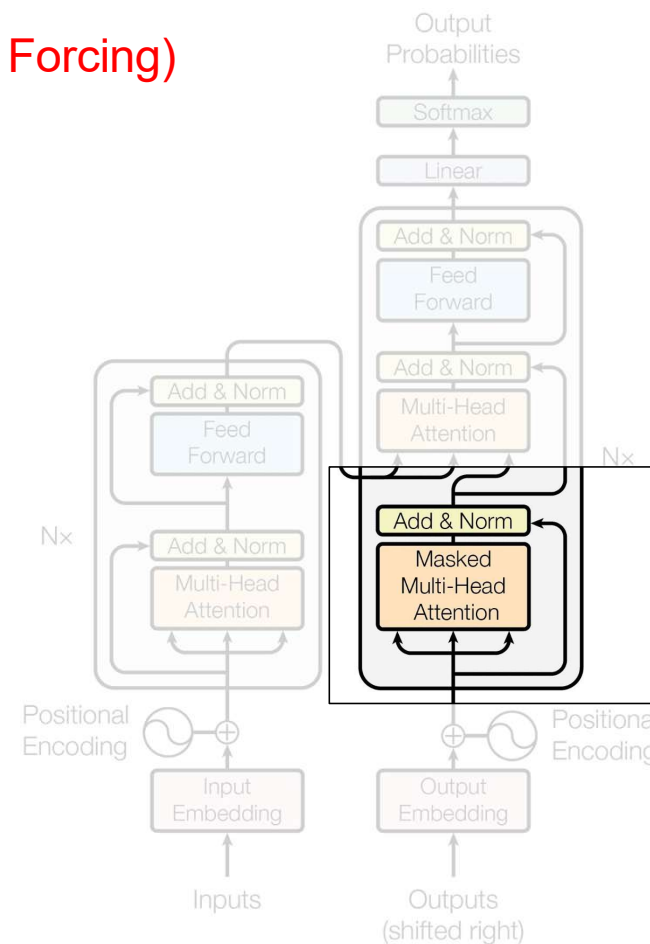
第 t 个时间步的输出只能看 t 之前, 也就是 $0:t-1$ 的内容



掩码多头注意力

Decoding step by step (using Teacher Forcing)

1	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
2	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
3	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
4	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
5	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
6	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
7	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>

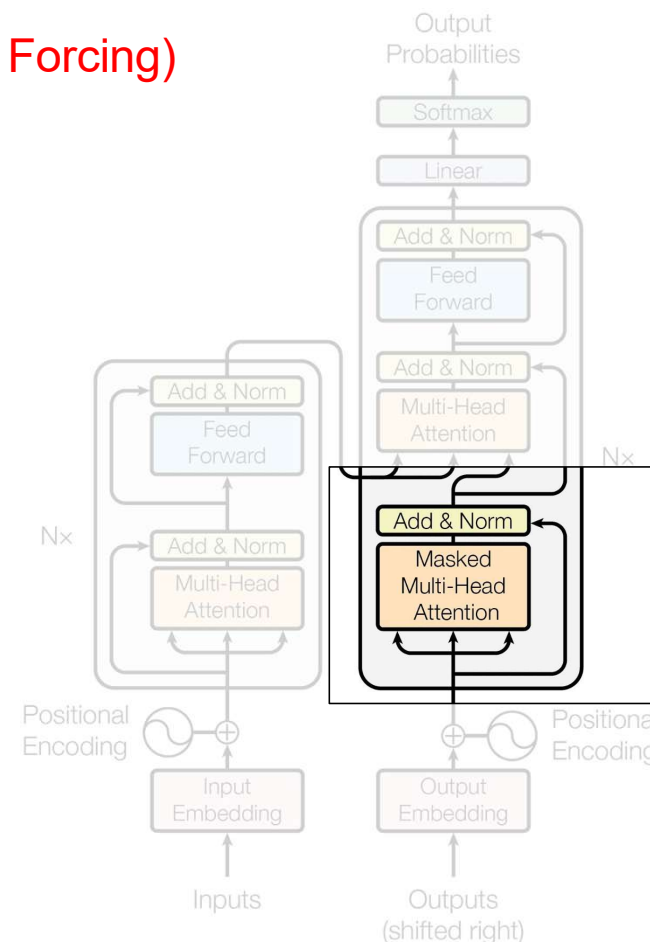


掩码多头注意力

Decoding step by step (using Teacher Forcing)

1	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
2	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
3	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
4	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
5	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
6	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
7	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>

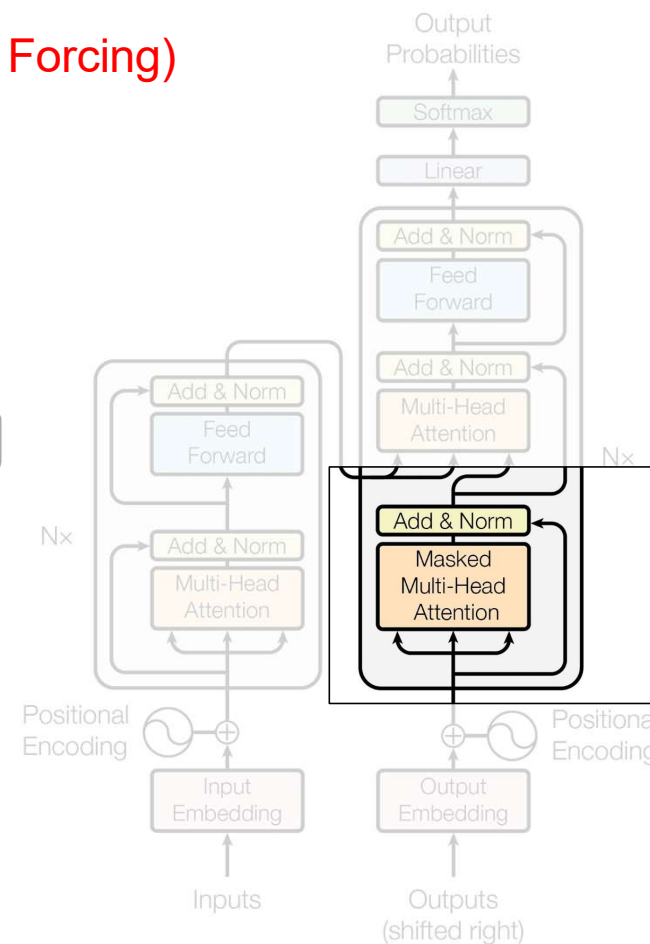
如何遮盖当前时间步后的token?



掩码多头注意力

Decoding step by step (using Teacher Forcing)

1	<sos>	- ∞	- ∞	- ∞	- ∞	- ∞
2	<sos>	Ich	- ∞	- ∞	- ∞	- ∞
3	<sos>	Ich	habe	- ∞	- ∞	- ∞
4	<sos>	Ich	habe	einen	- ∞	- ∞
5	<sos>	Ich	habe	einen	Apfel	- ∞
6	<sos>	Ich	habe	einen	Apfel	gegessen
7	<sos>	Ich	habe	einen	Apfel	gegessen
						<eos>

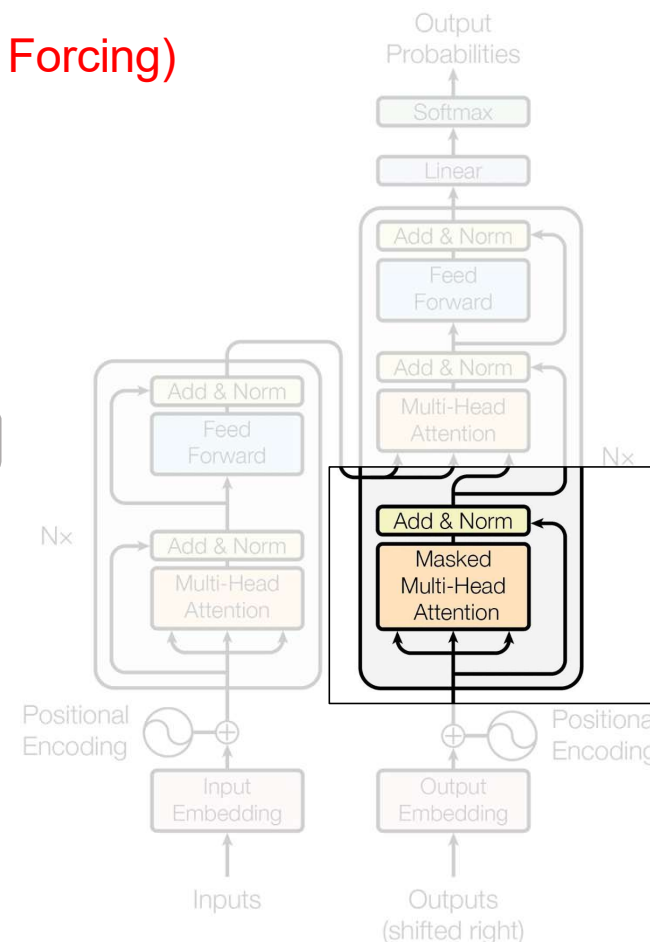


掩码多头注意力

Decoding step by step (using Teacher Forcing)

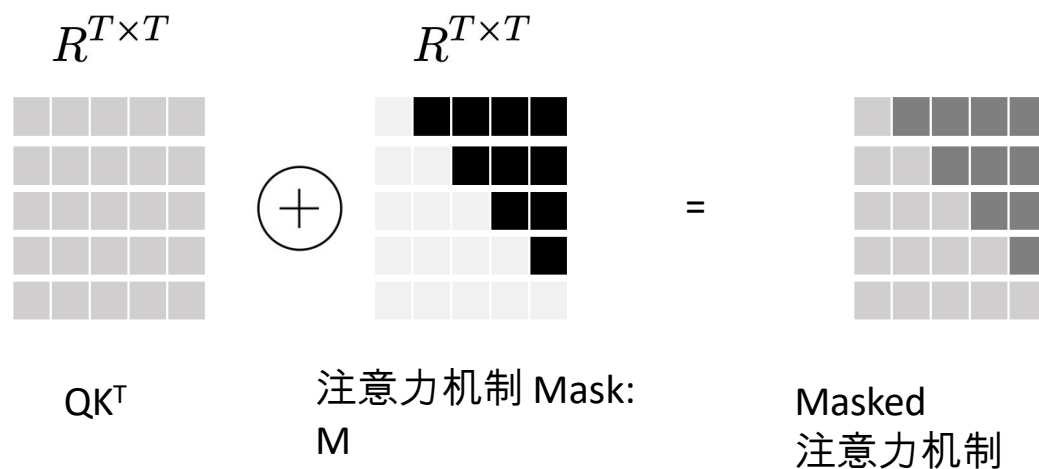
1	<sos>	- ∞	- ∞	- ∞	- ∞	- ∞
2	<sos>	Ich	- ∞	- ∞	- ∞	- ∞
3	<sos>	Ich	habe	- ∞	- ∞	- ∞
4	<sos>	Ich	habe	einen	- ∞	- ∞
5	<sos>	Ich	habe	einen	Apfel	- ∞
6	<sos>	Ich	habe	einen	Apfel	gegessen
7	<sos>	Ich	habe	einen	Apfel	gegessen

Softmax - ∞ -> 0

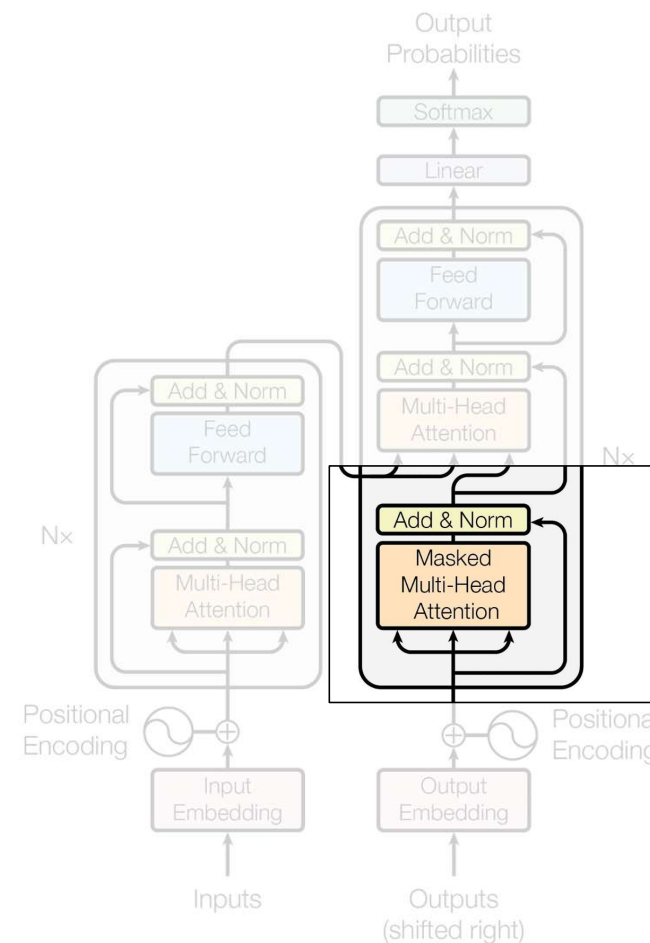


掩码多头注意力

掩码多头注意力

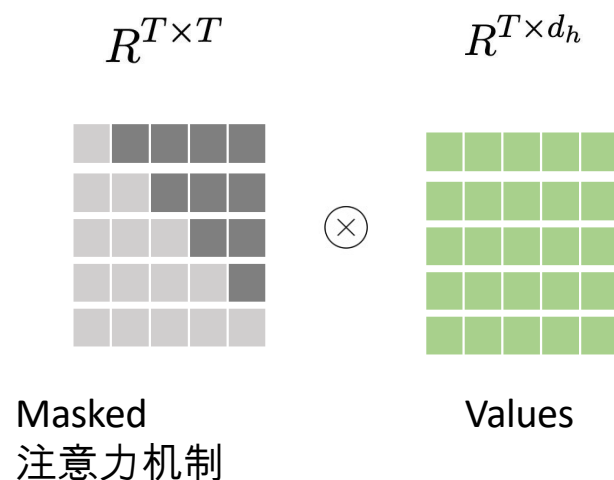


掩码多头注意力 : Z'

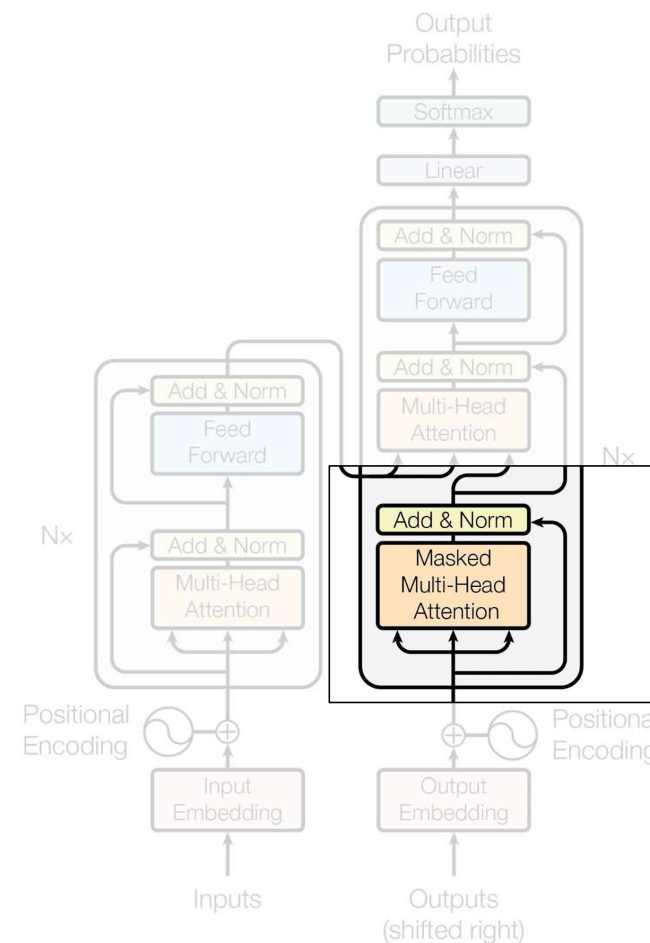


掩码多头注意力

掩码多头注意力



掩码多头注意力 : Z'

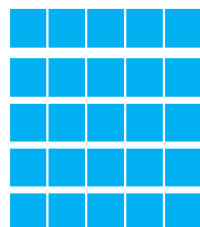


编码器-解码器注意力机制

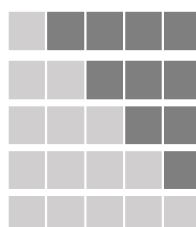
编码器-解码器注意力机制？



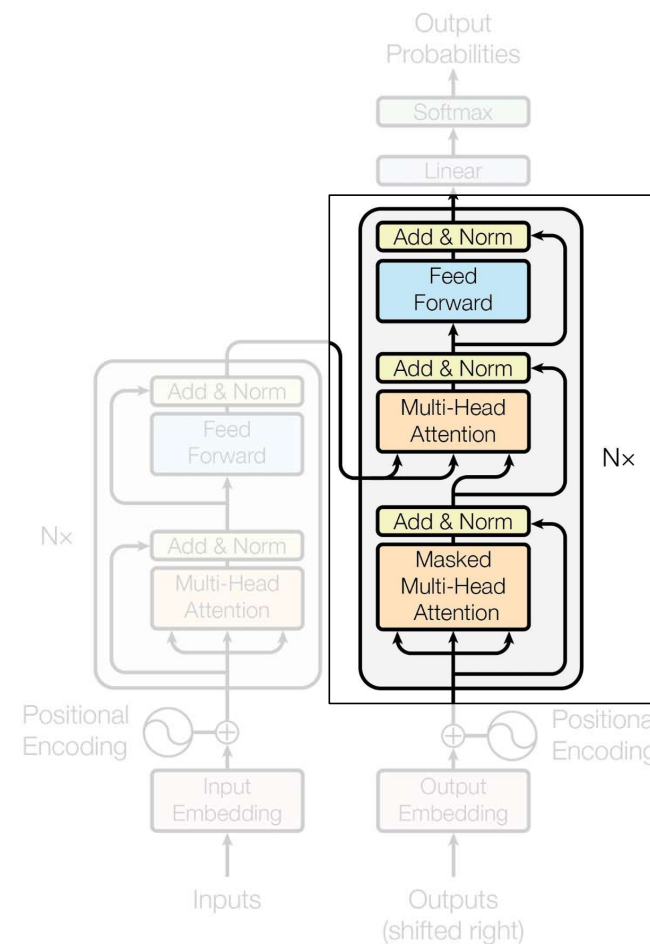
Add & Norm



Input

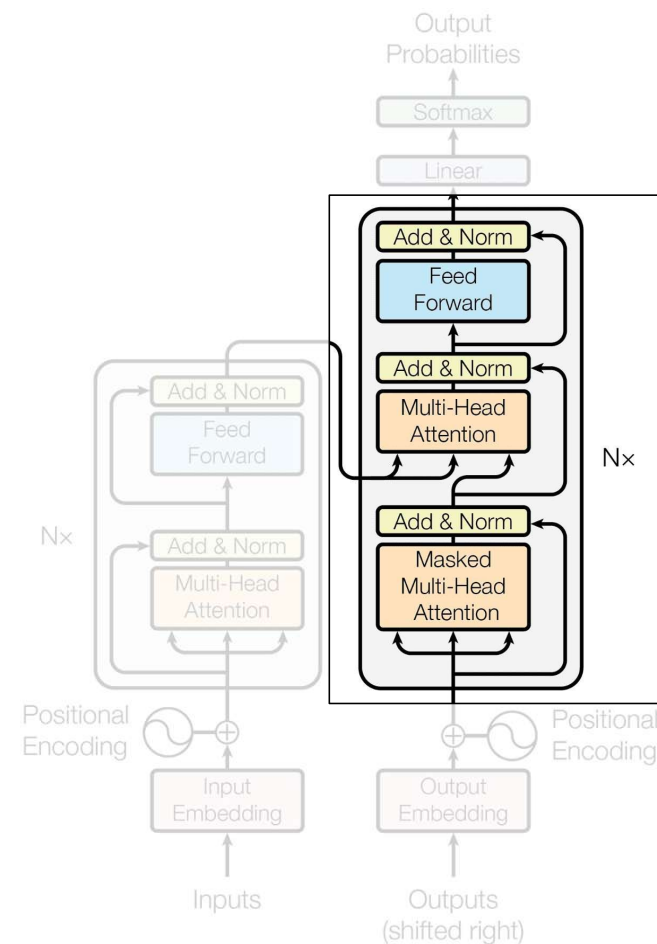


Norm(Z')



编码器-解码器注意力机制

编码器-解码器注意力机制？



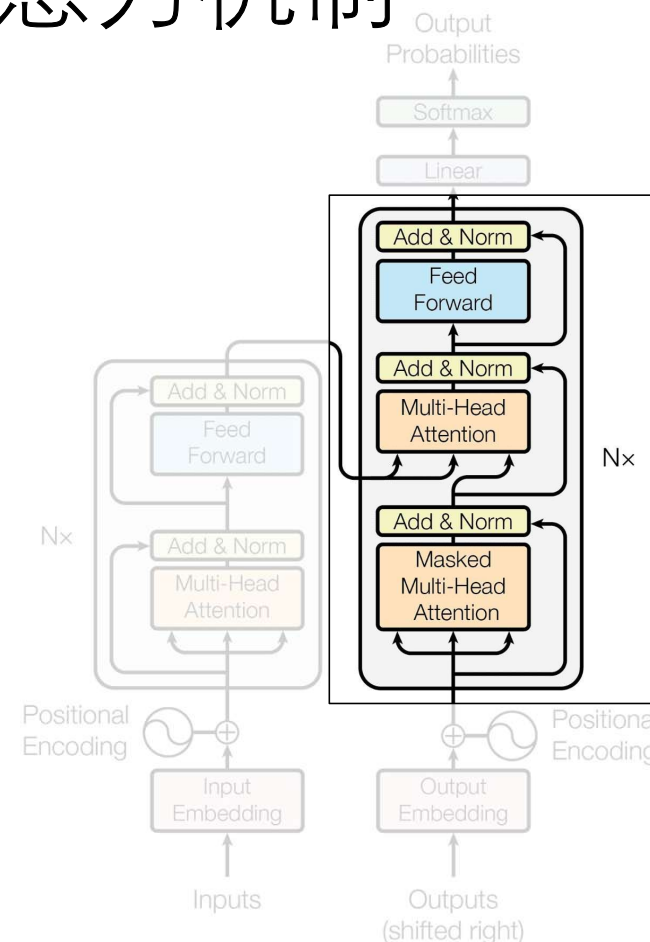
编码器-解码器注意力机制

编码器 自注意力

1. Queries from 编码器 Inputs
2. Keys from 编码器 Inputs
3. Values from 编码器 Inputs

解码器 Masked 自注意力

1. Queries from 解码器 Inputs
2. Keys from 解码器 Inputs
3. Values from 解码器 Inputs



编码器-解码器注意力机制

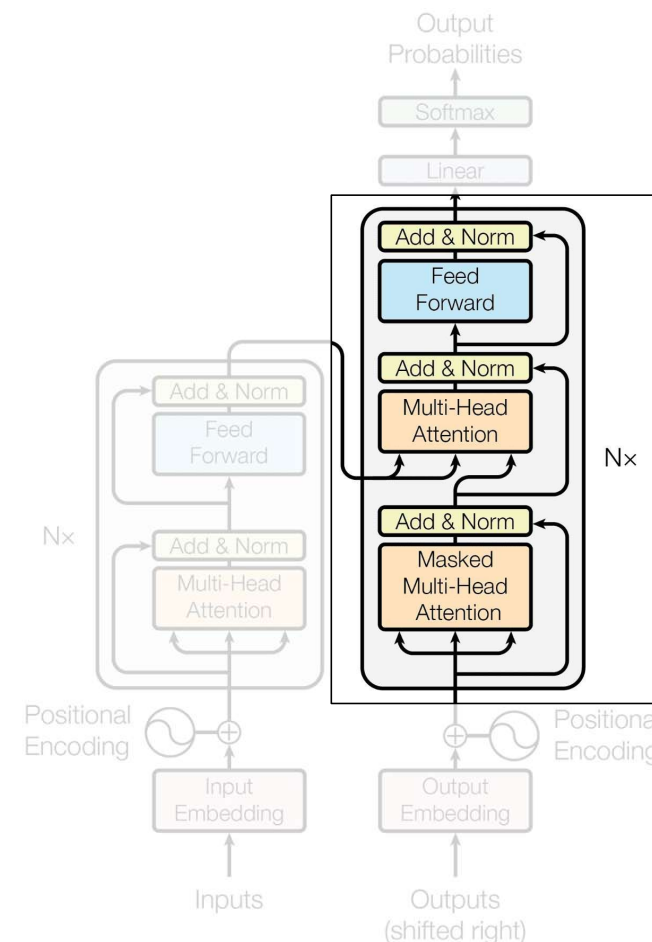
编码器

Keys from 编码器 Outputs
Values from 编码器 Outputs

解码器

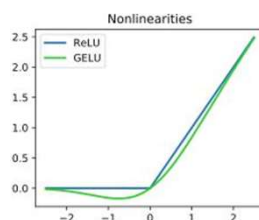
Queries from 解码器 Inputs

NOTE: Every 解码器 block receives the same FINAL 编码器 output

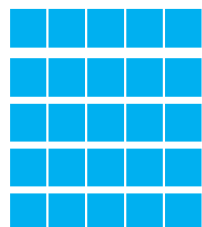


编码器-解码器注意力机制

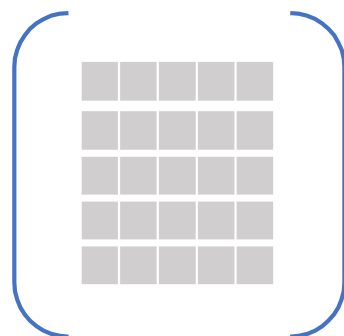
- Non Linearity
- Complex Relationships
- Learn from each other



Feed Forward

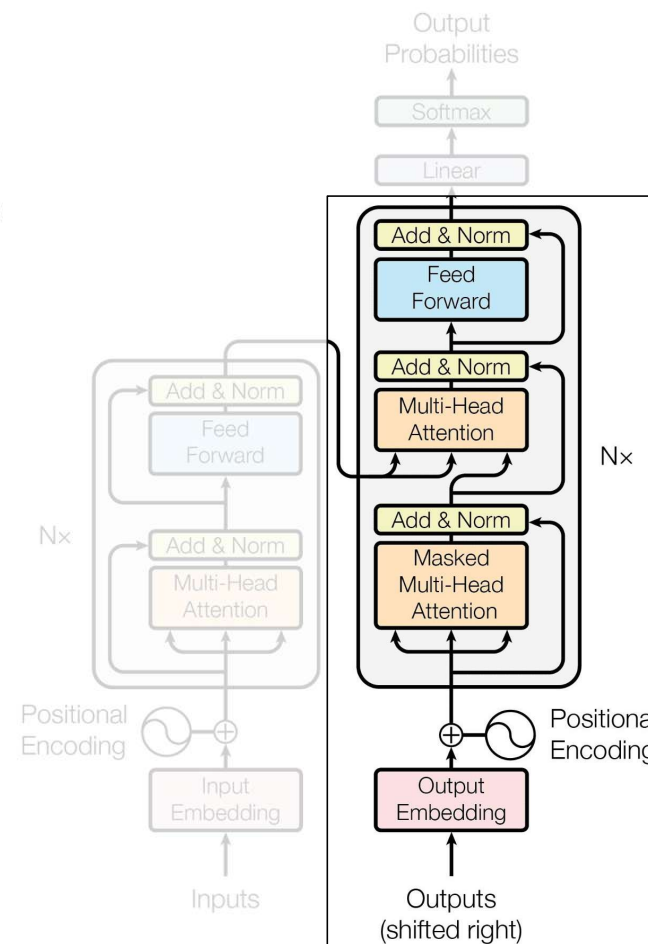


Residuals



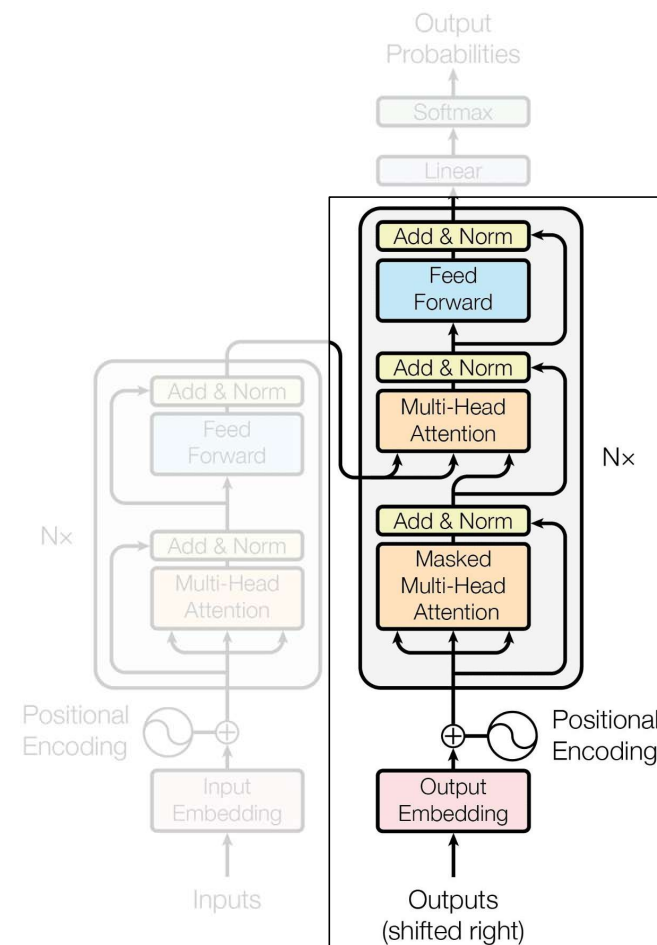
Norm(Z'')

Add n Norm 解码器 Self Attn



解码器

解码器



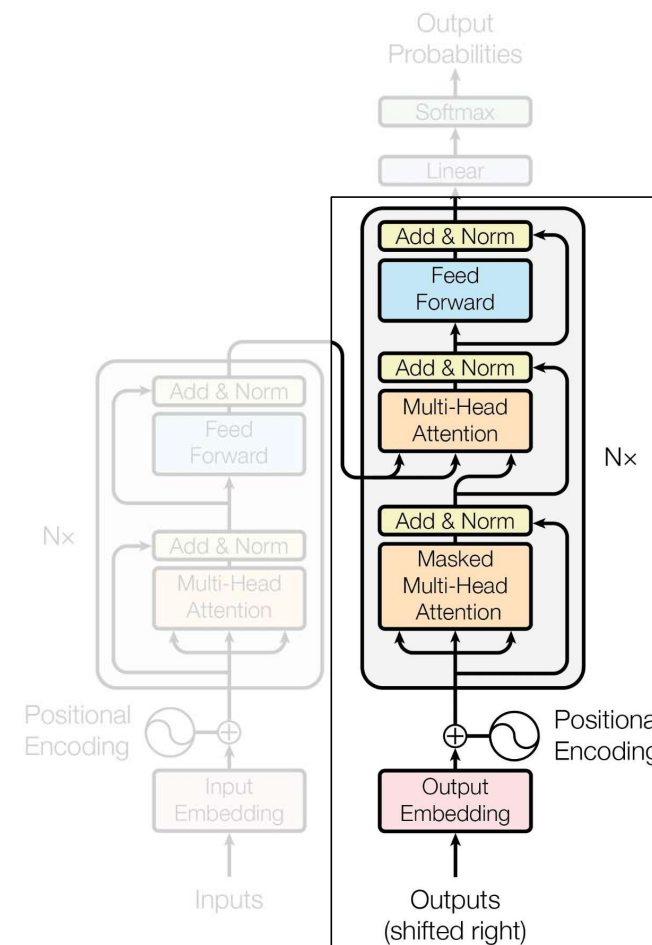
解码器



$$R^{T_d \times d_{model}}$$



解码器 output

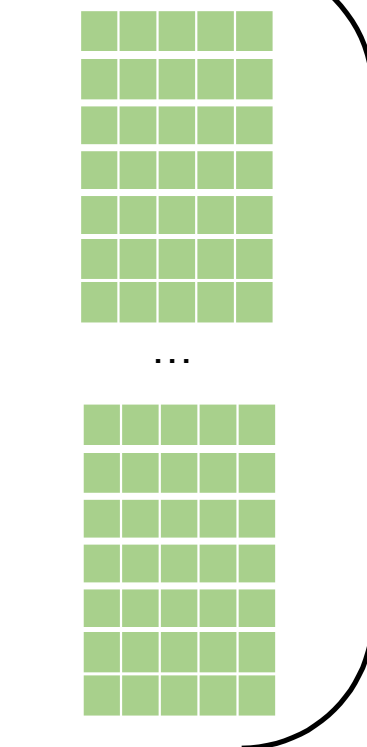


Line

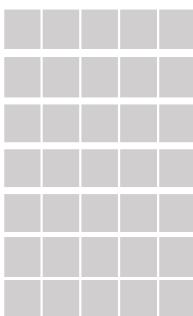
ar

Linear weights are often tied with input embedding matrix

$$R^{V \times d_{model}}$$



$$R^{T_d \times d_{model}}$$

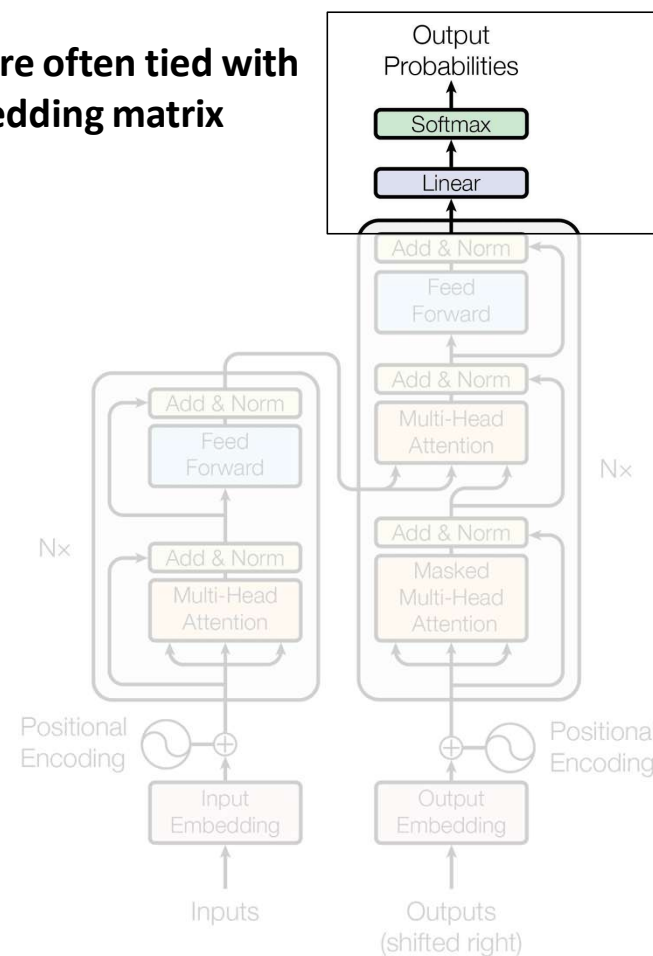


softmax

Final 解码器 Output

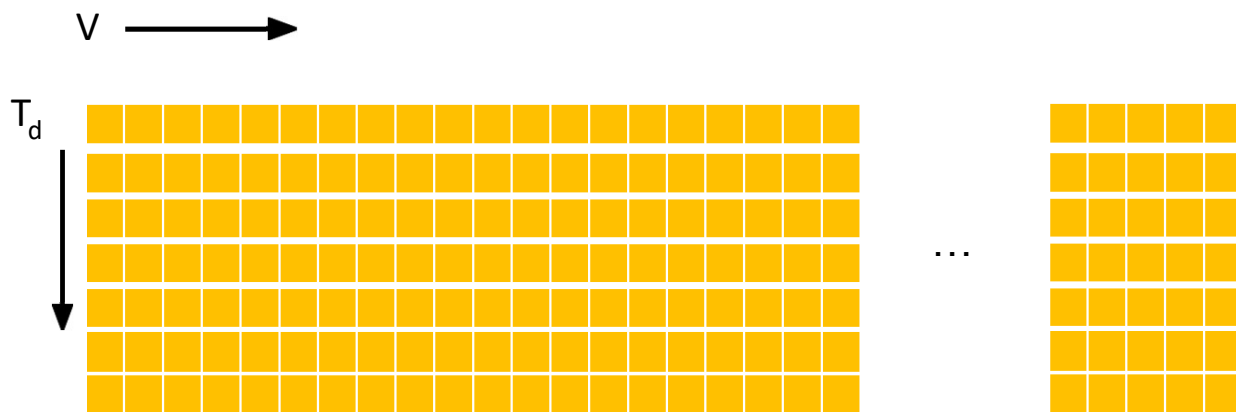
$$R^{T_d \times V}$$

Linear

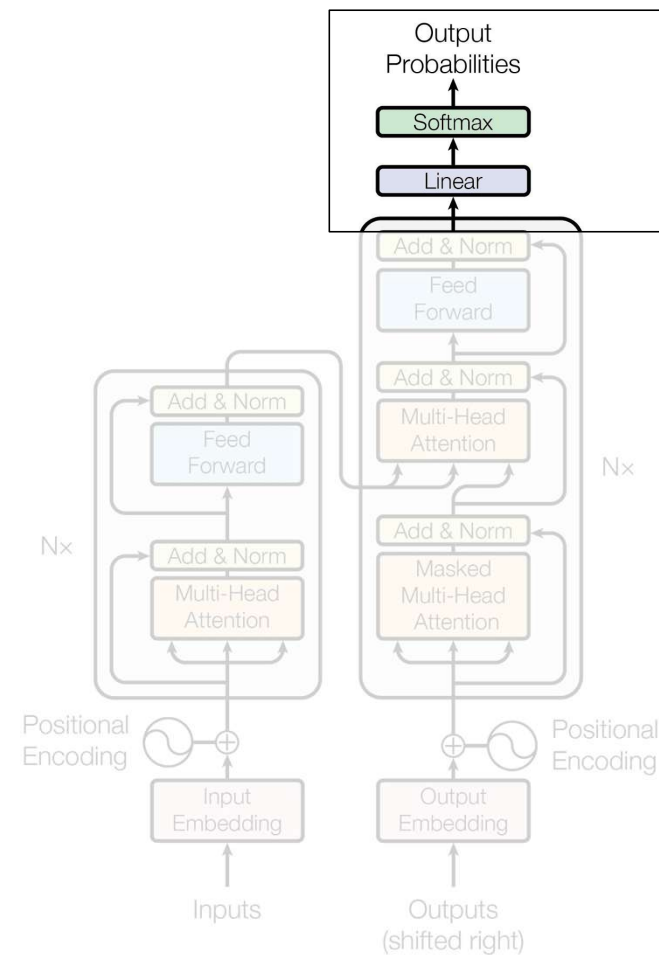


Softmax

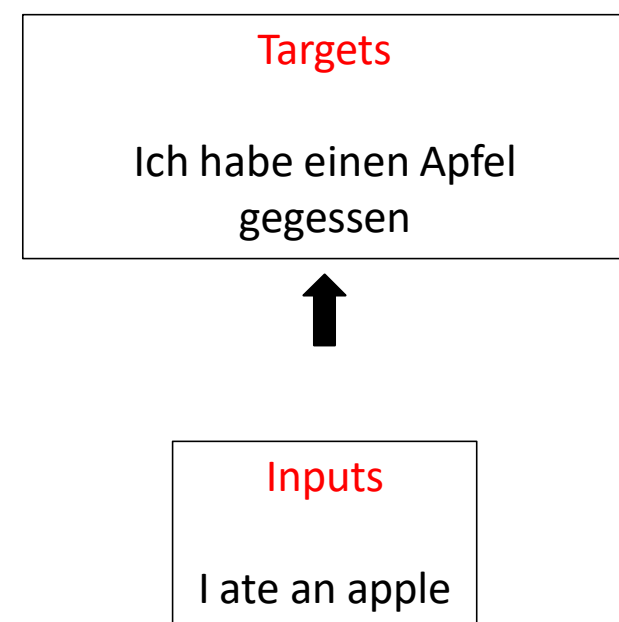
Output Probabilities



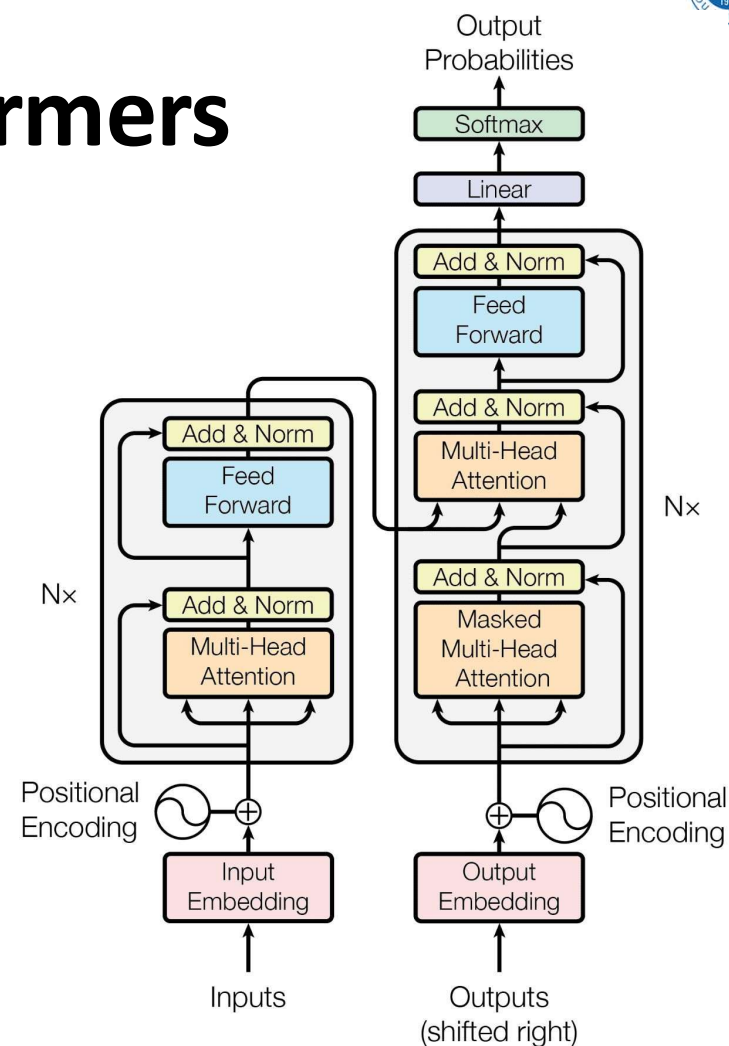
$$R^{T_d \times V}$$



Transformers



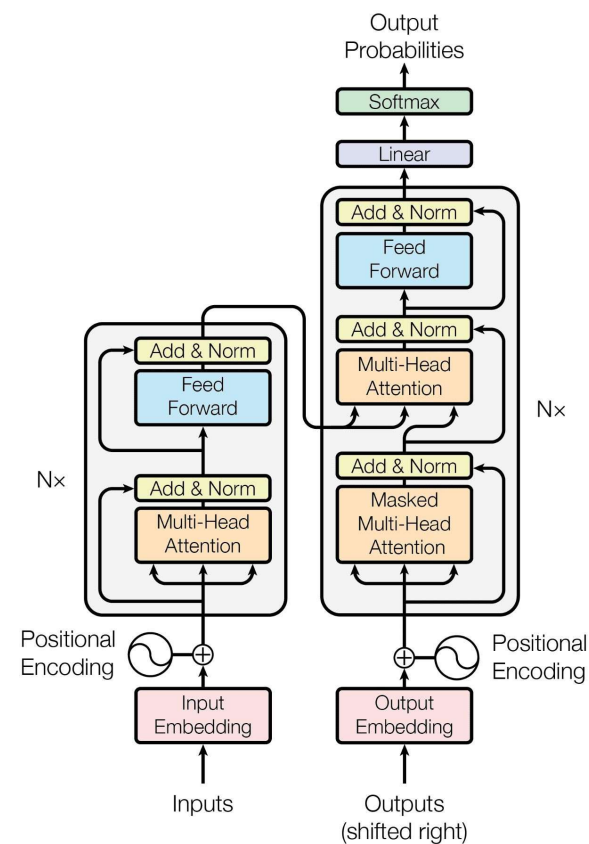
Machine Translation

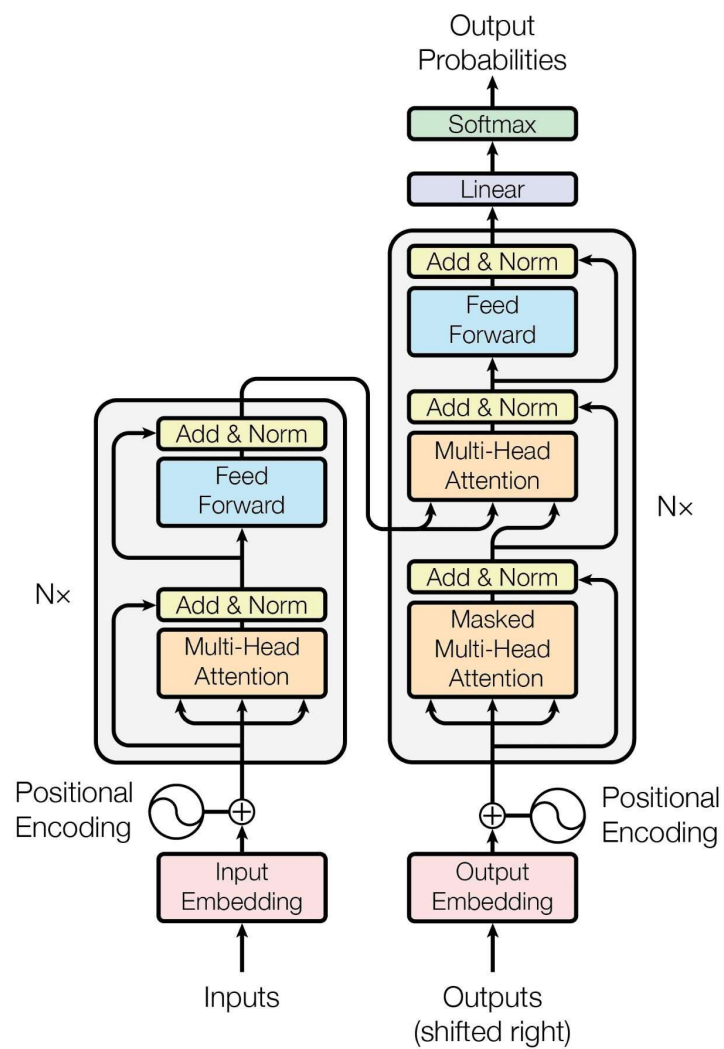


Transformers

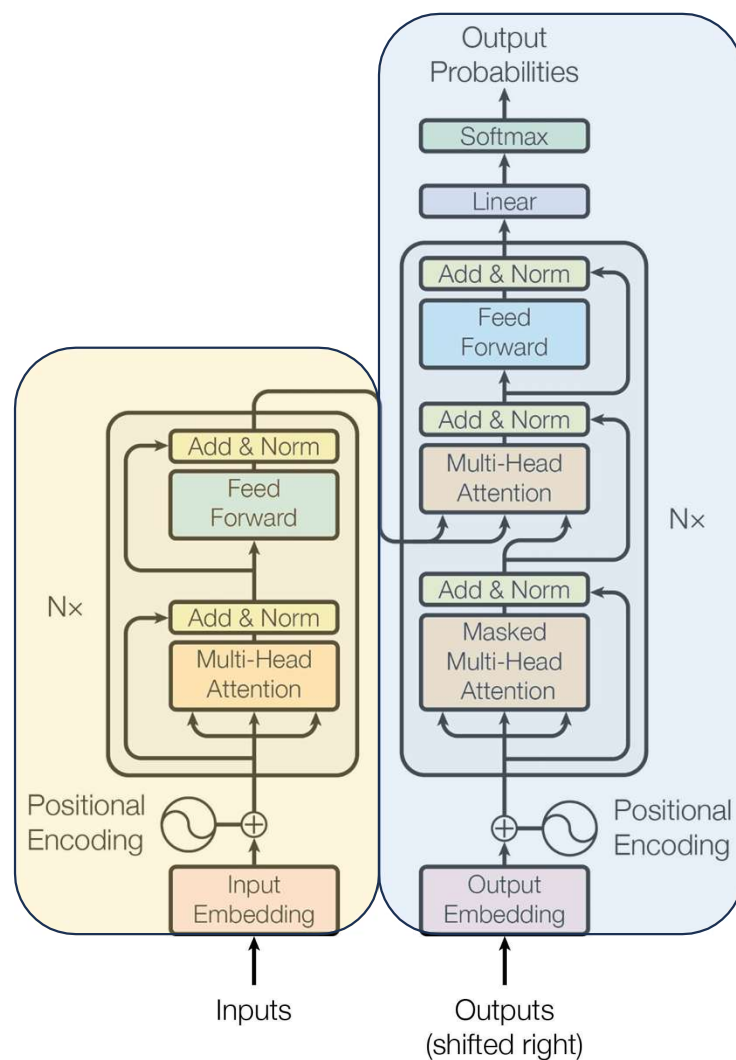
- ✓ Tokenization
- ✓ 输入嵌入
- ✓ 位置编码
- ✓ Query, Key, & Value
- ✓ 注意力机制
- ✓ 自注意力
- ✓ 多头注意力
- ✓ Feed Forward
- ✓ Add & Norm
- ✓ 编码器s

- ✓ Masked 注意力机制
- ✓ 编码器-解码器注意力机制
- ✓ Linear
- ✓ Softmax
- ✓ 解码器s
 - 编码器-解码器 Models





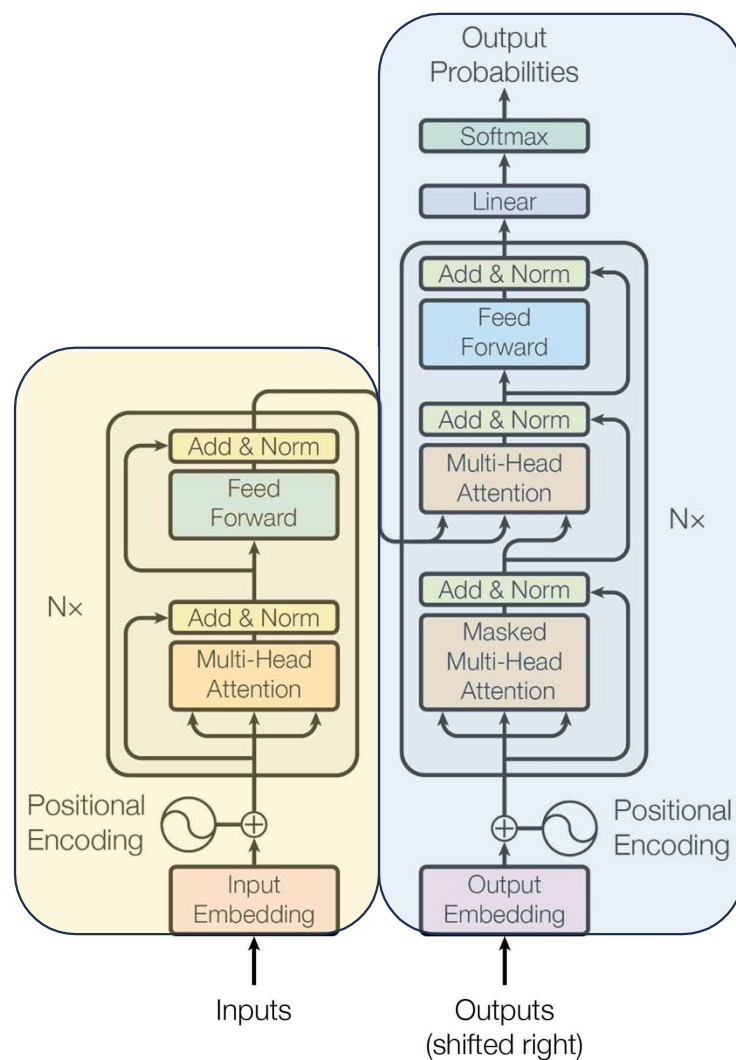
Representation



Generation

Input – input tokens
Output – hidden states

Representation



Input – output tokens and hidden states*

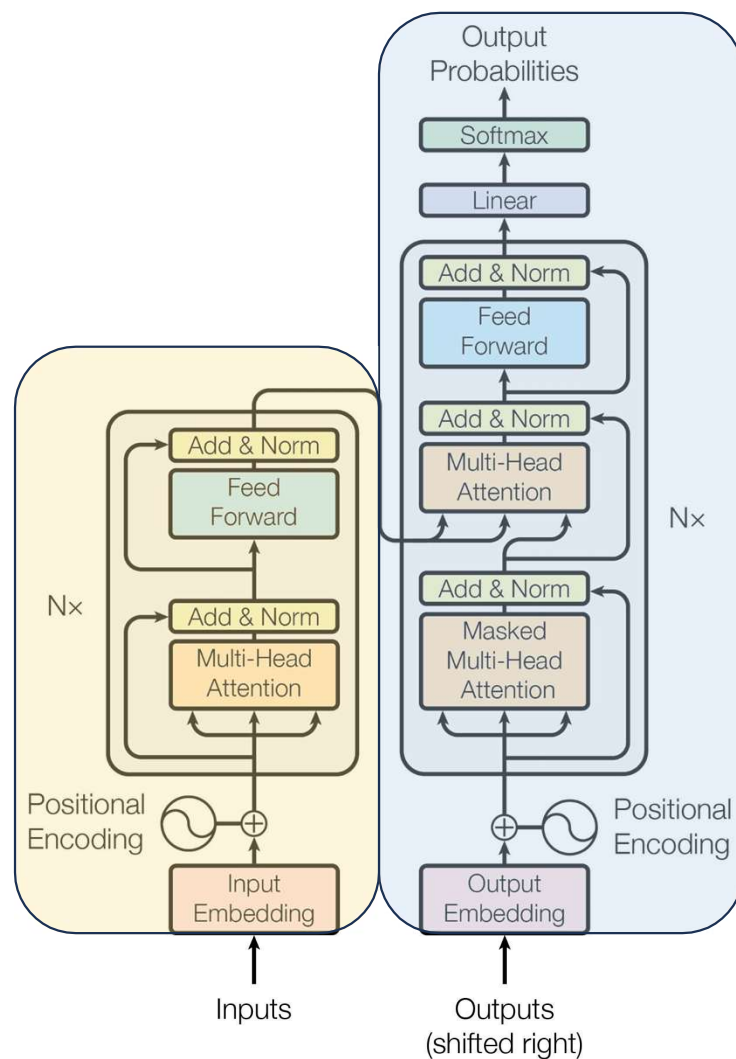
Output – output tokens

Generation

Input – input tokens
Output – hidden states

Model can see all timesteps

Representation



Input – output tokens and hidden states*

Output – output tokens

Model can only see previous timesteps

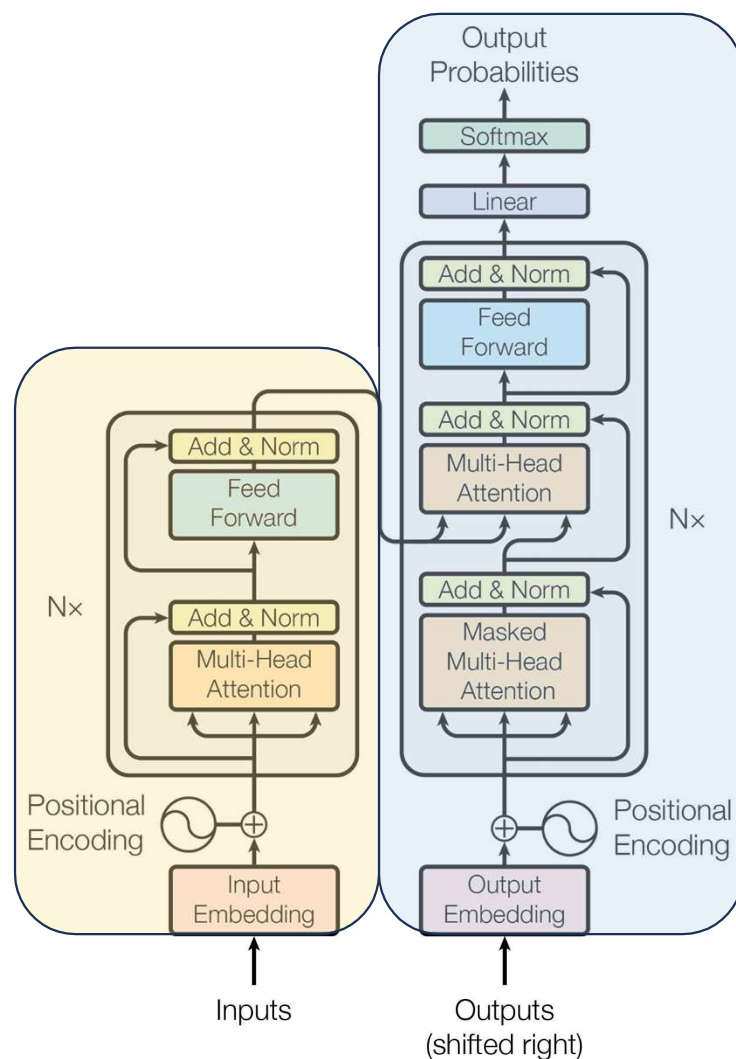
Generation

Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Representation



Input – output tokens and hidden states*

Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Generation

Input – input tokens

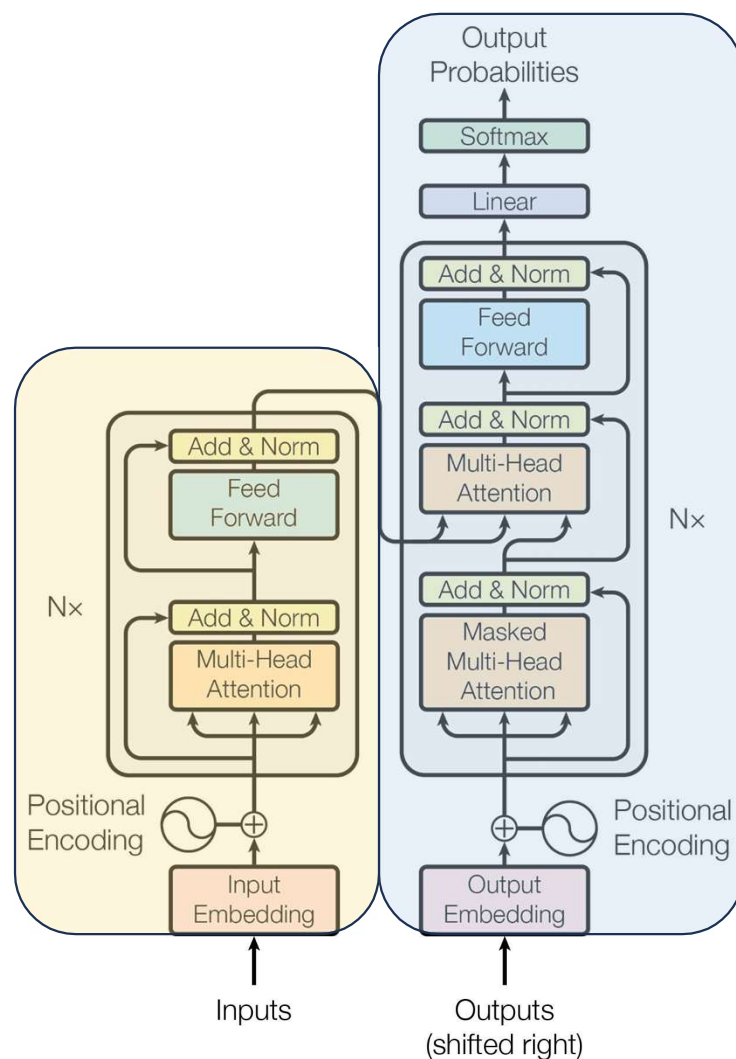
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Can also be adapted to generate tokens by appending a module that maps hidden state dimensionality to vocab size

Representation



Input – output tokens and hidden states*

Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Can also be adapted to generate hidden states by looking before token outputs

Generation

Transformers

- ✓ Tokenization
- ✓ 输入嵌入
- ✓ 位置编码
- ✓ Query, Key, & Value
- ✓ 注意力机制
- ✓ 自注意力
- ✓ 多头注意力
- ✓ Feed Forward
- ✓ Add & Norm
- ✓ 编码器s

- ✓ Masked 注意力机制
- ✓ 编码器-解码器注意力机制
- ✓ Linear
- ✓ Softmax
- ✓ 解码器s
- ✓ 编码器-解码器 Models

